ROS 2 DESIGN · SECURITY COLLECTION

Security and Threat Modeling

Historical reference reader

Juan Carlos Nevarez · Gh0stlyKn1ght

Security review, editorial direction, and print adaptation
github.com/Gh0stlyKn1ght

October 20, 2026

Historical source edition.

The four original articles are reproduced with original authors and dates. Technical claims, scores, and findings remain historical. Read the companion review report before using the instructions. The 27-, 11-, and 14-column matrices are converted into vertical records without dropping fields. Diagram previews are retained; larger views appear in the companion figure atlas.

Original document attribution: Open Source Robotics Foundation, Inc. Design documents: CC BY 3.0 except where otherwise noted. Repository LICENSE: Apache 2.0, reproduced in the appendix. No upstream endorsement is implied.

Contents

1. ROS 2 DDS-Security integration
2. ROS 2 Access Control Policies
3. ROS 2 Security Enclaves
4. ROS 2 Robotic Systems Threat Model
5. Supporting security passages
6. Sources and license notices

PDF bookmarks provide section navigation. The separate inventory lists every section and source line range. All source hyperlinks point to the reviewed revision.

Historical source: ROS 2 DDS-Security integration

Kyle Fazzari

Written 2019-07; declared modified 2020-07. Pinned original source. Original OSRF design-document attribution and CC BY 3.0 notice retained. Presentation adapted for this reader; historical instructions and findings have not been modernized.

ROS 2 DDS-Security integration

Robotics is full of experimentation: evaluating different hardware and software, pushing ahead with what works, and culling what doesn't. ROS was designed to be flexible to enable this experimentation; to allow existing components to easily be combined with new ones or swapped with others. In ROS 1, this flexibility was valued above all else, at the cost of security. By virtue of being designed on top of DDS, ROS 2 is able to retain that flexibility while obtaining the ability to be secured by properly utilizing the DDS-Security specification. This article describes how ROS 2 integrates with DDS-Security.

Authors: Kyle Fazzari

Date Written: 2019-07

Last Modified: 2020-07

DDS-Security overview

The DDS-Security specification expands upon the DDS specification, adding security enhancements by defining a Service Plugin Interface (SPI) architecture, a set of builtin implementations of the SPIs, and the security model enforced by the SPIs. Specifically, there are five SPIs defined:

- **Authentication**: Verify the identity of a given domain participant.
- **Access control**: Enforce restrictions on the DDS-related operations that can be performed by an authenticated domain participant.
- **Cryptographic**: Handle all required encryption, signing, and hashing operations.
- **Logging**: Provide the ability to audit DDS-Security-related events.
- **Data tagging**: Provide the ability to add tags to data samples.

ROS 2's security features currently utilize only the first three. This is due to the fact that neither **Logging** nor **Data Tagging** are required in order to be compliant with the DDS-Security spec (see section 2.3), and thus not all DDS implementations support them. Let's delve a little further into those first three plugins.

Authentication

The **Authentication** plugin (see section 8.3 of the DDS-Security spec) is central to the entire SPI architecture, as it provides the concept of a confirmed identity without which further enforcement would be impossible (e.g. it would be awfully hard to make sure a given ROS identity could only access specific topics if it was impossible to securely determine which identity it was).

The SPI architecture allows for a number of potential authentication schemes, but ROS 2 uses the builtin authentication plugin (called "DDS:Auth:PKI-DH", see section 9.3 of the DDS-Security spec), which uses the proven Public Key Infrastructure (PKI). It requires a public and private key per domain participant, as well as an x.509 certificate that binds the participant's public key to a specific name. Each x.509 certificate must be signed by (or have a signature chain to) a specific Certificate Authority (CA) that the plugin is configured to trust.

The rationale for using the builtin plugin as opposed to anything else is twofold:

1. It's the only approach described in detail by the spec.
2. It's mandatory for all compliant DDS implementations to interoperably support it (see section 2.3 of the DDS-Security spec), which makes the ROS 2 security features work across vendors with minimal effort.

Access control

The **Access control** plugin (see section 8.4 of the DDS-Security spec) deals with defining and enforcing restrictions on the DDS-related capabilities of a given domain participant. For example, it allows a user to restrict a particular participant to a specific DDS domain, or only allow the participant to read from or write to specific DDS topics, etc.

Again the SPI architecture allows for some flexibility in how the plugins accomplish this task, but ROS 2 uses the builtin access control plugin (called "DDS:Access:Permission", see section 9.4 of the DDS-Security spec), which again uses PKI. It requires two files per domain participant:

- **Governance** file: A signed XML document specifying how the domain should be secured.
- **Permissions** file: A signed XML document containing the permissions of the domain participant, bound to the name of the participant as defined by the authentication plugin (which is done via an x.509 cert, as we just discussed).

Both of these files must be signed by a CA which the plugin is configured to trust. This may be the same CA that the **Authentication** plugin trusts, but that isn't required.

The rationale for using the builtin plugin as opposed to anything else is the same as the **Authentication** plugin:

1. It's the only approach described in detail by the spec.
2. It's mandatory for all compliant DDS implementations to interoperably support it (see section 2.3 of the DDS-Security spec), which makes the ROS 2 security features work across vendors with minimal effort.

Cryptographic

The **Cryptographic** plugin (see section 8.5 of the DDS-Security spec) is where all the cryptography-related operations are handled: encryption, decryption, signing, hashing, etc. Both the **Authentication** and **Access control** plugins utilize the capabilities of the **Cryptographic** plugin in order to verify signatures, etc. This is also where the functionality to encrypt DDS topic communication resides.

While the SPI architecture again allows for a number of possibilities, ROS 2 uses the builtin cryptographic plugin (called "DDS:Crypto:AES-GCM-GMAC", see section 9.5 of the DDS-Security spec), which provides authenticated encryption using Advanced Encryption Standard (AES) in Galois Counter Mode (AES-GCM).

The rationale for using the builtin plugin as opposed to anything else is the same as the other plugins:

1. It's the only approach described in detail by the spec.
2. It's mandatory for all compliant DDS implementations to interoperably support it (see section 2.3 of the DDS-Security spec), which makes the ROS 2 security features work across vendors with minimal effort.

DDS-Security integration with ROS 2: SROS 2

Now that we have established some shared understanding of how security is supported in DDS, let's discuss how that support is exposed in ROS 2. By default, none of the security features of DDS are enabled in ROS 2. The set of features and tools in ROS 2 that are used to enable them are collectively named "Secure ROS 2" (SROS 2).

Features in the ROS client library (RCL)

Most of the user-facing runtime support for SROS 2 is contained within the ROS Client Library. Once its requirements are satisfied it takes care of configuring the middleware support for each supported DDS implementation. RCL includes the following features for SROS 2:

- Support for security files for each domain participant.
- Support for both permissive and strict enforcement of security.
- Support for a master "on/off" switch for all SROS 2 features.

Let's discuss each of these in turn.

Security files for each domain participant

As stated earlier, the DDS-Security plugins require a set of security files (e.g. keys, governance and permissions files, etc.) per domain participant. Domain participants map to a context within process in ROS 2, so each process requires a set of these files. RCL supports being pointed at a directory containing security files in two different ways:

- Directory tree of all security files.
- Manual specification.

Let's delve further into these.

Directory tree of all security files

RCL supports finding security files in one directory that is inside the reserved `enclaves` subfolder, within the root keystore, corresponding to the fully-qualified path of every enclave. For example, for the `/front/camera` enclave, the directory structure would look like:

```
<root>
├── enclaves
│   └── front
│       └── camera
│           ├── cert.pem
│           ├── key.pem
│           ├── ...
└── public
    ├── ...
```

The set of files expected within each enclave instance directory are:

- **identity_ca.cert.pem**: The x.509 certificate of the CA trusted by the **Authentication** plugin (the "Identity" CA).
- **cert.pem**: The x.509 certificate of this enclave instance (signed by the Identity CA).
- **key.pem**: The private key of this enclave instance.
- **permissions_ca.cert.pem**: The x.509 certificate of the CA trusted by the **Access control** plugin (the "Permissions" CA).
- **governance.p7s**: The XML document that specifies to the **Access control** plugin how the domain should be secured (signed by the Permissions CA).
- **permissions.p7s**: The XML document that specifies the permissions of this particular enclave instance to the **Access control** plugin (also signed by the Permissions CA).

This can be specified by setting the `ROS_SECURITY_KEYSTORE` environment variable to point to the root of the keystore directory tree, and then specifying the enclave path using the `--ros-args` runtime argument `-e`, `--enclave`, e.g.:

```
export ROS_SECURITY_KEYSTORE="/home/bob/.ros/sros2_keystore"
ros2 run <package> <executable> --ros-args --enclave="/front/camera"
```

Manual specification

RCL also supports specifying the enclave path for the process that needs to be launched using an overriding environmental variable. This can be done by setting the `ROS_SECURITY_ENCLAVE_OVERRIDE` environment variable to an alternate enclave path within the keystore. Note that this setting takes precedence over `ROS_SECURITY_KEYSTORE` with `--enclave`.

Note that the following two examples load from the same enclave path as demonstrated prior:

```
export ROS_SECURITY_KEYSTORE="/home/bob/.ros/sros2_keystore"
export ROS_SECURITY_ENCLAVE_OVERRIDE="/front/camera"
ros2 run <package> <executable>
```

```
export ROS_SECURITY_KEYSTORE="/home/bob/.ros/sros2_keystore"
export ROS_SECURITY_ENCLAVE_OVERRIDE="/front/camera"
ros2 run <package> <executable> --ros-args --enclave="/spam"
```

Support for both permissive and strict enforcement of security

Participants with the security features enabled will not communicate with participants that don't, but what should RCL do if one tries to launch a participant that has no discernable enclave with keys/permissions/etc.? It has two options:

- **Permissive mode**: Try to find security files, and if they can't be found, launch the participant without enabling any security features. This is the default behavior.
- **Strict mode**: Try to find security files, and if they can't be found, fail to run the participant.

The type of mode desired can be specified by setting the `ROS_SECURITY_STRATEGY` environment variable to "Enforce" (case-sensitive) for strict mode, and anything else for permissive mode.

Support for a master "on/off" switch for all SROS 2 features

In addition to the supported features just discussed, RCL also supports a master shutoff for security features for easy experimentation. If it's turned off (the default), none of the above security features will be enabled.

In order to enable SROS 2, set the `ROS_SECURITY_ENABLE` environment variable to "true" (case-sensitive). To disable, set to any other value.

Features in the SROS 2 CLI

Configuring a ROS 2 system to be secure in RCL involves a lot of new technology (PKI, DDS governance and permissions files and their syntax, etc.). If a user is comfortable with these technologies, the above information should be all that's necessary to properly lock things down. However, the SROS 2 CLI should include a tool `ros2 security` to help those who don't want to set it all up themselves, including the following capabilities:

- Create Identity and Permissions CA.
- Create directory tree containing all security files.
- Create a new identity for a given enclave, generating a keypair and signing its x.509 certificate using the Identity CA.
- Create a governance file that will encrypt all DDS traffic by default.
- Support specifying enclave permissions in familiar ROS terms which are then automatically converted into low-level DDS permissions.
- Support automatically discovering required permissions from a running ROS system.

Historical source: ROS 2 Access Control Policies

Ruffin White, Kyle Fazzari

Written 2019-08; declared modified 2021-06. Pinned original source. Original OSRF design-document attribution and CC BY 3.0 notice retained. Presentation adapted for this reader; historical instructions and findings have not been modernized.

ROS 2 Access Control Policies

This article specifies the policy format used for access control when securing ROS subsystem.

Authors: Ruffin White, Kyle Fazzari

Date Written: 2019-08

Last Modified: 2021-06

SROS 2 introduces several security properties, including encryption, authentication, and authorization. Authorization is obtained by combining the first two properties with a model for access control. Such models are often referred to as access control policies. A policy serves as a high-level abstraction of privileges associated with attributes that may then be transpiled into low-level permissions for individual identities, such as specific ROS nodes within a secure DDS network.

Concepts

Before detailing the SROS 2 policy design of the access control, by which the system constrains the ability of a subject to access an object, it is important to establish a few concepts useful in formalizing the design approach in terms of security. In this setting, a subject may be thought of as a participant on a distributed data-bus (e.g. a ROS node in the computation graph), whereas an object may be an instance of a particular subsystem (e.g. a ROS topic), and access is defined as the capability to act upon that object (e.g. publish or subscribe).

Mandatory Access Control

Mandatory Access Control (MAC) refers to allowing access to an object if and only if rules exist that allow a given subject to access the resource; the term mandatory denotes this requirement that a subject’s access to an object must always be explicitly provisioned. Most importantly, contrary to discretionary access control (DAC), such policies are enforced by a set of authorization rules that cannot be overridden or modified by the subject either accidentally or intentionally. This could also be referred to as “deny by default”.

Principle of Least Privilege

Principle of Least Privilege (PoLP) requires that in a particular abstraction layer, every subject must be able to access only the resources necessary for its legitimate purpose. This is also known as the principle of minimal privilege or the principle of least authority. Applying PoLP not only bolsters system security, but can also ease deployment and help improve system stability.

Privilege Separation

Privilege Separation requires that a subject's access be divided into parts which are limited to the specific privileges they require in order to perform a specific task. This is used to mitigate the potential damage of a security vulnerability. Systems unable to comply with this requirement may consequently fail to satisfy PoLP as well.

Separation of Concerns

Separation of Concerns (SoC) is a design principle for separating a system into distinct sections, so that each section addresses a separate concern. Separate concerns in this case may be how encryption is governed in a system versus how authorization is given to subjects.

Criteria

Design criteria for SROS 2 policies and for selecting the Extensible Markup Language (XML) are discussed here.

Validation

Prior to interpreting any user configuration input, such as an access control policy, data validation should be applied to ensure inputs are compliant and correctly formatted. Incorrect inputs can affect the soundness of most programs or tools, yet guarding against general malformations may itself require meticulous validation logic. Formalizing the description of the data using a precise schema allows for separate programs to assert inputs are compliant without replicating validation logic across implementations. In XML, this is achieved using XSD; allowing the policy markup to be defined by an extendable standard definition rather than a canonical implementation.

Transformation

For usability and generalizability, access control policies can be expressed using domain specific abstractions, such as ROS based subjects and objects. However such abstractions may translate into different representations when applied to lower level transports and policy enforcement points. Formalizing this data transformation using a transformation language allows for separate programs to transpile without replicating conversion logic across implementations. In XML, this is achieved using XSLT; allowing the policy markup to be easily transpiled for various transports by simply swapping or extending transforms.

Composition

When formulating an access control policy, many subjects may share fundamental privileges for basic access. To avoid unnecessary repetition that could exacerbate human errors or other discrepancies, policies should possess sufficient expressive power to remain DRY. In XML, this is achieved using XInclude; allowing the policy markup to easily include or substitute external reference to particular profiles and permissions that repeat across separate policies or profiles.

Schema

The SROS 2 policy schema is defined with XML. The elements and attributes that make up a policy are described below.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    xmlns:xml="http://www.w3.org/XML/1998/namespace"
    elementFormDefault="qualified" attributeFormDefault="unqualified">
    <xs:import namespace="http://www.w3.org/XML/1998/namespace"
               schemaLocation="http://www.w3.org/2001/03/xml.xsd" />


    <xs:element name="policy" type="Policy" />
    <xs:complexType name="Policy">
        <xs:sequence minOccurs="1" maxOccurs="1">
            <xs:element name="enclaves" type="Enclaves" />
        </xs:sequence>
        <xs:attribute name="version" type="xs:string" use="required" fixed="0.2.0"/>
    </xs:complexType>

    <xs:complexType name="Enclaves">
        <xs:sequence minOccurs="1" maxOccurs="unbounded">
            <xs:element name="enclave" type="Enclave" />
        </xs:sequence>
    </xs:complexType>

    <xs:complexType name="Enclave">
        <xs:sequence minOccurs="1" maxOccurs="unbounded">
            <xs:element name="profiles" type="Profiles" />
        </xs:sequence>
        <xs:attribute name="path" type="xs:string" use="required" />
    </xs:complexType>

    <xs:complexType name="Profiles">
        <xs:sequence minOccurs="1" maxOccurs="1">
            <xs:sequence minOccurs="1" maxOccurs="unbounded">
                <xs:element name="profile" type="Profile" />
            </xs:sequence>
            <xs:sequence minOccurs="0" maxOccurs="1">
                <xs:element name="metadata" type="xs:anyType" />
            </xs:sequence>
        </xs:sequence>
        <xs:attribute name="type" type="xs:string" use="optional" />
    </xs:complexType>

    <xs:complexType name="Profile">
        <xs:sequence minOccurs="0" maxOccurs="unbounded">
            <xs:choice minOccurs="1" maxOccurs="1">
                <xs:element name="topics" minOccurs="1" type="TopicExpressionList" />
                <xs:element name="services" minOccurs="1" type="ServicesExpressionList" />
                <xs:element name="actions" minOccurs="1" type="ActionsExpressionList" />
            </xs:choice>
        </xs:sequence>
        <xs:attribute name="ns" type="xs:string" use="required" />
        <xs:attribute name="node" type="xs:string" use="required" />
        <xs:attribute ref="xml:base" />
    </xs:complexType>

    <xs:complexType name="TopicExpressionList">
        <xs:sequence minOccurs="1" maxOccurs="unbounded">
            <xs:element name="topic" type="Expression" />
        </xs:sequence>
        <xs:attribute name="publish" type="RuleQualifier" use="optional" />
        <xs:attribute name="subscribe" type="RuleQualifier" use="optional" />
        <xs:attribute ref="xml:base" />
    </xs:complexType>
```

```
    <xs:complexType name="ServicesExpressionList">
        <xs:sequence minOccurs="1" maxOccurs="unbounded">
            <xs:element name="service" type="Expression" />
        </xs:sequence>
        <xs:attribute name="reply" type="RuleQualifier" use="optional" />
        <xs:attribute name="request" type="RuleQualifier" use="optional" />
        <xs:attribute ref="xml:base" />
    </xs:complexType>

    <xs:complexType name="ActionsExpressionList">
        <xs:sequence minOccurs="1" maxOccurs="unbounded">
            <xs:element name="action" type="Expression" />
        </xs:sequence>
        <xs:attribute name="call" type="RuleQualifier" use="optional" />
        <xs:attribute name="execute" type="RuleQualifier" use="optional" />
        <xs:attribute ref="xml:base" />
    </xs:complexType>

    <xs:simpleType name="Expression">
        <xs:restriction base="xs:string" />
    </xs:simpleType>

    <xs:simpleType name="RuleQualifier">
        <xs:restriction base="xs:string">
            <xs:enumeration value="ALLOW" />
            <xs:enumeration value="DENY" />
        </xs:restriction>
    </xs:simpleType>

</xs:schema>
```

`<policy>` Tag

Root tag of the policy file. There must be only one `<policy>` tag per policy file.

Attributes: - **version**: declared version of schema version in use - Allows for advancing future revisions of the schema

`<enclaves>` Tag

Encapsulates a sequence of unique enclaves. This method of nesting sequences allows for additional tags to be extended to the `<policy>` root.

`<enclave>` Tag

Encapsulates a collection of profiles. This is specific to an enclave as determined by associative attributes.

Attributes: - **path**: Fully qualified enclave path

Given that multiple nodes can be composed into a single process, an enclave is used to contain the collection of profiles of all respective nodes. An enclave may therefore be considered the union of contained profiles. Note that the union of profiles within an enclave will result in denied privileges of any profile to supersede all allowed privileges for every profile. E.g. if a profile asks for a permission but a matching permission has been explicitly denied by another profile in the enclave, the deny rule will take precedence. See section `<profile>` Tag for more info on how MAC is applied.

`<profiles>` Tag

Encapsulates a sequence of unique profiles and designated metadata. This method of nesting sequences allows for additional tags to be extended to the `<enclave>` root, as well as associating particular metadata or constraints to the contained profile elements.

Attributes: - **type**: Specifies the transport type of profiles and metadata

`<profile>` Tag

Encapsulates a collection of subject privileges. This is specific to a unique node instance as determined by associative attributes.

Attributes: - **ns**: Namespace of the node - **node**: Name of the node

In accordance with MAC, privileges must be explicitly qualified as allowed. Additionally, as with many other MAC languages, while composed privileges may overlap, any particular denied privilege will suppress any similarly applicable allowed privileges. That is to say the priority of denied privileges conservatively supersedes allowed privileges, avoiding potential lapses in PoLP. This method of flatting privileges enables users to provision general access to a larger set of objects, while simultaneously revoking access to a smaller subset of sensitive objects. Although recursion of qualifiers is subsequently prevented, transformations are subsequently simplified, preventing potential for unintended access.

`<metadata>` Tag

Encapsulates arbitrary metadata or constraints. This could include transport specific permission details applicable to sibling profile elements. There can only one `metadata` element per `profiles` parent element.

Attributes: - To be defined

Given the use cases for bridge interfaces where an enclave's credentials may be used to interconnect across multiple transports or to transport specific domains, it may be necessary to qualify certain profile sequences with particular constraints, while doing so multiple times for separate profiles per enclave. This allows advanced users to holistically control the intersect of permissions across transport domains, while retaining accurate model fidelity of security permissions. Given how security sensitive bridge interfaces are and the attack surface they expose, it is vital that information flow control within a bridge remains formally verifiable for safe and secure operation.

Privileges

Privileges are defined as configuration of rules and permissions for object access. As objects can be categorized by subsystem type, rules and respective permission are identically structured. Given an average profile is likely to reference more unique objects with multiple permissions than number of rules, the subsequent hierarchy of rules/permissions/objects is chosen to minimize verbosity.

rule types	**permissions**
actions	call, execute
services	reply, request
topics	publish, subscribe

Each subsystem is associated to a given rule type, while permissions are expressed as attributes in their respective respective rule tags. Uniqueness or ordering of rules in this sequence is not required, as this is accounted for by transformation templates. In fact, a profile may contain an empty set of privileges; particularly useful when a node may require no subsystem permissions, but must still be provisioned an identity nonetheless for discovery purposes in DDS.

Each rule includes a sequence of objects that the permissions apply. For some secure transports, such as Secure DDS, matching expressions may also be used to expand the scope further using globbing patterns, specifically those supported by fnmatch as specified in the POSIX standard. However, caution should be taken when using expression matching, as discussed further in the concerns section.

Basic fnmatch-style patterns are supported:

Pattern	**Meaning**
*	Matches everything
?	Matches any single character
[sequence]	Matches any character in sequence
[!sequence]	Matches any character not in sequence

`<topics>` Tag

A group of `<topic>` tags with the specified permissions.

Attributes: - **publish**: Whether or not publication on this set of topics is allowed - i.e. whether the node can be a topic publisher - Valid values are "ALLOW" or "DENY" - **subscribe**: Whether or not subscription on this set of topics is allowed - i.e. whether the node can be a topic subscriber - Valid values are "ALLOW" or "DENY"

`<services>` Tag

A group of `<service>` tags with the specified permissions.

Attributes: - **request**: Whether or not requesting the service is allowed - i.e. whether the node can be a service client - Valid values are "ALLOW" or "DENY" - **reply**: Whether or not replying to service requests is allowed - i.e. whether the node can be a service server - Valid values are "ALLOW" or "DENY"

`<actions>` Tag

A group of `<action>` tags with the specified permissions.

Attributes: - **call**: Whether or not calling the action is allowed - i.e. whether the node can be an action client - Valid values are "ALLOW" or "DENY" - **execute**: Whether or not executing the action is allowed - i.e. whether the node can be an action server - Valid values are "ALLOW" or "DENY"

Templating

To transpile SROS 2 policies into security artifacts for a targeted access controlled transport, XSLT templates can be used to perform this level of document conversion. This may include any number of optimisations or adjustments specific for the target transport. For example, the pipeline stages for targeting Secure DDS is as follows:

1. An XML document with a root of the SROS 2 policy is specified
2. The document is fully expanded using XInclude to reference external elements
3. The expanded document is then validated with the equivalent schema version
4. At this point the document tree may or may not be pruned to a particular profile
5. The valid document is then transpiled using the transform template
6. For each profile, a matching DDS grant is appended into the permission document - privileges and namespaces are remapped into a DDS centric representations - privileges with matching attributes are conjoined to reduce payload size - duplicate objects in the same privilege are pruned to reduce payload size - privileges are sorted deny first, abiding the priority of qualifiers when using DDS - objects are also sorted alphabetically to improve readability and change diffs

Alternatives

This section lists concerns about the proposed design and alternatives that were considered, including different Markup Languages and policy formats.

YAML

YAML, a recursive acronym for “YAML Ain't Markup Language”, was originally adopted for specifying access control policies in the first version of SROS [1]. Although the policy model used in SROS 1 was semantically equivalent, the YAML format lent it being quite verbose due to repetition of permissions per namespaced resource given the lack of clear element attributes. For SROS 2 we decided switched away from YAML to XML for many of the reasons weighed in the following pros and cons:

- Pros
  - Human Readable: Minimal Line Noise
    - YAML has very minimal syntax and is targeted for human editable configuration files, making it simple to read and write manually.
  - Data Model: Intuitive Interpretation
    - YAML has very simple data model forming tree structure using key-value pair dictionaries and lists making it quite approachable.
- Cons
  - Parsability: Implicit Type Casting
    - Given YAML is a data-serialization language, it may attempt to type cast where possible. However this does not always have the desired effect and may lead to unintended behaviors. Parsing of booleans v.s. strings are notable example of ambiguity.
  - Interpreters: Validation and Transformation
    - Although YAML is supported for many programming languages, YAML itself provides no schema to enforce document structure.

        - Validation must be repeated for each interpreter implementation, rendering it non-agnostic to the programing language used.
        - Similarly, transformations for transpiling policies into transport security artifacts is less generalizable across implementations.
    - Composability: Reuse of Profiles
        - Although YAML supports a degree of composability via Anchors, Aliases and Extensions, allowing documents to be more DRY, these do not extend to separate files or external resources.
    - Expressiveness: Succinct Representation
        - Given YAML's inherit data model, it's expressive power is quite limited, necessitating either verbose file structures, or unintuitive options to achieve similar access control configurations.

Custom

As an alternative to choosing an existing markup format, it would be possible to define our own formal language for expressing access control permissions for ROS 2 using a custom file syntax. An example of a MAC based policy language would include that which is used in AppArmor. Although affording the flexibility to succinctly express profile permission while minimizing general syntactic overhead, this approach was not pursued for many of the reasons weighed in the following pros and cons:

- Pros
    - Expressiveness: Succinct Representation
        - Complete control of syntax and interpretation, allowing for domain specific optimizations for SROS policy representation.
- Cons
    - Interpreters: Validation and Transformation
        - Specification and implementation for parsing and interpreting a custom policy format would be considered undertaking.
        - Validation must be repeated for each interpreter implementation, rendering it non-agnostic to the programing language used.
        - Similarly, transformations for transpiling policies into transport security artifacts is less generalizable across implementations.
    - Correctness: Policy Remaining Sound
        - Maintaining and synchronizing parsing support across multiple programing language could affect policy soundness.

ComArmor

ComArmor [2], a predecessor and inspiration for the existing XML ROS 2 policy format now used in SROS 2, is itself inspired from AppArmor's policy language. ComArmor facilitates composition through the use of a nested tree structure of policy/profile/permission primitives. As with AppArmor, it also supports nesting of profiles, i.e. importation of child profiles into that of a parent profile. While this greatly extends the flexibility given the nesting of imported sun-profile hierarchies, it also adds complexity to the transpiling process when converting policies to security transport artifacts.

In an effort to strike a balance between simplicity and flexibility, a flat sequence of single level profiles was opted for SROS 2 instead, allowing the policy format to serve as both a grounded intermediate representation for higher level policy languages and tools to build upon, such as XACML or Keymint, while remaining succinctly expressive of ROS concepts for general use.

Concerns

Separation of Privileges

ROS 2 subsystems such as topics, services, actions, and parameters must eventually map to transport layer interfaces, such as DDS topic, that can sufficiently enforce the desired access control policy in order to secure the ROS application layer. However, any quirks between mapping of subsystems and separation of privileges can degrade security.

As an example, if granting access to all topics and services starting with `/foo` additionally grants access to all actions starting with `/foo`, this would be a weak example of privilege separation. Such can be exacerbated when using globbing expressions that include matching patterns, such as with `fnmatch`, leading to innocuous and sound policies being inaccurately applied to the underlying transport security. While such privilege separation in remains week between ROS 2 and DDS, perhaps it is wise to discourage the use of expression matching for general use in permissions.

Separation of Concerns

Middleware transports, such as DDS, offer a myriad of features and options, such as those for QoS as well as security. Drawing a boundary between many of them when deciding what to expose from a configuration standpoint can be tricky. Still, among the objective for ROS 2 includes remaining as agnostic of transport as reasonable.

Although the SROS 2 policy format was intentionally structured to mimic that of Secure DDS's permission.xml format, care should be taken when adding extensions to surface non-functional security properties tangential to ROS 2 data flow, such as governance on encryption or discovery.

Yet, if the intended purpose of SROS 2 policy becomes that of an intermediate representation across transports, and is subsequently auto generated from higher level tooling/representations, or composability is adequate preserved, then perhaps this concern is of lesser priority.

Composability

ROS 2 allows for the remapping of many namespaced subsystems at runtime, such as when reusing launch files to orchestrate larger applications. While it is perhaps unreasonable to expect this dynamic flexibility from staticky provisioned permissions without allocating such capabilities prior, it should be made possible to infer the necessary capabilities from composed launch files and similar orchestrations.

Static analysis of such remapping in conjunction with the setting of the nominal requirements of respective nodes could be used to auto generate the new satisfactory policies. However, inferring such policies from the source code could be equated to the halting problem. Thus, it stands to reason nodes could instead provide a manifest or IDL defining these nominal requirements so that permission may as easily be remapped, at least at design time.

References

1. SROS1: Using and Developing Secure ROS1 Systems

```
@inbook{White2019,
    title     = {SROS1: Using and Developing Secure ROS1 Systems},
```

```
    author    = {White, Ruffin and Caiazza, Gianluca and Christensen, Henrik and Cortesi, Agostino},
    year      = 2019,
    booktitle = {Robot Operating System (ROS): The Complete Reference (Volume 3)},
    doi       = {10.1007/978-3-319-91590-6_11},
    isbn      = {978-3-319-91590-6},
    url       = {https://doi.org/10.1007/978-3-319-91590-6_11}}
```

1. Procedurally Provisioned Access Control for Robotic Systems

```
@inproceedings{White2018,
    title     = {Procedurally Provisioned Access Control for Robotic Systems},
    author    = {White, Ruffin and Caiazza, Gianluca and Christensen, Henrik and Cortesi, Agostino},
    year      = 2018,
    booktitle = {2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)},
    doi       = {10.1109/IROS.2018.8594462},
    issn      = {2153-0866},
    url       = {https://doi.org/10.1109/IROS.2018.8594462}}
```

Historical source: ROS 2 Security Enclaves

Ruffin White, Mikael Arguedas

Written 2020-05; declared modified 2020-07. Pinned original source. Original OSRF design-document attribution and CC BY 3.0 notice retained. Presentation adapted for this reader; historical instructions and findings have not been modernized.

ROS 2 Security Enclaves

This article specifies the integration of security enclaves.

Authors: Ruffin White, Mikael Arguedas

Date Written: 2020-05

Last Modified: 2020-07

This design document formalizes the integration of ROS 2 with security enclaves. In summary, all secure processes must use an enclave that contains the runtime security artifacts unique to that enclave, yet each process may not necessarily have a unique enclave. Multiple enclaves can be encapsulated in a single security policy to accurately model the information flow control. Users can tune the fidelity of such models by controlling at what scope enclaves are applied at deployment. E.g. one unique enclave per OS process, or per OS user, or per device/robot, or per swarm, etc. The rest of this document details how enclaves can be organized and used by convention.

Concepts

Before detailing the SROS 2 integration of the enclaves, the following concepts are introduced.

Participant

Participant is the object representing a single entity on the network. In the case of DDS, the `Participant` is a DDS DomainParticipant, which has both access control permissions and a security identity.

Namespaces

Namespaces are a fundamental design pattern in ROS and are widely used to organize and differentiate many types of resources as to be uniquely identifiable; i.e. for topics, services, actions, and node names. As such, the concept of namespacing is well known and understood by current users, as well as strongly supported by the existing tooling. Namespaces are often configurable at runtime via command line arguments or statically/programmatically via launch file declarations.

Previously, the Fully Qualified Name (FQN) of a node was used directly by a selected security directory lookup strategy to load the necessary key material. However, now that Participants map to contexts and not nodes, such a direct mapping of node FQN to security artifacts is no longer appropriate.

Contexts

With the advent of ROS 2, multiple nodes may now be composed into one process for improved performance. Previously however, each node would retain it's one to one mapping to a separate middleware `Participant`. Given the non-negligible overhead incurred of multiple `Participant`s per process, a change was introduced to map a single `Participant` to a context, and allow for multiple nodes to share that context.

Based on the DDS Security specification v1.1, a `Participant` can only utilise a single security identity; consequently the access control permissions applicable to every node mapped to a given context must be consolidated and combined into a single set of security artifacts, or enclave. Thus, all contexts and their respective participants in each process will then use that single enclave. As such, additional tooling and extensions to SROS 2 are necessary to support this new paradigm.

Keystore

With the addition of contexts, it’s perhaps best to take the opportunity to restructure the keystore layout as well. Rather than a flat directory of namespaced node security directories, we can push all such security directories down into a designated `enclaves` sub-folder. Similarly, private and public keystore materials can also be pushed down into their own respective sub-folders within the root keystore directory. This is reminiscent of the pattern used earlier in Keymint [1].

```
$ tree keystore/
keystore
├── enclaves
│   └── ...
│       └── ...
├── private
│   ├── ca.csr.pem
│   └── ca.key.pem
└── public
    ├── ca.cert.pem
    ├── identity_ca.cert.pem -> ca.cert.pem
    └── permissions_ca.cert.pem -> ca.cert.pem
```

`public`

The `public` directory contains anything permissible as public, such as public certificates for the identity or permissions certificate authorities. As such, this can be given read access to all executables. Note that in the default case, both the `identity_ca` and `permissions_ca` points to the same CA certificate.

`private`

The `private` directory contains anything permissable as private, such as private key material for aforementioned certificate authorities. This directory should be redacted before deploying the keystore onto the target device/robot.

`enclaves`

The `enclaves` directory contains the security artifacts associated with individual enclaves, and thus node directories are no longer relevant. Similar to node directories however, the `enclaves` folder may still recursively nest sub-paths for organizing separate enclaves. The `ROS_SECURITY_KEYSTORE` environment variable should by convention point to this directory.

Integration

With the introduction of contexts into rcl, instead of relying on node namespaces to lookup security artifacts from the keystore, the enclave path is now completely disassociated from node namespaces, instead serving as its own unique resource identifier. Although having to book keep both identifier spaces simultaneously may introduce more degrees of freedom, it should still be possible to organize enclaves within the keystore to mimic node namespace hierarchy for transparent traceability of allocated permissions.

Future Work

Introspective tooling and launchfile interfaces should be updated to help lessen the complexity introduced with the migration to contexts and enclaves.

Runtime

Given the normative case where an enclave within a policy may be specific to a single node/container process, the namespace the node is remapped to will inevitably affect the required security permissions necessary within the enclave. To highlight this interdependency, and to help avoid enclave path collisions, a hierarchy borrowing namespaces is appropriate. By convention, ros2launch could be used to prefix relative enclave paths for single process node or containers using the namespace in scope, to enable a convention of composable launch files with adjustable and parameterized enclave paths. Given the runtime command argument for specifying the fully qualified enclave path, ros2launch would accordingly resolve relative enclave paths for executables, as defined by launch attributes.

Unqualified enclave path

For single process nodes with unqualified enclave paths, the enclave directory will subsequently default to the root level enclave.

```
<launch>
  <node pkg="demo_nodes_cpp" exec="talker"/>
  <node pkg="demo_nodes_cpp" exec="listener"/>
</launch>
```

```
$ tree enclaves/
enclaves/
├── cert.pem
├── governance.p7s
├── identity_ca.cert.pem -> ../public/identity_ca.cert.pem
├── key.pem
├── permissions_ca.cert.pem -> ../public/permissions_ca.cert.pem
└── permissions.p7s
```

Pushed unqualified enclave path

For single process nodes with unqualified enclave paths pushed by a namespace, the enclave directory will subsequently be pushed to the relative sub-folder.

```
<launch>
  <node pkg="demo_nodes_cpp" exec="talker"/>
```

```
  <group>
    <push_ros_namespace namespace="foo"/>
    <node pkg="demo_nodes_cpp" exec="listener"/>
  </group>
</launch>
```

```
$ tree --dirsfirst enclaves/
enclaves/
├── foo
│   ├── cert.pem
│   ├── governance.p7s
│   ├── identity_ca.cert.pem
│   ├── key.pem
│   ├── permissions_ca.cert.pem
│   └── permissions.p7s
├── cert.pem
├── governance.p7s
├── identity_ca.cert.pem
├── key.pem
├── permissions_ca.cert.pem
└── permissions.p7s
```

> Symbolic links suppressed for readability

Relatively pushed qualified enclave path

For single process nodes with qualified enclave paths pushed by a namespace, the qualified enclave directory will subsequently be pushed to the relative sub-folder.

```
<launch>
  <group>
    <push_ros_namespace namespace="foo"/>
    <node pkg="demo_nodes_cpp" exec="listener" enclave="bar"/>
  </group>
</launch>
```

```
$ tree --dirsfirst enclaves/
enclaves/
└── foo
    └── bar
        ├── cert.pem
        ├── governance.p7s
        ├── identity_ca.cert.pem
        ├── key.pem
        ├── permissions_ca.cert.pem
        └── permissions.p7s
```

Fully qualified enclave path

For single process nodes with absolute enclave paths, namespaces do not subsequently push the relative sub-folder.

```
<launch>
  <group>
    <push_ros_namespace namespace="foo"/>
    <node pkg="demo_nodes_cpp" exec="listener" enclave="/bar"/>
  </group>
</launch>
```

```
$ tree --dirsfirst enclaves/
enclaves/
└── bar
    ├── cert.pem
    ├── governance.p7s
    ├── identity_ca.cert.pem
    ├── key.pem
    ├── permissions_ca.cert.pem
    └── permissions.p7s
```

Alternatives

`<push_ros_namespace namespace="..." enclave="foo"/>`

One such approach could be done by adding a `enclave` attribute to `push_ros_namespace` element. This also keeps the pushing of enclaves close/readable to pushing of namespaces.

`<push_ros_enclave enclave="foo"/>`

Another alternative approach could be to add an entirely new `push_ros_enclave` element. This could ensure the pushing of enclave path independent/flexable from namespaces.

Concerns

Multiple namespaces per context

For circumstances where users may compose multiple nodes of dissimilar namespaces into a single context, the user must still subsequently specify a common enclave path that is applicable for all nodes composed. For circumstances where the enclave path is orthogonal to node namespace, the use of fully qualifying all relevant enclave paths could be tedious, but could perhaps could still be parametrized via the use of `<var/>`, and `<arg/>` substitution and expansion.

Modeling permissions of nodes in a process v.s. permission of the middleware `Participant`

Before the use of contexts, multiple nodes composed into a single process where each mapped to a separate `Participant`. Each `Participant` subsequently load a security identity and access control credential prevalent to its' respective node. However, all nodes in that process share the same memory space and can thus access data from other nodes. There is a mismatch between the middleware credentials/permissions loaded and the resources accessible within the process.

By using enclaves, all nodes in a context share the same security identity and access control credentials. This inevitably means that code compiled to node `foo` can access credentials/permissions only trusted to node `bar`. This consequence of composition could unintendedly subvert the minimal spanning policy as architected by the policy designer or measured/generated via ROS 2 tooling/IDL.

With the introduction of enclaves, it becomes possible to describe the union of access control permission by defining a collection of SROS 2 policy profiles as element within a specific enclave. This would allow for formal analysis tooling [2] to check for potential violations in information flow control given the composing of nodes at runtime. If a process loads a single enclave, this reconciles the permissions of a `Participant` and the ones of the process.

However, should multiple enclaves be loaded per process, then such security guaranties are again lost because of shared same memory space. Thus it should be asked whether if multiple enclaves per process should even be supported.

In summary, the distinction here is that before, the composition of multiple node permissions could not be conveyed to the tooling. Whether nodes could gain the permission of others in the same process space is not the hinge point of note; it's the fact that such side effects could not be formally modeled or accounted for by the designer. It will now be possible with enclaves, however allowing for multiple contexts per process that load separate enclaves would reintroduce and exacerbate the same modeling inaccuracies.

Composable launchfile includes

A particular challenge in using launchfiles with security enclaves is that of keeping the include hierarchy composable. An inherit tradeoff between simplicity and configurability can arise when writing launchfiles for downstream use. Authors can selectively choose what attributes to expose as input arguments, while users may implicitly override provided defaults.

In case of enclaves, it is not inherently clear what best practices either package authors or users should employ to retain a composable and intuitive launchfile structure. E.g: Should authors parametrize enclave paths for each node as input arguments? Should users push namespaces of included launchfiles to unique enclaves?

To be sure though, the setting of security environment variables from within launchfiles should be discouraged, as this would restrict the use of static analysis of launchfiles combined with Node IDL for procedural policy generation.

Composable nodes in container

Given that containers can be dynamic, where nodes can be added or removed at runtime, there is perhaps some question as to how containers should integrate with secure enclaves. In ros2launch, the namespace in scope at the container's instantiation could be used to resolve the container's specified relative enclave path, thus to all nodes/components inside that container. This should be further deliberated when eventually extending the launch API for containers.

Migration for RMW implementations

As it may take time before all RMW implementations implement the new system of contexts, a defined fallback behavior should still be designated. For such implementations, the enclave security directory determined by RCL should be loaded for the participant as specified in the "ROS 2 DDS-Security integration" design doc. This primarily desists the use of including the node name in the default lookup path, consequently getting users in the habit of creating separate enclaves for separate processes, or explicitly specifying unique enclave paths via launchfiles.

References

1. Procedurally Provisioned Access Control for Robotic Systems

```
@inproceedings{White2018,
    title     = {Procedurally Provisioned Access Control for Robotic Systems},
    author    = {White, Ruffin and Caiazza, Gianluca and Christensen, Henrik and Cortesi, Agostino},
    year      = 2018,
```

```
    booktitle = {2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)},
    doi       = {10.1109/IROS.2018.8594462},
    issn      = {2153-0866},
    url       = {https://arxiv.org/pdf/1810.08125.pdf}}
```

1. Network Reconnaissance and Vulnerability Excavation of Secure DDS Systems

```
@inproceedings{White2019,
    title     = {Network Reconnaissance and Vulnerability Excavation of Secure DDS Systems},
    author    = {White, Ruffin and Caiazza, Gianluca and Jiang, Chenxu and Ou, Xinyue and Yang, Zhiyue and Cortesi,
Agostino and Christensen, Henrik},
    year      = 2019,
    booktitle = {2019 IEEE European Symposium on Security and Privacy Workshops (EuroS PW)},
    doi       = {10.1109/EuroSPW.2019.00013},
    pages     = {57-66},
    url       = {https://arxiv.org/abs/1908.05310.pdf}}
```

Historical source: ROS 2 Robotic Systems Threat Model

Thomas Moulard, Juan Hortala Xabi Perez Gorka Olalde Borja Erice Odei Olalde David Mayoral

Written 2019-03; declared modified 2021-01. Pinned original source. Original OSRF design-document attribution and CC BY 3.0 notice retained. Presentation adapted for this reader; historical instructions and findings have not been modernized.

ROS 2 Robotic Systems Threat Model

Authors: Thomas Moulard, Juan Hortala Xabi Perez Gorka Olalde Borja Erice Odei Olalde David Mayoral

Date Written: 2019-03

Last Modified: 2021-01

This is a **DRAFT DOCUMENT**.

Disclaimer:

- This document is not exhaustive. Mitigating all attacks in this document does not ensure any robotic product is secure.
- This document is a live document. It will continue to evolve as we implement and mitigate attacks against reference platforms.

Document Scope

This document describes potential threats for ROS 2 robotic systems. The document is divided into two parts:

1. Robotic Systems Threats Overview
2. Threat Analysis for the TurtleBot 3 Robotic Platform
3. Threat Analysis for the MARA Robotic Platform

The first section lists and describes threats from a theoretical point of view. Explanations in this section should hold for any robot built using a component-oriented architecture. The second section instantiates those threats on a widely-available reference platform, the TurtleBot 3. Mitigating threats on this platform enables us to demonstrate the viability of our recommendations.

Robotic Systems Threats Overview

This section is intentionally independent from ROS as robotic systems share common threats and potential vulnerabilities. For instance, this section describes "robotic components" while the next section will mention "ROS 2 nodes".

Defining Robotic Systems Threats

We will consider as a robotic system one or more general-purpose computers connected to one or more actuators or sensors. An actuator is defined as any device producing physical motion. A sensor is defined as any device capturing or recording a physical property.

Robot Application Actors, Assets, and Entry Points

This section defines actors, assets, and entry points for this threat model.

Actors are humans or external systems interacting with the robot. Considering which actors interact with the robot is helpful to determine how the system can be compromised. For instance, actors may be able to give commands to the robot which may be abused to attack the system.

Assets represent any user, resource (e.g. disk space), or property (e.g. physical safety of users) of the system that should be defended against attackers. Properties of assets can be related to achieving the business goals of the robot. For example, sensor data is a resource/asset of the system and the privacy of that data is a system property and a business goal.

Entry points represent how the system is interacting with the world (communication channels, API, sensors, etc.).

Robot Application Actors

Actors are divided into multiple categories based on whether or not they are physically present next to the robot (could the robot harm them?), are they human or not and are they a "power user" or not. A power user is defined as someone who is knowledgeable and executes tasks which are normally not done by end-users (build and debug new software, deploy code, etc.).

Actor	Co-Located?	Human?	Power User?	Notes
Robot User	Y	Y	N	Human interacting physically with the robot.
Robot Developer / Power User	Y	Y	Y	User with robot administrative access or developer.
Third-Party Robotic System	Y	N	-	Another robot or system capable of physical interaction with the robot.
Teleoperator / Remote User	N	Y	N	A human tele-operating the robot or sending commands to it through a client application (e.g. smartphone app)
Cloud Developer	N	Y	Y	A developer building a cloud service connected to the

Actor	Co-Located?	Human?	Power User?	Notes
				robot or an analyst who has been granted access to robot data.
Cloud Service	N	N	-	A service sending commands to the robot automatically (e.g. cloud motion planning service)

Assets

Assets are categorized in privacy (robot private data should not be accessible by attackers), integrity (robot behavior should not be modified by attacks) and availability (robot should continue to operate even under attack).

Asset	Description
Privacy	
Sensor Data Privacy	Sensor data must not be accessed by unauthorized actors.
Robot Data Stores Privacy	Robot persistent data (logs, software, etc.) must not be accessible by unauthorized actors.
Integrity	
Physical Safety	The robotic system must not harm its users or environment.
Robot Integrity	The robotic system must not damage itself.
Robot Actuators Command Integrity	Unallowed actors should not be able to control the robot actuators.
Robot Behavior Integrity	The robotic system must not allow attackers to disrupt its tasks.
Robot Data Stores Integrity	No attacker should be able to alter robot data.
Availability	
Compute Capabilities	Robot embedded and distributed (e.g. cloud) compute resources. Starving a robot from its compute resources can prevent it from operating correctly.
Robot Availability	The robotic system must answer commands in a reasonable time.
Sensor Availability	Sensor data must be available to allowed actors shortly after being produced.

Entry Points

Entry points describe the system attack surface area (how do actors interact with the system?).

Name	Description
Robot Components Communication Channels	Robotic applications are generally composed of multiple components talking over a shared bus. This bus may be accessible over the robot WAN link.
Robot Administration Tools	Tools allowing local or remote users to connect to the robot computers directly (e.g. SSH, VNC).
Remote Application Interface	Remote applications (cloud, smartphone application, etc.) can be used to read robot data or send robot commands (e.g. cloud REST API, desktop GUI, smartphone application).
Robot Code Deployment Infrastructure	Deployment infrastructure for binaries or configuration files are granted read/write access to the robot computer's filesystems.
Sensors	Sensors are capturing data which usually end up being injected into the robot middleware communication channels.
Embedded Computer Physical Access	External (HDMI, USB...) and internal (PCI Express, SATA...) ports.

Robot Application Components and Trust Boundaries

The system is divided into hardware (embedded general-purpose computer, sensors, actuators), multiple components (usually processes) running on multiple computers (trusted or non-trusted components) and data stores (embedded or in the cloud).

While the computers may run well-controlled, trusted software (trusted components), other off-the-shelf robotics components (non-trusted) nodes may be included in the application. Third-party components may be malicious (extract private data, install a root-kit, etc.) or their QA validation process may not be as extensive as in-house software. Third-party components releasing process create additional security threats (third-party component may be compromised during their distribution).

A trusted robotic component is defined as a node developed, built, tested and deployed by the robotic application owner or vetted partners. As the process is owned end-to-end by a single organization, we can assume that the node will respect its specifications and will not, for instance, try to extract and leak private information. While carefully controlled engineering processes can reduce the risk of malicious behavior (accidentally or voluntarily), it cannot completely eliminate it. Trusted nodes can still leak private data, etc.

Trusted nodes should not trust non-trusted nodes. It is likely that more than one non-trusted component is embedded in any given robotic application. It is important for non-trusted components to not trust each other as one malicious non-trusted node may try to compromise another non-trusted node.

An example of a trusted component could be an in-house (or carefully vetted) IMU driver node. This component may communicate through unsafe channels with other driver nodes to reduce sensor data fusion latency. Trusting components is never ideal but it may be acceptable if the software is well-controlled.

On the opposite, a non-trusted node can be a third-party object tracker. Deploying this node without adequate sandboxing could impact:

- User privacy: the node is streaming back user video without their consent
- User safety: the robot is following the object detected by the tracker and its speed is proportional to the object distance. The malicious tracker estimates the object position very far away on purpose to trick the robot into suddenly accelerating and hurting the user.
- System availability: the node may try to consume all available computing resources (CPU, memory, disk) and prevent the robot from performing correctly.
- System Integrity: the robot is following the object detected by the tracker. The attacker can tele-operate the robot by controlling the estimated position of the tracked object (detect an object on the left to make the robot move to the left, etc.).

Nodes may also communicate with the local filesystem, cloud services or data stores. Those services or data stores can be compromised and should not be automatically trusted. For instance, URDF robot models are usually stored in the robot file system. This model stores robot joint limits. If the robot file system is compromised, those limits could be removed which would enable an attacker to destroy the robot.

Finally, users may try to rely on sensors to inject malicious data into the system (Akhtar, Naveed, and Ajmal Mian. “Threat of Adversarial Attacks on Deep Learning in Computer Vision: A Survey.”).

The diagram below illustrates an example application with different trust zones (trust boundaries showed with dashed green lines). The number and scope of trust zones is depending on the application.

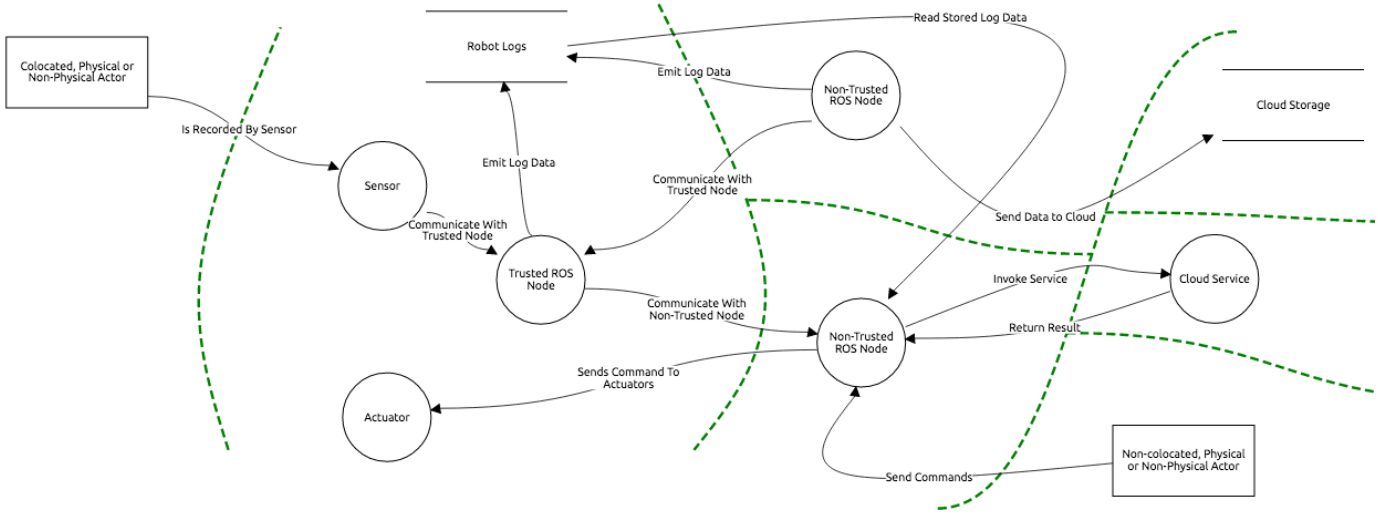


Figure 1 · RobotSystemThreatModel.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (edited with Threat Dragon)

Threat Analysis and Modeling

The table below lists all *generic* threats which may impact a robotic application.

Threat categorization is based on the STRIDE (Spoofing / Tampering / Repudiation / Integrity / Denial of service / Elevation of privileges) model. Risk assessment relies on DREAD (Damage / Reproducibility / Exploitability / Affected users / Discoverability).

In the following table, the "Threat Category (STRIDE)" columns indicate the categories to which a threat belongs. If the "Spoofing" column is marked with a check sign (✓), it means that this threat can be used to spoof a component of the system. If it cannot be used to spoof a component, a cross sign will be present instead (✘).

The "Threat Risk Assessment (DREAD)" columns contain a score indicating how easy or likely it is for a particular threat to be exploited. The allowed score values are 1 (not at risk), 2 (may be at risk) or 3 (at risk, needs to be mitigated). For instance, in the damage column a 1 would mean "exploitation of the threat would cause minimum damages", 2 "exploitation of the threat would cause significant damages" and 3 "exploitation of the threat would cause massive damages". The "total score" is computed by adding the score of each column. The higher the score, the more critical the threat.

Impacted assets, entry points and business goals columns indicate whether an asset, entry point or business goal is impacted by a given threat. A check sign (✓) means impacted, a cross sign (✘) means not impacted. A triangle (▲) means "impacted indirectly or under certain conditions". For instance, compromising the robot kernel may not be enough to steal user data but it makes stealing data much easier.

> Historical matrix reflowed as vertical records. ✓, ✘ and ▲ retain their source meanings. SROS Enabled? means the attack can occur with SROS enabled, not that SROS prevents it. Scores and mitigation claims are historical and unverified in this review.

Embedded / Software / Communication / Inter-Component Communication

---

GEN-001 · An attacker spoofs a component identity.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✓

Historical DREAD scores

Damage: 3; Reproducibility: 1; Exploitability: 1; Affected users: 2; Discoverability: 3; Total: 10

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should authenticate themselves.
- Components should not be attributed similar identifiers.

- Component identifiers should be chosen carefully.

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-002 · An attacker intercepts and alters a message.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ▲

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Messages should be signed and/or encrypted.

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-003 · An attacker writes to a communication channel without authorization.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should only communicate on encrypted channels.
- Sensitive inter-process communication should be done through shared memory whenever possible.

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-004 · An attacker listens to a communication channel without authorization.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should only communicate on encrypted channels.
- Sensitive inter-process communication should be done through shared memory whenever possible.

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-005 · An attacker prevents a communication channel from being usable.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✓; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should only be allowed to access channels they require.
- Internet-facing channels and robot-only channels should be isolated.
- Components behaviors should be tolerant of a loss of communication (e.g. go to x,y vs set velocity to vx, vy).

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

Embedded / Software / Communication / Long-Range Communication (e.g. WiFi, Cellular Connection)

GEN-006 · An attacker hijacks robot long-range communication

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 1; Total: 10

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ▲; Robot integrity: ✓; Data integrity: ✓; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Long-range communication should always use a secure transport layer (WPA2 for WiFi for instance)

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-007 · An attacker intercepts robot long-range communications (e.g. MitM)

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 1; Total: 8

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Long-range communication should always use a secure transport layer (WPA2 for WiFi for instance)

Similar attacks in the literature
Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

GEN-008 · An attacker disrupts (e.g. jams) robot long-range communication channels.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 1; Affected users: 1; Discoverability: 3; Total: 9

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✘; Data integrity: ✘; Data availability: ✓; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Multiple long-range communication transport layers should be used when possible (e.g. cellular and WiFi)

Similar attacks in the literature

Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015.

Embedded / Software / Communication / Short-Range Communication (e.g. Bluetooth)

GEN-009 · An attacker executes arbitrary code using a short-range communication protocol vulnerability.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✓; Information disclosure: ✓; Denial of service: ✓; Elevation of privilege: ✓

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 1; Discoverability: 3; Total: 10

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Communications protocols should be disabled if unused (by using e.g. rfkill).
- Binaries and libraries required to support short-range communications should be kept up-to-date.

Similar attacks in the literature

Checkoway, Stephen, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, Stefan Savage, Karl Koscher, Alexei Czeskis, Franziska Roesner, and Tadayoshi Kohno. “Comprehensive Experimental Analyses of Automotive Attack Surfaces.” In Proceedings of the 20th USENIX Conference on Security, 6–6. SEC’11. Berkeley, CA, USA: USENIX Association, 2011.

Embedded / Software / Communication / Remote Application Interface

---

GEN-010 · An attacker gains unauthenticated access to the remote application interface.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✓; Elevation of privilege: ▲

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 1; Affected users: 1; Discoverability: 3; Total: 11

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✘; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Implement authentication and authorization methods.
- Enable RBAC to limit permissions for the users.

Similar attacks in the literature

(Empty in source.)

GEN-011 · An attacker could eavesdrop communications to the Robot’s remote application interface.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 1; Exploitability: 1; Affected users: 1; Discoverability: 3; Total: 7

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Communications with the remote application interface should be done over a secure channel.

Similar attacks in the literature

(Empty in source.)

GEN-012 · An attacker could alter data sent to the Robot’s remote application interface.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✓; Elevation of privilege: ▲

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 1; Affected users: 1; Discoverability: 3; Total: 11

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✘; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Communications with the remote application interface should be done over a secure channel.

Similar attacks in the literature

(Empty in source.)

Embedded / Software / OS & Kernel

GEN-013 · An attacker compromises the real-time clock to disrupt the kernel RT scheduling guarantees.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 2; Total: 11

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Hardened kernel (prevent dynamic loading of kernel modules)
- Ensure only trustable kernels are used (e.g. Secure Boot)
- /boot should not be accessible by robot processes
- [NTP security best practices][ietf_ntp_bcp] should be enforced to ensure no attacker can manipulate the robot computer clock. See also [RFC 7384][rfc_7384], [NTPsec][ntpsec] and [Emerging Solutions in Time Synchronization Security][emerging_solutions_time_sync_sec]. Additionally, [PTP protocol][ieee_1588_2008] can be considered instead of NTP.

Similar attacks in the literature
Dessiatnikoff, Anthony, Yves Deswarte, Eric Alata, and Vincent Nicomette. “Potential Attacks on Onboard Aerospace Systems.” IEEE Security & Privacy 10, no. 4 (July 2012): 71–74.

GEN-014 · An attacker compromises the OS or kernel to alter robot data.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 2; Total: 11

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✓; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- OS user accounts should be properly secured (randomized password or e.g. SSH keys)
- Hardened kernel (prevent dynamic loading of kernel modules)
- Ensure only trustable kernels are used (e.g. Secure Boot)
- /boot should not be accessible by robot processes

Similar attacks in the literature
Clark, George W., Michael V. Doran, and Todd R. Andel. “Cybersecurity Issues in Robotics.” In 2017 IEEE Conference on Cognitive and Computational Aspects of Situation Management (CogSIMA), 1–5. Savannah, GA, USA: IEEE, 2017.

GEN-015 · An attacker compromises the OS or kernel to eavesdrop on robot data.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✓; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 2; Total: 9

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✓; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- OS user accounts should be properly secured (randomized password or e.g. SSH keys)
- Hardened kernel (prevent dynamic loading of kernel modules)
- Ensure only trustable kernels are used (e.g. Secure Boot)
- /boot should not be accessible by robot processes

Similar attacks in the literature

Clark, George W., Michael V. Doran, and Todd R. Andel. “Cybersecurity Issues in Robotics.” In 2017 IEEE Conference on Cognitive and Computational Aspects of Situation Management (CogSIMA), 1–5. Savannah, GA, USA: IEEE, 2017.

GEN-016 · An attacker gains access to the robot OS through its administration interface.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✓; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Administrative interface should be properly secured (e.g. no default/static password).
- Administrative interface should be accessible by a limited number of physical machines. For instance, one may require the user to be physically co-located with the robot (see e.g. ADB for Android)

Similar attacks in the literature

(Empty in source.)

Embedded / Software / Component-Oriented Architecture

GEN-017 · A node accidentally writes incorrect data to a communication channel.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 13

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should always validate received messages.
- Invalid message events should be logged and users should be notified.

Similar attacks in the literature
Jacques-Louis Lions et al. "Ariane S Flight 501 Failure." ESA Press Release 33–96, Paris, 1996.

GEN-018 · An attacker deploys a malicious component on the robot.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should not trust other components (received messages needs to be validated, etc.).
- Users should not be able to deploy components directly.
- Components binary should be digitally signed.
- Components source code should be audited.
- Components should run with minimal privileges (CPU and memory quota, minimal I/O and access to the filesystem)

Similar attacks in the literature

Checkoway, Stephen, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, Stefan Savage, Karl Koscher, Alexei Czeskis, Franziska Roesner, and Tadayoshi Kohno. “Comprehensive Experimental Analyses of Automotive Attack Surfaces.” In Proceedings of the 20th USENIX Conference on Security, 6–6. SEC’11. Berkeley, CA, USA: USENIX Association, 2011.

GEN-019 · An attacker can prevent a component running on the robot from executing normally.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 13

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✘; Data integrity: ✘; Data availability: ✓; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Components should not be trusted and be properly isolated (e.g. run as different users)
- When safe, components should attempt to restart automatically when a fatal error occurs.

Similar attacks in the literature

Dessiatnikoff, Anthony, Yves Deswarte, Eric Alata, and Vincent Nicomette. “Potential Attacks on Onboard Aerospace Systems.” IEEE Security & Privacy 10, no. 4 (July 2012): 71–74.

Embedded / Software / Configuration Management

GEN-020 · An attacker modifies configuration values without authorization.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ▲; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Configuration data access control list should be implemented.
- Configuration data modifications should be logged.
- Configuration write-access should be limited to the minimum set of users and/or components.

Similar attacks in the literature

Ahmad Yousef, Khalil, Anas AlMajali, Salah Ghalyon, Waleed Dweik, and Bassam Mohd. “Analyzing Cyber-Physical Threats on Robotic Platforms.” Sensors 18, no. 5 (May 21, 2018): 1643.

GEN-021 · An attacker accesses configuration values without authorization.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✓; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 13

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Configuration data should be considered as private.
- Configuration data should accessible by the minimum set of users and/or components.

Similar attacks in the literature
Ahmad Yousef, Khalil, Anas AlMajali, Salah Ghalyon, Waleed Dweik, and Bassam Mohd. “Analyzing Cyber-Physical Threats on Robotic Platforms.” Sensors 18, no. 5 (May 21, 2018): 1643.

GEN-022 · A user accidentally misconfigures the robot.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ▲; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Configuration data changes should be reversible.
- Large change should be applied atomically.
- Fault monitoring should be able to automatically reset the configuration to a safe state if the robot becomes unavailable.

Similar attacks in the literature

(Empty in source.)

Embedded / Software / Data Storage (File System)

GEN-023 · An attacker modifies the robot file system by physically accessing it.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Robot filesystem must be encrypted. The key should be stored in a secure enclave (TPM).
- Robot filesystem should be wiped out if the robot is physically compromised.

Similar attacks in the literature

(Empty in source.)

GEN-024 · An attacker eavesdrops on the robot file system by physically accessing it.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✓; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 13

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Robot filesystem must be encrypted. The key should be stored in a secure enclave (TPM).
- Robot filesystem should be wiped out if the robot perimeter is breached.

Similar attacks in the literature

(Empty in source.)

GEN-025 · An attacker saturates the robot disk with data.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 1; Affected users: 3; Discoverability: 3; Total: 13

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✓

Mitigation strategies

- Robot components disk quota should be bounded.
- Disk usage should be properly monitored, logged and reported.
- Optionally, components may have the option to run w/o any file system access. This should be preferred whenever possible.

Similar attacks in the literature

(Empty in source.)

Embedded / Software / Logs

GEN-026 · An attacker exfiltrates log data to a remote server.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 12

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Logs should never contain private data. Log data should be anonymized when needed.
- Logs should be rotated and deleted after a pre-determined retention period.
- Logs should be encrypted in-transit and at-rest.
- Logs access should be ACL protected.
- Logs access should be monitored to enable later audits.

Similar attacks in the literature

(Empty in source.)

Embedded / Hardware / Sensors

GEN-027 · An attacker spoofs a robot sensor (by e.g. replacing the sensor itself or manipulating the bus).

STRIDE

Spoofing: ✓; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 3; Total: 12

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✓; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Sensors should embed an identifier to detect hardware tampering.
- Components should try to explicitly refer to which sensor ID they expect data from.
- Sensor data should be signed and ideally encrypted over the wire.

Similar attacks in the literature

(Empty in source.)

Embedded / Hardware / Actuators

GEN-028 · An attacker spoofs a robot actuator.

STRIDE

Spoofing: ✓; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 3; Total: 10

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✓; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Actuators should embed an identifier.
- Command vector should be signed (ideally encrypted) to prevent manipulation.

Similar attacks in the literature

(Empty in source.)

GEN-029 · An attacker modifies the command sent to the robot actuators. (intercept & retransmit)

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 3; Total: 12

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✓; Deployment infrastructure: ✘

Mitigation strategies

- Actuators should embed an identifier.
- Command vector should be signed (ideally encrypted) to prevent manipulation.

Similar attacks in the literature

(Empty in source.)

GEN-030 · An attacker intercepts the robot actuators command (can recompute localization).

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✓; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 3; Total: 10

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Command vector should be encrypted.

Similar attacks in the literature

(Empty in source.)

GEN-031 · An attacker sends malicious command to actuators to trigger the E-Stop

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 3; Affected users: 3; Discoverability: 1; Total: 11

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- If a joint command is exceeding the joint limits, a specific code path for handling out-of-bounds command should be executed instead of triggering the E-Stop. Whenever safe, the command could be discarded and the error reported to the user for instance.

Similar attacks in the literature

(Empty in source.)

Embedded / Hardware / Auxilliary Functions

GEN-032 · An attacker compromises the software or sends malicious commands to drain the robot battery.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✓; Robot integrity: ✘; Data integrity: ✘; Data availability: ✓; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

- Per-node CPU quota should be enforced.
- Appropriate protection should be implemented to prevent actuators from over-heating.
- If the battery level becomes critically low, the robot should be able to bring itself to a stop.

Similar attacks in the literature

Dessiatnikoff, Anthony, Yves Deswarte, Eric Alata, and Vincent Nicomette. “Potential Attacks on Onboard Aerospace Systems.” IEEE Security & Privacy 10, no. 4 (July 2012): 71–74.

Embedded / Hardware / Communications

GEN-033 · An attacker connects to an exposed debug port.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✓; Information disclosure: ✓; Denial of service: ✓; Elevation of privilege: ✓

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✘; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✓; Robot administration tools: ✓; Remote application interface: ✓; Deployment infrastructure: ✓

Mitigation strategies

- Close the communication port to external communications or disable the service on non-development devices.

Similar attacks in the literature

(Empty in source.)

GEN-034 · An attacker connects to an internal communication bus.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✘; Information disclosure: ▲; Denial of service: ▲; Elevation of privilege: ▲

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 3; Affected users: 3; Discoverability: 3; Total: 15

Impacted assets

Robot compute resources: ▲; Physical safety: ▲; Robot availability: ▲; Robot integrity: ✘; Data integrity: ▲; Data availability: ▲; Data privacy: ▲

Impacted entry points

Embedded hardware: ▲; Robot communication channels: ▲; Robot administration tools: ▲; Remote application interface: ▲; Deployment infrastructure: ▲

Mitigation strategies

Limit access to internal communications and buses.

Similar attacks in the literature

(Empty in source.)

Remote / Client Application

GEN-035 · An attacker intercepts the user credentials on their desktop machine.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✓

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 2; Affected users: 3; Discoverability: 1; Total: 10

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ▲; Robot administration tools: ✘; Remote application interface: ✓; Deployment infrastructure: ✘

Mitigation strategies

- Remote users should be granted minimum privileges
- Credentials on desktop machines should be stored securely (secure enclave, TPM, etc.)
- User credentials should be revokable or expire automatically
- User credentials should be tied to the user identity for audit purposes

Similar attacks in the literature

(Empty in source.)

Remote / Cloud Integration

GEN-036 · An attacker intercepts cloud service credentials deployed on the robot.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✓

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 1; Total: 9

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✘; Data availability: ▲; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✓; Deployment infrastructure: ✘

Mitigation strategies

- Cloud services should be granted minimal privileges.
- Cloud services credentials should be revokable.
- Cloud services should be audited for abuse / unauthorized access.

Similar attacks in the literature

(Empty in source.)

GEN-037 · An attacker gains read access to robot cloud data.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✓

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 1; Total: 9

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✘; Data availability: ▲; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✓; Deployment infrastructure: ✘

Mitigation strategies

- Cloud data stores should encrypt data at rest

Similar attacks in the literature

(Empty in source.)

GEN-038 · An attacker alters or deletes robot cloud data.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 2; Reproducibility: 2; Exploitability: 1; Affected users: 3; Discoverability: 1; Total: 9

Impacted assets

Robot compute resources: ✘; Physical safety: ▲; Robot availability: ✘; Robot integrity: ✓; Data integrity: ✘; Data availability: ▲; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✓; Deployment infrastructure: ✘

Mitigation strategies

- Cloud data should have proper backup mechanisms.
- Cloud data access should be audited. If an intrusion is detected, a process to restore the system back to a previous "uncompromised" state should be available.

Similar attacks in the literature

(Empty in source.)

Remote / Software Deployment

GEN-039 · An attacker spoofs the deployment service.

STRIDE

Spoofing: ✓; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ▲; Robot integrity: ✓; Data integrity: ✓; Data availability: ▲; Data privacy: ▲

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✓

Mitigation strategies

- Deployment service should be authenticated• Communication with the deployment service should be done over a secure channel.

Similar attacks in the literature

(Empty in source.)

GEN-040 · An attacker modifies the binaries sent by the deployment service.

STRIDE

Spoofing: ✘; Tampering: ✓; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 3; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 14

Impacted assets

Robot compute resources: ✘; Physical safety: ✓; Robot availability: ▲; Robot integrity: ✓; Data integrity: ✓; Data availability: ▲; Data privacy: ▲

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✓

Mitigation strategies

- Deployment service should be authenticated• Communication with the deployment service should be done over a secure channel.

Similar attacks in the literature

(Empty in source.)

GEN-041 · An attacker intercepts the binaries sent by the deployment service.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✓; Information disclosure: ✘; Denial of service: ✘; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 12

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✓

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✓

Mitigation strategies

- Deployment service should be authenticated• Communication with the deployment service should be done over a secure channel.

Similar attacks in the literature

(Empty in source.)

GEN-042 · An attacker prevents the robot and the deployment service from communicating.

STRIDE

Spoofing: ✘; Tampering: ✘; Repudiation: ✘; Information disclosure: ✘; Denial of service: ✓; Elevation of privilege: ✘

Historical DREAD scores

Damage: 1; Reproducibility: 3; Exploitability: 2; Affected users: 3; Discoverability: 3; Total: 12

Impacted assets

Robot compute resources: ✘; Physical safety: ✘; Robot availability: ✘; Robot integrity: ✘; Data integrity: ✘; Data availability: ✘; Data privacy: ✘

Impacted entry points

Embedded hardware: ✘; Robot communication channels: ✘; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✓

Mitigation strategies

(Empty in source.)

Similar attacks in the literature

(Empty in source.)

Cross-Cutting Concerns / Credentials, PKI and Secrets

GEN-043 · An attacker compromises a Certificate Authority trusted by the robot.

STRIDE

Spoofing: ✓; Tampering: ✓; Repudiation: ✘; Information disclosure: ✓; Denial of service: ✘; Elevation of privilege: ✓

Historical DREAD scores

Damage: 3; Reproducibility: 1; Exploitability: 1; Affected users: 2; Discoverability: 3; Total: 10

Impacted assets

Robot compute resources: ✓; Physical safety: ✓; Robot availability: ✓; Robot integrity: ✓; Data integrity: ✓; Data availability: ✓; Data privacy: ✓

Impacted entry points

Embedded hardware: ✓; Robot communication channels: ✓; Robot administration tools: ✘; Remote application interface: ✘; Deployment infrastructure: ✘

Mitigation strategies

(Empty in source.)

Similar attacks in the literature

(Empty in source.)

Including a new robot into the threat model

The following steps are recommended in order to extend this document with additional threat models:

1. Determine the robot evaluation scenario. This will include: * System description and specifications * Data assets
2. Define the robot environment: * External actors * Entry points * Use cases
3. Design security boundary and architectural schemas for the robotic application.
4. Evaluate and prioritize entry points * Make use of the RSFrsf to find applicable weaknesses on the robot. * Take existing documentation as help for finding applicable entry points.
5. Evaluate existing threats based on general threat table and add new ones to the specific threat table. * Evaluate new threats with DREADwikipedia_dread and STRIDEwikipedia_stride methodologies.

6. Design hypothetical attack trees for each of the entry points, detailing the affected resources on the process.
7. Create a Pull Request and submit the changes to the ros2/design repository.

Threat Analysis for the `TurtleBot 3` Robotic Platform

System description

The application considered in this section is tele-operation of a Turtlebot 3 robot using an Xbox controller.

The robot considered in this section is a TurtleBot 3 Burger. It is a small educational robot embedding an IMU and a Lidar, two motors / wheels, and a chassis that hosts a battery, a Pi Raspberry Pi 3 Model B+ Single Board Computer, and a OpenCR 1.0 Arduino compatible board that interacts with the sensors. The robot computer runs two nodes:

- `turtlebot3_node` forwarding sensor data and actuator control commands,
- `raspicam2_node` which is forwarding camera data

We also make the assumption that the robot is running a ROS 2 port of the AWS CloudWatch sample application). A ROS 2 version of this component is not yet available, but it will help us demonstrate the threats related to connecting a robot to a cloud service.

For the purpose of demonstrating the threats associated with distributing a ROS graph among multiple hosts, an Xbox controller is connected to a secondary computer (“remote host”). The secondary computer runs two additional nodes:

- `joy_node` is forwarding joystick input as ROS 2 messages,
- `teleop_twist_joy` is converting ROS 2 joystick messages to control commands.

Finally, the robot data is accessed by a test engineer through a “field testing” computer.

Architecture Dataflow diagram

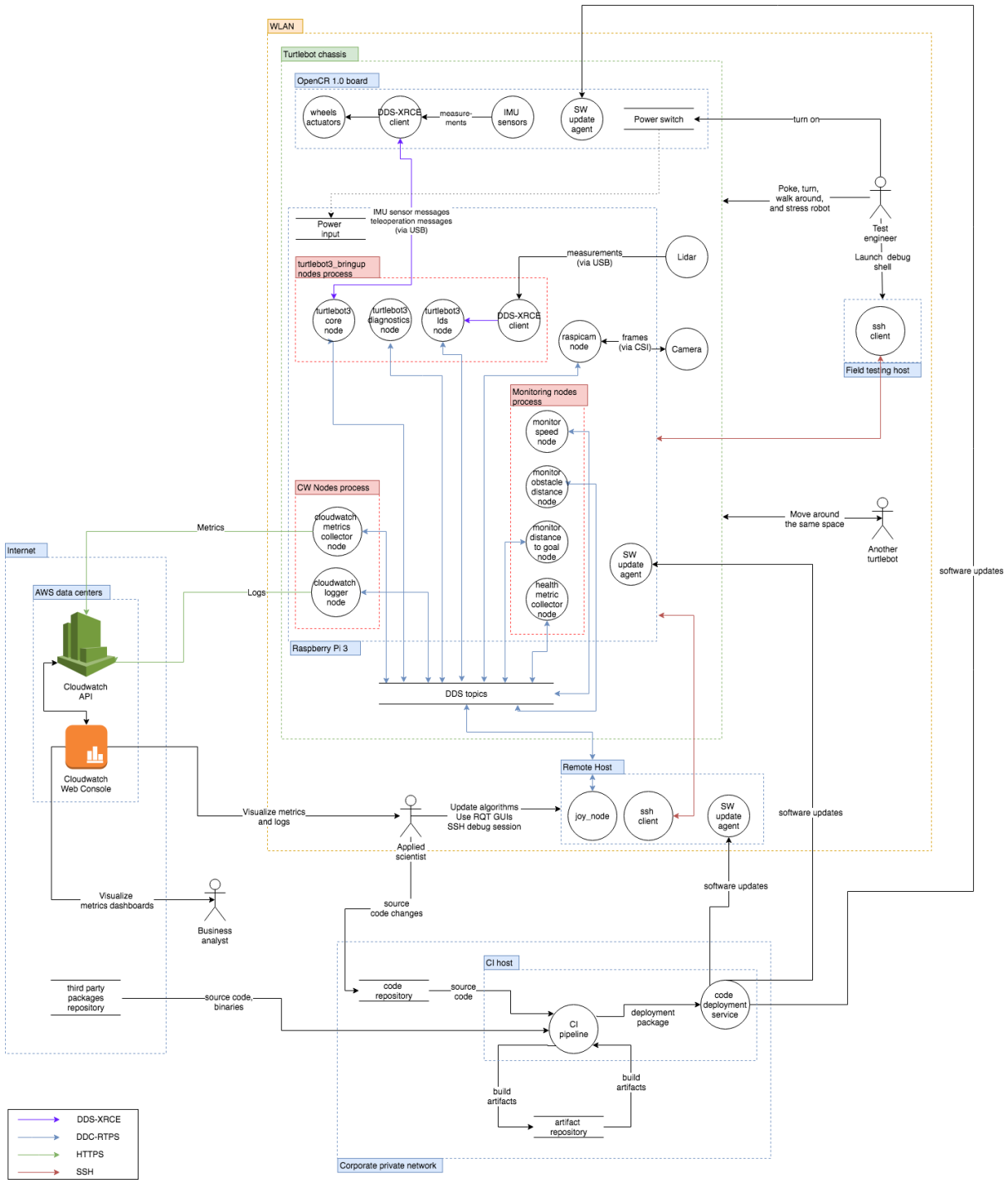


Figure 2 · ROS2_Application.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

Assets

Hardware

- TurtleBot 3 Burger is a small, ROS-enabled robot for education purposes.

- Compute resources
  - Raspberry PI 3 host: OS Raspbian Stretch with ROS 2 Crystal running natively (without Docker), running as root.
  - OpenCR board: using ROS 2 firmware as described in the TurtleBot3 ROS 2 setup instructions.
- Hardware components) include:
  - The Lidar is connected to the Raspberry PI through USB.
  - A Raspberry PI camera module is connected to the Raspberry PI 3 through its Camera Serial Interface (CSI).
- Field testing host: conventional laptop running OS Ubuntu 18.04, no ROS installed. Running as a sudoer user.
- Remote Host: any conventional server running OS Ubuntu 18.04 with ROS 2 Crystal.
- CI host: any conventional server running OS Ubuntu 18.04.
- WLAN: a wifi local area network without security enabled, open for anyone to connect
- Corporate private network: a secure corporate wide-area network, that spans multiple cities. Only authenticated user with suitable credentials can connect to the network, and good security practices like password rotation are in place.

Processes

- Onboard TurtleBot3 Raspberry Pi
- `turtlebot3_node`
- CloudWatch nodes (hypothetical as it has not been yet ported to ROS 2)
  - `cloudwatch_metrics_collector`: subscribes to a the /metrics topic where other nodes publish MetricList messages that specify CloudWatch metrics data, and sends the corresponding metric data to CloudWatch metrics using the PutMetricsData API.
  - `cloudwatch_logger`: subscribes to a configured list of topics, and publishes all messages found in that topic to a configured log group and log stream, using the CloudWatch metrics API.
- Monitoring Nodes
  - `monitor_speed`: subscribes to the topic /odom, and for each received odometry message, extract the linear and angular speed, build a MetricList message with those values, and publishes that message to data to the /metrics topic.
  - `health_metric_collector`: collects system metrics (free RAM, total RAM, total CPU usage, per core CPU usage, uptime, number of processes) and publishes it as a MetricList message to the /metrics topic.
- `raspicam2_node` a node publishing Raspberry Pi Camera Module data to ROS 2.
- An XRCE Agent runs on the Raspberry, and is used by a DDS-XRCE client running on the OpenCR 1.0 board, that publishes IMU sensor data to ROS topics, and controls the wheels of the TurtleBot based on the teleoperation messages published as ROS topics. This channel uses serial communication.
- A Lidar driver process running on the Raspberry interfaces with the Lidar, and uses a DDS-XRCE client to publish data over UDP to an XRCE agent also running on the Raspberry Pi. The agent sends the sensor data to several ROS topics.
- An SSH client process is running in the field testing host, connecting to the Raspberry PI for diagnostic and debugging.
- A software update agent process is running on the Raspberry PI, OpenCR board, and navigation hosts. The agent polls for updates a code deployment service process running on the CI host, that responds with a list of packages and versions, and a new version of each package when an update is required.

- The CI pipeline process on the CI host is a Jenkins instance that is polling a code repository for new revisions of a number of ROS packages. Each time a new revision is detected it rebuilds all the affected packages, packs the binary artifacts into several deployment packages, and eventually sends the package updates to the update agent when polled.

Software Dependencies

- OS / Kernel
- Ubuntu 18.04
- Software
- ROS 2 Core Libraries
- ROS 2 Nodes: `joy_node`, `turtlebot3_node`, `rospicam2_node`, `teleop_twist_joy`
- ROS 2 system dependencies as defined by rosdep:
  - RMW implementation: we assume the RMW implementation is `rmw_fastrtps`.
  - The threat model describes attack with the security enabled or disabled. If the security is enabled, the security plugins are assumed to be configured and enabled.

See TurtleBot3 ROS 2 setup instructions for details about TurtleBot3 software dependencies.

External Actors

- A test engineer is testing the robot.
- A user operates the robot with the joystick.
- A business analyst periodically checks dashboards with performance information for the robot in the AWS Cloudwatch web console.

Robot Data Assets

- Topic Message
- Private Data
  - Camera image messages
  - Logging messages (might describe camera data)
  - CloudWatch metrics and logs messages (could contain Intellectual Property such as which algorithms are implemented in a particular node).
  - Restricted Data
  - Robot Speed and Orientation. Some understanding of the current robot task may be reconstructed from those messages..
- AWS CloudWatch data
- Metrics, logs, and aggregated dashboard are all private data, as they are different serializations of the corresponding topics.
- Raspberry Pi System and ROS logs on Raspberry PI
- Private data just like CloudWatch logs.
- AWS credentials on all hosts
- Secret data, provide access to APIs and other compute assets on the AWS cloud.
- SSH credentials on all hosts
- Secret data, provide access to other hosts.
- Robot embedded algorithm (e.g. `teleop_twist_joy`)
- Secret data. Intellectual Property (IP) theft is a critical issue for nodes which are either easy to decompile or written in an interpreted language.

Robot Compute Assets

- AWS CloudWatch APIs and all other AWS resources accessible from the AWS credentials present in all the hosts.
- Robot Topics
- `/cmd_vel` could be abused to damage the robot or hurt users.

Entry points

- Communication Channels
- DDS / ROS Topics
  - Topics can be listened or written to by any actor: 1. Connected to a network where DDS packets are routed to, 1. Have necessary permissions (read / write) if SROS is enabled.
  - When SROS is enabled, attackers may try to compromise the CA authority or the private keys to generate or intercept private keys as well as emitting malicious certificates to allow spoofing.
  - USB connection is used for communication between Raspberry Pi and OpenCR board and LIDAR sensor.
- SSH
  - SSH access is possible to anyone on the same LAN or WAN (if port-forwarding is enabled). Many images are setup with a default username and password with administrative capabilities (e.g. sudoer).
  - SSH access can be compromised by modifying the robot
- Deployed Software
- ROS nodes are compiled either by a third-party (OSRF build-farm) or by the application developer. It can be compiled directly on the robot or copied from a developer workstation using scp or rsync.
- An attacker compromising the build-farm or the developer workstation could introduce a vulnerability in a binary which would then be deployed to the robot.
- Data Store (local filesystem)
- Robot Data
  - System and ROS logs are stored on the Raspberry Pi filesystem.
  - Robot system is subject to physical attack (physically removing the disk from the robot to read its data).
- Remote Host Data
  - Machines running additional ROS nodes will also contain log files.
  - Remote hosts is subject to physical attack. Additionally, this host may not be as secured as the robot host.
- Cloud Data
  - AWS CloudWatch data is accessible from public AWS API endpoints, if credentials are available for the corresponding AWS account.
  - AWS CloudWatch can be credentials can allow attackers to access other cloud resources depending on how the account has been configured.
- Secret Management
  - DDS / ROS Topics
  - If SROS is enabled, private keys are stored on the local filesystem.
  - SSH
  - SSH credentials are stored on the robot filesystem.
  - Private SSH keys are stored in any computer allowed to log into the robot where public/private keys are relied on for authentication purposes.

    - AWS Credentials
    - AWS credentials are stored on the robot file system.

Use case scenarios

- Development, Testing and Validation
- An engineer develops code or runs tests on the robot. They may:
    - Restart the robot
    - Restart the ROS graph
    - Physically interact with the robot
    - Log into the robot using SSH
    - Check AWS console for metrics and log
- End-User
- A user tele-operates the robot. They may:
    - Start the robot.
    - Control the joystick.
- A business analyst may access AWS CloudWatch data on the console to assess the robot performance.

Threat model

Each generic threat described in the previous section can be instantiated on the TurtleBot 3.

This table indicates which TurtleBot particular assets and entry points are impacted by each threat. A check sign (✓) means impacted while a cross sign (✘) means not impacted. The "SROS Enabled?" column explicitly states out whether using SROS would mitigate the threat or not. A check sign (✓) means that the threat could be exploited while SROS is enabled while a cross sign (✘) means that the threat requires SROS to be disabled to be applicable.

> Historical matrix reflowed as vertical records. ✓, ✘ and ▲ retain their source meanings. SROS Enabled? means the attack can occur with SROS enabled, not that SROS prevents it. Scores and mitigation claims are historical and unverified in this review.

Embedded / Software / Communication / Inter-Component Communication

---

TB3-001 · An attacker spoofs a component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS any node may have any name so spoofing is trivial.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result
Risk is reduced if SROS is used.

Additional notes / open questions
Authentication codes need to be generated for every pair of participants rather than every participant sharing a single authentication code with all other participants. Refer to section 7.1.1.3 of the DDS Security standard.

TB3-002 · An attacker spoofs a component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
An attacker deploys a malicious node which is not enabling DDS Security Extension and spoofs the `joy_node` forcing the robot to stop.

Mitigation

- DDS Security Governance document must set `allow_unauthenticated_participants` to False to avoid non-authenticated participants to be allowed to communicate with authenticated nodes.
- DDS Security Governance document must set `enable_join_access_control` to True to explicitly whitelist node-to-node-communication. permissions.xml should be as restricted as possible.

Mitigation result
Risk is mitigated.

Additional notes / open questions

- Which actions would still be possible even with restrictive permission set? (e.g. does a node have the ability to fetch parameters from the parameter server?
- How about set parameters? Query other node's lifecycle state? Etc.).

TB3-003 · An attacker spoofs a component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
An attacker steals node credentials and spoofs the `joy_node` forcing the robot to stop.

Mitigation

- Store node credentials in a secure location (secure enclave, RoT) to reduce the probability of having a private key leaked.
- Run nodes in isolated sandboxes to ensure one node cannot access another node data (including credentials)
- Permissions CA should digitally sign nodes binaries to prevent running tampered binaries.
- Permissions CA should be able to revoke certificates in case credentials get stolen.

Mitigation result
Mitigation risk requires additional work.

Additional notes / open questions

- AWS Robotics and Automation is currently evaluating the feasibility of storing DDS-Security credentials in a TPM.
- Complete mitigation would require isolation using e.g. Snap or Docker.
- Deploying an application with proper isolation would require us to revive discussions around [ROS 2 launch system][ros2_launch_design_pr]
- Yocto / OpenEmbedded / Snap support should be considered

TB3-004 · An attacker intercepts and alters a message.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS an attacker can modify `/cmd_vel` messages sent through a network connection to e.g. stop the robot.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications. Message tampering is mitigated by DDS security as message authenticity is verified by default (with preshared HMACs / digital signatures)

Mitigation result
Risk is reduced if SROS is used.

Additional notes / open questions
Additional hardening could be implemented by forcing part of the TurtleBot topic to only be communicated over shared memory.

TB3-005 · An attacker writes to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS, any node can publish to any topic.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

TB3-006 · An attacker writes to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
An attacker obtains credentials and publishes messages to `/cmd_vel`

Mitigation

- `permissions.xml` must be kept as closed as possible.
- Publication to sensitive topics permission must only be granted to a limited set of nodes the robot can trust.
- ROS nodes must run in sandboxes to prevent interferences"

Mitigation result
Transfer risk to user configuring permissions.xml

Additional notes / open questions
`permissions.xml` should ideally be generated as writing it manually is cumbersome and error-prone.

TB3-007 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS: any node can listen to any topic.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result
Risk is mitigated if SROS is used.

Additional notes / open questions

(Empty in source.)

TB3-008 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
DDS participants are enumerated and fingerprinted to look for potential vulnerabilities.

Mitigation

- DDS Security Governance document must set `metadata_protection_kind` to ENCRYPT to prevent malicious actors from observing communications.
- DDS Security Governance document must set `enable_discovery_protection` to True to prevent malicious actors from enumerating and fingerprinting DDS participants.
- DDS Security Governance document must set `enable_liveliness_protection` to True

Mitigation result
Risk is mitigated if DDS-Security is configured appropriately.

Additional notes / open questions

(Empty in source.)

TB3-009 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
TurtleBot camera images are saved to a remote location controlled by the attacker.

Mitigation

- DDS Security Governance document must set `metadata_protection_kind` to ENCRYPT to prevent malicious actors from observing communications.
- DDS Security Governance document must set `enable_discovery_protection` to True to prevent malicious actors from enumerating and fingerprinting DDS participants.
- DDS Security Governance document must set `enable_liveliness_protection` to True

Mitigation result
Risk is mitigated if DDS-Security is configured appropriately.

Additional notes / open questions

(Empty in source.)

TB3-010 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
TurtleBot LIDAR measurements are saved to a remote location controlled by the attacker.

Mitigation
Local communication using XRCE should be done over the loopback interface.

Mitigation result
This doesn't protect the serial communication from the LIDAR sensor to the Raspberry Pi.

Additional notes / open questions

(Empty in source.)

TB3-011 · An attacker prevents a communication channel from being usable.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS: any node can ""spam"" any other component.

Mitigation

- Enable SROS to use the DDS Security Extension. This does not prevent nodes from being flooded but it ensures that only communication from allowed participants are processed.

Mitigation result
Risk may be reduced when using SROS.

Additional notes / open questions

(Empty in source.)

TB3-012 · An attacker prevents a communication channel from being usable.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
A node can ""spam"" another node it is allowed to communicate with.

Mitigation

- Implement rate limitation on topics
- Define a method for topics to declare their required bandwidth / rate.

Mitigation result
Mitigating risk requires additional work.

Additional notes / open questions
How to enforce when nodes are malicious? Observe and kill?

Embedded / Software / Communication / Long-Range Communication (e.g. WiFi, Cellular Connection)

TB3-013 · An attacker hijacks robot long-range communication

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker connects to the same unprotected WiFi network than a TurtleBot.

Mitigation

- Prevent TurtleBot from connecting to non-protected WiFi network
- SROS should always be used for long-range DDS communication.

Mitigation result
Risk is reduced for DDS if SROS is used. Other protocols may still be vulnerable.

Additional notes / open questions
Enforcing communication though a VPN could be an idea (see PR2 manual for a reference implementation) or only DDS communication could be allowed on long-range links (SSH could be replaced by e.g. adbd and be only available using a dedicated short-range link).

TB3-014 · An attacker intercepts robot long-range communications (e.g. MitM)

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker connects to the same unprotected WiFi network than a TurtleBot.

Mitigation

- Prevent TurtleBot from connecting to non-protected WiFi network
- SROS should always be used for long-range DDS communication.

Mitigation result
Risk is reduced for DDS if SROS is used. Other protocols may still be vulnerable.

Additional notes / open questions

(Empty in source.)

TB3-015 · An attacker disrupts (e.g. jams) robot long-range communication channels.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
Jam the WiFi network a TurtleBot is connected to.

Mitigation
If network connectivity is lost, switch to cellular network.

Mitigation result
Mitigating is impossible on TurtleBot (no secondary long-range communication system).

Additional notes / open questions

(Empty in source.)

Embedded / Software / Communication / Short-Range Communication (e.g. Bluetooth)

TB3-016 · An attacker executes arbitrary code using a short-range communication protocol vulnerability.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
Attacker runs a blueborne attack to execute arbitraty code on the TurtleBot Raspberry Pi.

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions
A potential mitigation may be to build a minimal kernel with e.g. Yocto which does not enable features the robot does not use. In this particular case, the TurtleBot does not require Bluetooth but OS images enable it by default.

Embedded / Software / OS & Kernel

TB3-017 · An attacker compromises the real-time clock to disrupt the kernel RT scheduling guarantees.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious node attempts to write a compromised kernel to `/boot`

Mitigation

(Empty in source.)

Mitigation result
TurtleBot / Zymbit Key integration will mostly mitigate this threat.

Additional notes / open questions
Some level of mitigation will be possible through Turtlebot / Zymkey integration. SecureBoot support would probably be needed to completely mitigate this threat.

TB3-018 · An attacker compromises the real-time clock to disrupt the kernel RT scheduling guarantees.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious actor sends incorrect NTP packages to enable other attacks (allow the use of expired certificates) or to prevent the robot software from behaving properly (time between sensor readings could be miscomputed).

Mitigation

- [Implement NTP Best Practices][ntp_best_practices]
- Use a hardware RTC clock such as the one provided by the Zymbit key to reduce the system reliance on NTP.
- If your robot relies on GPS for localization purpose, consider using time from your GPS received (note that it opens the door to other vulnerabilities such as GPS jamming).

Mitigation result
TurtleBot / Zymbit Key integration and following best practices for NTP configuration will mostly mitigate this threat.

Additional notes / open questions

(Empty in source.)

TB3-019 · An attacker compromises the OS or kernel to alter robot data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious node attempts to write a compromised kernel to `/boot`

Mitigation

(Empty in source.)

Mitigation result
TurtleBot / Zymbit Key integration will mostly mitigate this threat.

Additional notes / open questions

(Empty in source.)

TB3-020 · An attacker compromises the OS or kernel to eavesdrop on robot data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious node attempts to write a compromised kernel to `/boot`

Mitigation

(Empty in source.)

Mitigation result
TurtleBot / Zymbit Key integration will mostly mitigate this threat.

Additional notes / open questions

(Empty in source.)

TB3-021 · An attacker gains access to the robot OS through its administration interface.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker connects to the TurtleBot using Raspbian default username and password (pi / raspberry pi)

Mitigation

- Default password should be changed and whenever possible password based authentication should be replaced by SSH key based authentication

Mitigation result
Demonstrating a mitigation would require significant work. Building out an image using Yocto w/o a default password could be a way forward.

Additional notes / open questions
SSH is significantly increasing the attack surface. Adbd may be a better solution to handle administrative access.

TB3-022 · An attacker gains access to the robot OS through its administration interface.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker intercepts SSH credentials.

Mitigation

- Disable password-based auth for SSH. Rotate SSH keys periodically for all hosts.
- Set up a [bastion hosts][wikipedia_bastion_hosts] that are the only host that can connect to the navigation and Raspberry PI hosts. Distribute temporary credentials for accessing the bastion hosts through some well known secure [federated identity][wikipedia_federated_identity] system.
- Whenever possible administrative interfaces should only be accessible by users colocated with the robot. In this case, ssh can be replaced by e.g. [an adbd implementation for Linux][adbd_linux]

Mitigation result
Demonstrating a mitigation based on replacing sshd by adbd would require significant work.

Additional notes / open questions

(Empty in source.)

Embedded / Software / Component-Oriented Architecture

TB3-023 · A node accidentally writes incorrect data to a communication channel.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot `joy_node` could accidentally publish large or randomized joystick values.

Mitigation

- Expand DDS IDL to allow users to embed validity criteria to automate input sanitization (i.e. validate ranges, etc.)
- Expand RMW to define per-topic strategies for invalid messages (drop, throw, abort, etc.)

Mitigation result
Mitigation would require significant amount of engineering.

Additional notes / open questions
This task would require designing and implementing a new system from scratch in ROS 2 to document node interactions.

TB3-024 · An attacker deploys a malicious node on the robot.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot is running a malicious `joy_node`. This node steals robot credentials try to kill other processes fill the disk with random data.

Mitigation

- Modify TurtleBot launch scripts to run each node as a different user (or using some kind of containerization?)
- Each ROS user should have very limited permissions and quota should be configured.

Mitigation result
It is possible to mitigate this attack on the TurtleBot by creating multiple user accounts.

Additional notes / open questions
While the one-off solution involving creating user accounts manually is interesting, it would be better to generalize this through some kind of mechanism (e.g. AppArmor / SecComp support for roslaunch)

TB3-025 · An attacker deploys a malicious node on the robot.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot `joy_node` repository is compromised by an attacker and malicious code is added to the repository. `joy_node` is released again and the target TurtleBot is compromised when the node gets updated.

Mitigation

- For tampering a package on a third-party repository: set up a third-party package validation process so packages are not consumed directly from a third party, but instead each package version has to be approved and imported into a private trusted repository:
  - During import packages can be manually inspected and only downloaded from trusted official sources accessed through secure protocols.
  - Prefer importing from source instead of in binary formats and periodically run automatic analysis tools on the source code.
  - Track external version identifier for each imported package and flag an imported package version when a vulnerability is discovered on the same version of the original third party package.
  - Review the third package license with your legal team as part of the import process to ensure compliance with your company's legal requirements.
- For tampering a package on a private code repository or a package artifact on an artifact repository: use standard mitigations for data stores including encryption of data at rest (server side encryption) and in transit (HTTPS/TLS) implement authorization and use use least privilege permissions to access to the data store restrict network access to least privilege.
- For tampering a package artifact in flight during deployment: besides in transit encryption (HTTPS/TLS) use [code signing][wikipedia_code_signing] (e.g. with [AWS IoT code signing API][aws_code_signing]) so hosts verify the package origin and integrity before installing them. This typically requires deploying a [public key][wikipedia_public_key] for the signing authority to the update agents on all hosts that receive software updates.

Mitigation result
Risk is mitigated by enforcing processes around source control but the end-to-end story w.r.t deployment is still unclear.

Additional notes / open questions
Mitigation of this threat is also detailed in the diagram below this table.

TB3-026 · An attacker can prevent a component running on the robot from executing normally.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious TurtleBot joy_node is randomly killing other nodes to disrupt the robot behavior.

Mitigation

- TurtleBot launch files should set the exit_handler to restart_exit_handler.
- ros2 node kill is too successous for production use cases. There should be mechanisms for safely asking a node to restart and/or forcing a process to be killed as a last resort. Those topics should be protected using DDS permissions to severely restrict the users able to run those operations (or fully disable those features for some particular scenario).

Mitigation result
Risk is mitigated for the TurtleBot platform. This approach is unlikely to scale with robots whose nodes cannot be restarted at any time safely though.

Additional notes / open questions
Systemd should be investigated as another way to handle and/or run ROS nodes.

Embedded / Software / Configuration Management

TB3-027 · An attacker modifies configuration values without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✘

SROS Enabled? (historical legend)

(Empty in source.)

Attack
Node parameters are freely modifiable by any DDS domain participant.

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-028 · An attacker access configuration values without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✘

SROS Enabled? (historical legend)

(Empty in source.)

Attack
Node parameters values can be read by any DDS domain participant.

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-029 · A user accidentally misconfigures the robot.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)

(Empty in source.)

Attack
Node parameters values can be modified by anyone to any value.

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Embedded / Software / Data Storage (File System)

TB3-030 · An attacker modifies the robot file system by physically accessing it.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot SD card is physically accessed and a malicious daemon in installed.

Mitigation
Use a TPM ZymKey to encrypt the filesystem (using dm-crypt / LUKS)

Mitigation result
TurtleBot / Zymbit Key integration will mitigate this threat.

Additional notes / open questions

(Empty in source.)

TB3-031 · An attacker eavesdrops on the robot file system by physically accessing it.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot SD card is cloned to steal robot logs, credentials, etc.

Mitigation
Use a TPM ZymKey to encrypt the filesystem (using dm-crypt / LUKS)

Mitigation result
TurtleBot / Zymbit Key integration will mitigate this threat.

Additional notes / open questions

(Empty in source.)

TB3-032 · An attacker saturates the robot disk with data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot is running a malicious `joy_node`. This node steals robot credentials try to kill other processes fill the disk with random data.

Mitigation

- Modify TurtleBot launch scripts to run each node as a different user (or using some kind of containerization?)
- Each ROS user should have very limited permissions and quota should be configured.
- `joy_node` should run w/o any file system access at all

Mitigation result

(Empty in source.)

Additional notes / open questions
A separate user account with disk quota enabled may be a solution but this needs to be demonstrated.

Embedded / Software / Logs

TB3-033 · An attacker exfiltrates log data to a remote server.

Impacted assets

Human assets: ✘; Robot application: ✘

Impacted entry points

DDS topic: ✘; OSRF build farm: ✓; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot logs are exfiltered by a malicious joy_node.

Mitigation

- TurtleBot nodes should be properly isolated and log into files only they can access.
- TurtleBot filesystem should be encrypted (dm-crypt / LUKS).

Mitigation result
Risk is mitigated by limiting access to logs. However, how to run handle logs once stored requires more work. Ensuring no sensitive data is logged is also a challenge.

Additional notes / open questions
Production nodes logs should be scanned to detect leakage of sensitive information.

Embedded / Hardware / Sensors

TB3-034 · An attacker spoofs a robot sensor (by e.g. replacing the sensor itself or manipulating the bus).

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
The USB connection of the TurtleBot laser scanner is modified to be turned off under some conditions.

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Embedded / Hardware / Actuators

TB3-035 · An attacker spoofs a robot actuator.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot connection to the OpenCR board is intercepted motor control commands are altered.

Mitigation
Enclose OpenCR board and Raspberry Pi with a case and use ZimKey perimeter breach protection.

Mitigation result
Robot can be rendered inoperable if perimeter is breached.

Additional notes / open questions

(Empty in source.)

TB3-036 · An attacker modifies the command sent to the robot actuators. (intercept & retransmit)

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot connection to the OpenCR board is intercepted, motor control commands are altered.

Mitigation
Enclose OpenCR board and Raspberry Pi with a case and use ZimKey perimeter breach protection.

Mitigation result
Robot can be rendered inoperable if perimeter is breached.

Additional notes / open questions

(Empty in source.)

TB3-037 · An attacker intercepts the robot actuators command (can recompute localization).

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
TurtleBot connection to the OpenCR board is intercepted motor control commands are logged.

Mitigation
Enclose OpenCR board and Raspberry Pi with a case and use ZimKey perimeter breach protection.

Mitigation result
Robot can be rendered inoperable if perimeter is breached.

Additional notes / open questions

(Empty in source.)

TB3-038 · An attacker sends malicious command to actuators to trigger the E-Stop

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Embedded / Hardware / Ancillary Functions

TB3-039 · An attacker compromises the software or sends malicious commands to drain the robot battery.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✘/✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Remote / Client Application

TB3-040 · An attacker intercepts the user credentials on their desktop machine.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack
A roboticist connects to the robot with Rviz from their development machine. Another user with root privileges steals the credentials.

Mitigation

- On Mac: SROS credentials should be stored in OS X Secure Enclave"

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Remote / Cloud Integration

TB3-041 · An attacker intercepts cloud service credentials deployed on the robot.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack
TurtleBot CloudWatch node credentials are extracted from the filesystem through physical access.

Mitigation

- Reduce blast radius using IAM roles with minimal permissions and expiring credentials following AWS security best practices.
- Distribute temporary AWS credentials through some well known secure federated identity system.

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-042 · An attacker alters or deletes robot cloud data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✘; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack
TurtleBot CloudWatch cloud data is accessed by an unauthorized user.

Mitigation

- Setup encryption in Cloud Watch logs.

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-043 · An attacker alters or deletes robot cloud data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✘; OSRF build farm: ✘; SSH: ✘

SROS Enabled? (historical legend)
✓

Attack
TurtleBot CloudWatch data is deleted by an attacker.

Mitigation

- Cloud data backup should be realized on a regular basis.

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Remote / Software Deployment

TB3-044 · An attacker spoofs the deployment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-045 · An attacker modifies the binaries sent by the deployment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-046 · An attacker intercepts the binaries sent by the depoyment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

TB3-047 · An attacker prevents the robot and the deployment service from communicating.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

DDS topic: ✓; OSRF build farm: ✓; SSH: ✓

SROS Enabled? (historical legend)
✓

Attack

(Empty in source.)

Mitigation

(Empty in source.)

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

Threat Diagram: An attacker deploys a malicious node on the robot

This diagram details how code is built and deployed for the “An attacker deploys a malicious node on the robot” attack.

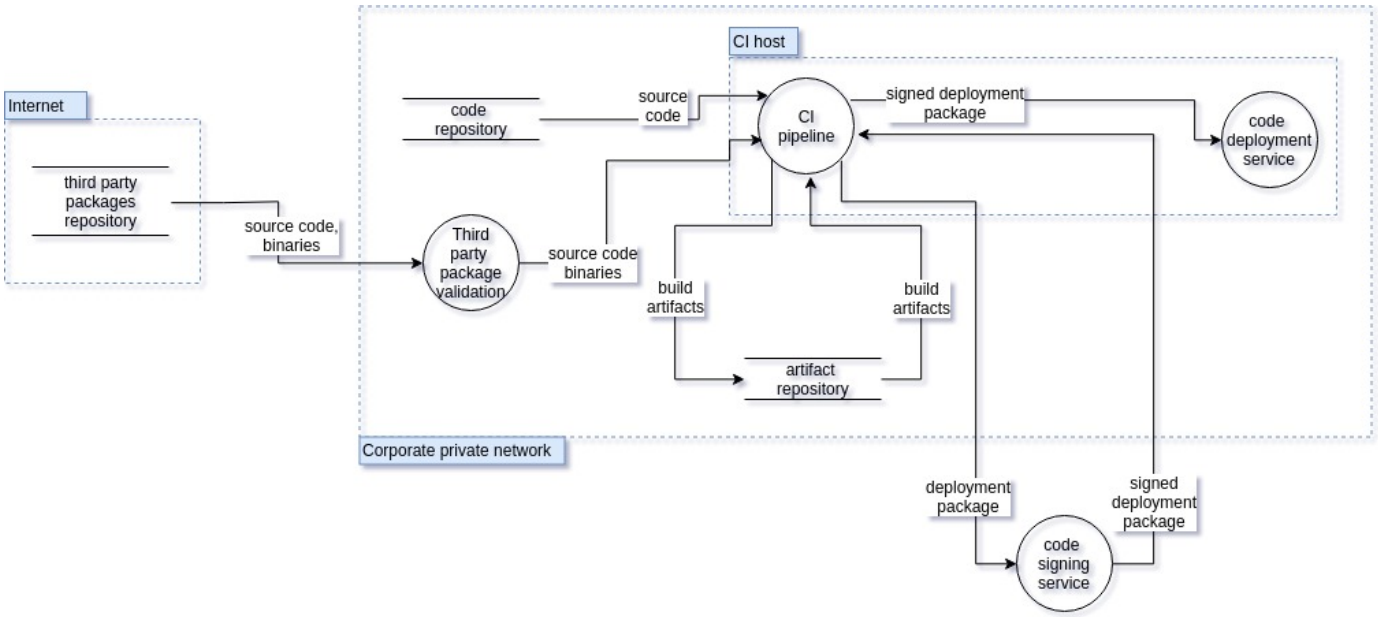


Figure 3 · signing_service_mitigation.jpg. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

Attack Tree

Threats can be organized in the following attack tree.

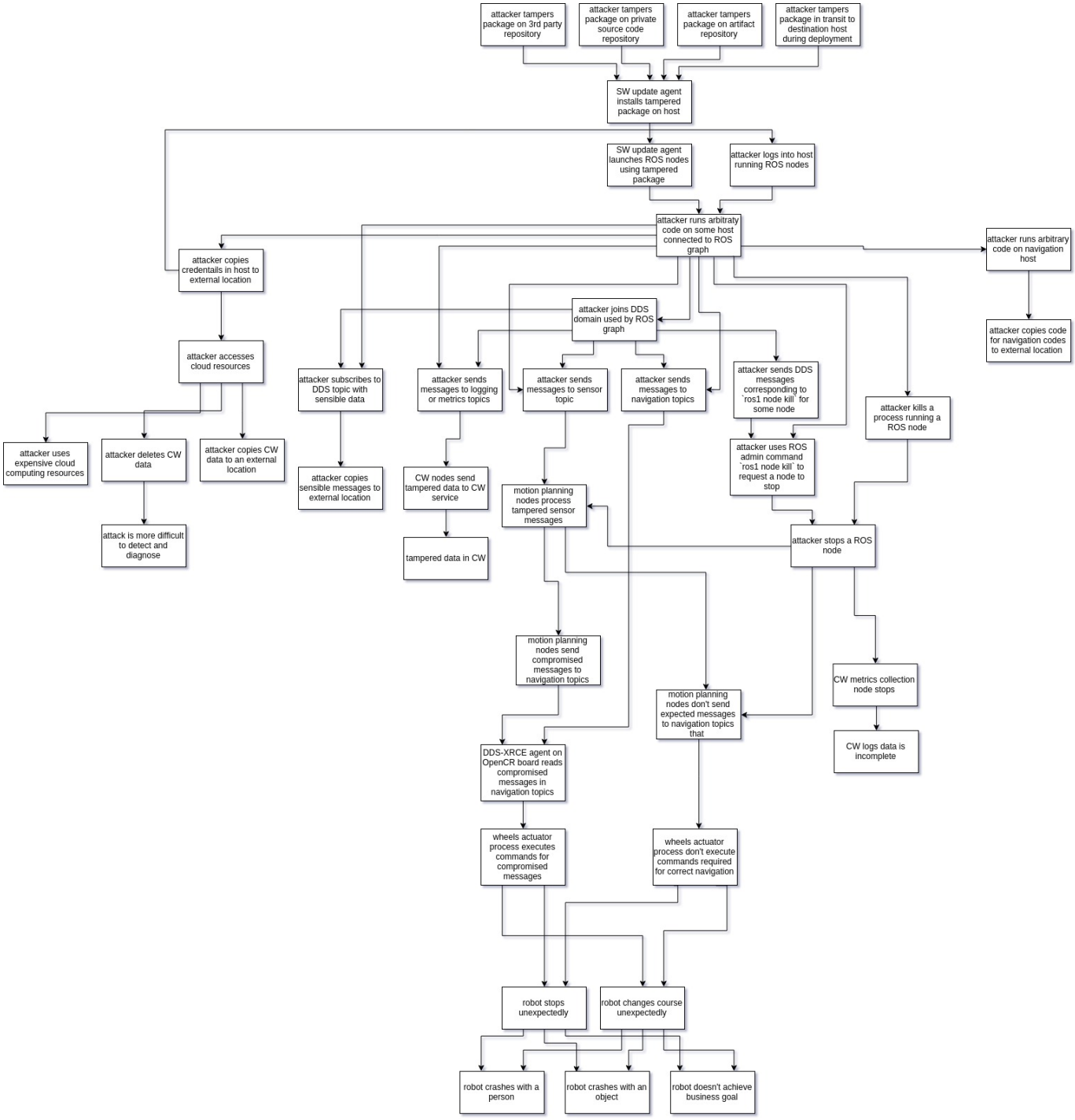


Figure 4 · attack_tree.jpg. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

Threat Model Validation Strategy

Validating this threat model end-to-end is a long-term effort meant to be distributed among the whole ROS 2 community. The threats list is made to evolve with this document and additional reference may be introduced in the future.

1. Setup a TurtleBot with the exact software described in the TurtleBot3 section of this document.

2. Penetration Testing * Attacks described in the spreadsheet should be implemented. For instance, a malicious `joy_node` could be implemented to try to disrupt the robot operations. * Once the vulnerability has been exploited, the exploit should be released to the community so that the results can be reproduced. * Whether the attack has been successful or not, this document should be updated accordingly. * If the attack was successful, a mitigation strategy should be implemented. It can either be done through improving ROS 2 core packages or it can be a platform-specific solution. In the second case, the mitigation will serve as an example to publish best practices for the development of secure robotic applications.

Over time, new reference platforms may be introduced in addition to the TurtleBot 3 to define new attacks and allow other types of mitigations to be evaluated.

Threat Analysis for the `MARA` Robotic Platform

System description

The application considered in this section is a **MARA modular robot operating on an industrial environment while performing a pick & place activity**. MARA is the first robot to run ROS 2 natively. It is an industrial-grade collaborative robotic arm which runs ROS 2 on each joint, end-effector, external sensor or even on its industrial controller. Throughout the H-ROS communication bus, MARA is empowered with new possibilities in the professional landscape of robotics. It offers millisecond-level distributed bounded latencies for usual payloads and submicrosecond-level synchronization capabilities across ROS 2 components.

Built out of individual modules that natively run on ROS 2, MARA can be physically extended in a seamless manner. However, this also moves towards more networked robots and production environments which brings new challenges, especially in terms of security and safety.

The robot considered is a MARA, a 6 Degrees of Freedom (6DoF) modular and collaborative robotic arm with 3 kg of payload and repeatibility below 0.1 mm. The robot can reach angular speeds up to 90º/second and has a reach of 656 mm. The robot contains a variety of sensors on each joint and can be controlled from any industrial controller that supports ROS 2 and uses the HRIM information model. Each of the modules contains the H-ROS communication bus for robots enabled by the H-ROS SoM, which delivers real-time, security and safety capabilities for ROS 2 at the module level.

No information is provided about how ROS 2 nodes are distributed on each module. Each joint offers the following ROS 2 API capabilities as described in their documentation (MARA joint):

- **Topics**
- `GoalRotaryServo.msg` model allows to control the position, velocity or/and effort (generated from models/actuator/servo/topics/goal.xml, see HRIM for more).
- `StateRotaryServo.msg` publishes the status of the motor.
- `Power.msg` publishes the power consumption.
- `Status.msg` informs about the resources that are consumed by the H-ROS SoM,
- `StateCommunication.msg` is created to inform about the state of the device network.
- **Services**
- `ID.srv` publishes the general identity of the component.
- `Simulation3D.srv` and `SimulationURDF.srv` send the URDF and the 3D model of the modular component.

- `SpecsCommunication.srv` is a HRIM service which reports the specs of the device network.
- `SpecsRotaryServo.srv` is a HRIM message which reports the main features of the device.
- `EnableDisable.srv` disables or enables the servo motor.
- `ClearFault.srv` sends a request to clear any fault in the servo motor.
- `Stop.srv` requests to stop any ongoing movement.
- `OpenCloseBrake.srv` opens or closes the servo motor brake.
- **Actions**:
- `GoalJointTrajectory` allows to move the joint using a trajectory msg.

Such API gets translated into the following abstractions:

Topic	**Name**
goal	/hrim_actuation_servomotor_XXXXXXXXXXXX/ goal_axis1
goal	/hrim_actuation_servomotor_XXXXXXXXXXXX/ goal_axis2
state	/hrim_actuation_servomotor_XXXXXXXXXXXX/ state_axis1
state	/hrim_actuation_servomotor_XXXXXXXXXXXX/ state_axis2

Service	**Name**
specs	/hrim_actuation_servomotor_XXXXXXXXXXXX/specs
enable servo	/hrim_actuation_servomotor_XXXXXXXXXXXX/enable
disable servo	/hrim_actuation_servomotor_XXXXXXXXXXXX/disable
clear fault	/hrim_actuation_servomotor_XXXXXXXXXXXX/ clear_fault
stop	/hrim_actuation_servomotor_XXXXXXXXXXXX/ stop_axis1
stop	/hrim_actuation_servomotor_XXXXXXXXXXXX/ stop_axis2
close brake	/hrim_actuation_servomotor_XXXXXXXXXXXX/ close_brake_axis1
close brake	/hrim_actuation_servomotor_XXXXXXXXXXXX/ close_brake_axis2
open brake	/hrim_actuation_servomotor_XXXXXXXXXXXX/ open_brake_axis1
open brake	

Service	Name
	/hrim_actuation_servomotor_XXXXXXXXXXXX/ open_brake_axis2

Action	Name
goal joint trajectory	/hrim_actuation_servomotor_XXXXXXXXXXXX/ trajectory_axis1
goal joint trajectory	/hrim_actuation_servomotor_XXXXXXXXXXXX/ trajectory_axis2

Parameters	Name
ecat_interface	Ethercat interface
reduction_ratio	Factor to calculate the position of the motor
position_factor	Factor to calculate the position of the motor
torque_factor	Factor to calculate the torque of the motor
count_zeros_axis1	Axis 1 absolute value of the encoder for the zero position
count_zeros_axis2	Axis 2 absolute value of the encoder for the zero position
enable_logging	Enable/Disable logging
axis1_min_position	Axis 1 minimum position in radians
axis1_max_position	Axis 1 maximum position in radians
axis1_max_velocity	Axis 1 maximum velocity in radians/s
axis1_max_acceleration	Axis 1 maximum acceleration in radians/s^2
axis2_min_position	Axis 2 minimum position in radians
axis2_max_position	Axis 2 maximum position in radians
axis2_max_velocity	Axis 2 maximum velocity in radians/s
axis2_max_acceleration	Axis 2 maximum acceleration in radians/s^2

The controller used in the application is a ROS 2-enabled industrial PC, specifically, an ORC. This PC also features the H-ROS communication bus for robots which commands MARA in a deterministic manner. This controller makes direct use of the aforementioned communication abstractions (topics, services and actions).

Architecture Dataflow diagram

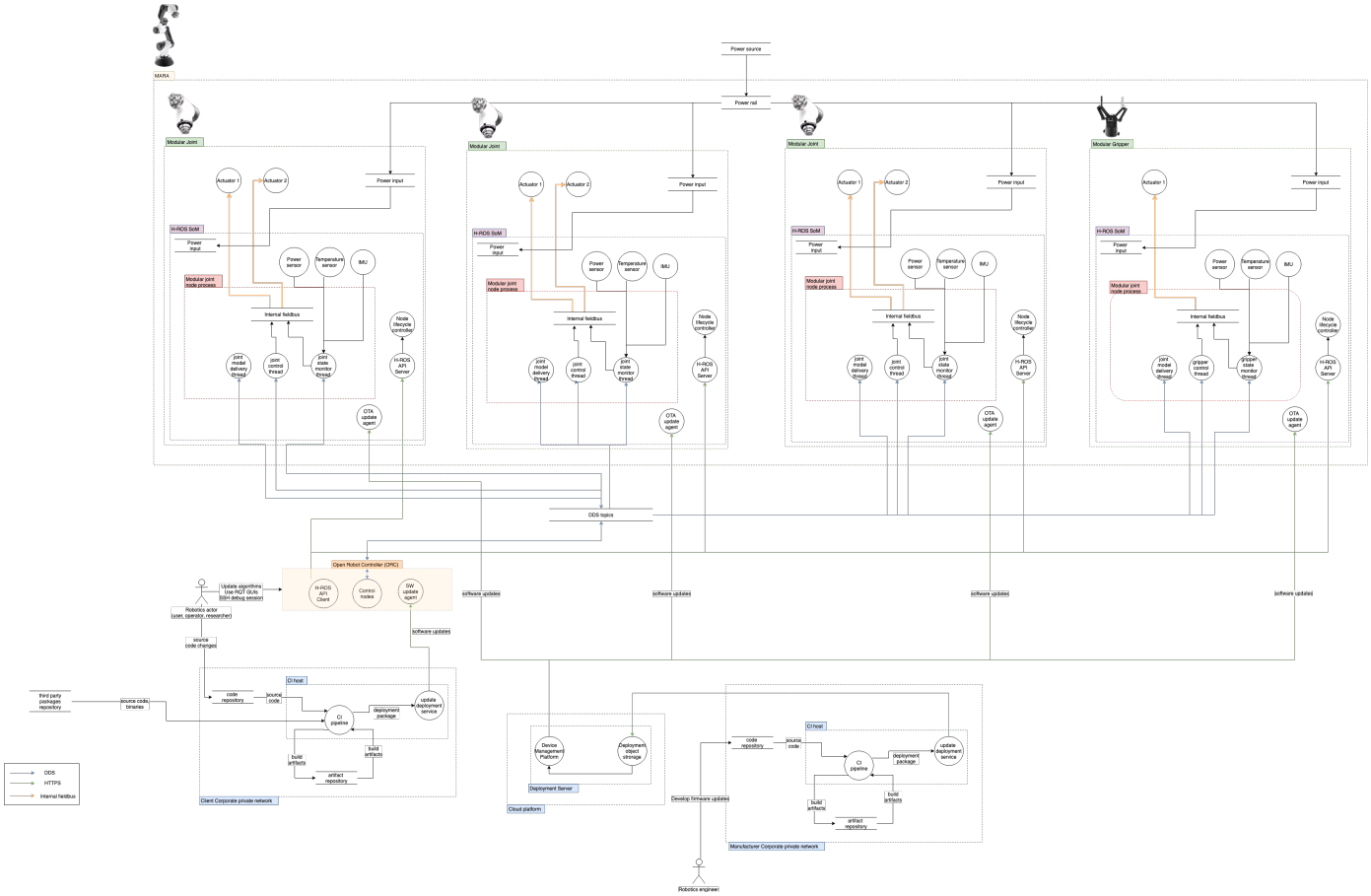


Figure 5 · MARA-ROS2_Application.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

Assets

This section aims for describing the components and specifications within the MARA robot environment. Below the different aspects of the robot are listed and detailed in Hardware, Software and Network. The external actors and Data assests are described independently.

Hardware

- 1x **MARA modular robot** is a modular manipulator, ROS 2 enabled robot for industrial automation purposes.
- **Robot modules**
  - 3 x Han's Robot Modular Joints: 2 DoF electrical motors that include precise encoders and an electro-mechanical breaking system for safety purposes. Available in a variety of different torque and size combinations, going from 2.8 kg to 17 kg weight and from 9.4 Nm to 156 Nm rated torque.

    - Mechanical connector: H-ROS connector A
    - Power input: 48 Vdc
    - Communication: H-ROS robot communication bus
      - Link layer: 2 x Gigabit (1 Gbps) TSN Ethernet network interface
      - Middleware: Data Distribution Service (DDS)
    - On-board computation: Dual core ARM® Cortex-A9
    - Operating System: Real-Time Linux
    - ROS 2 version: Crystal Clemmys
    - Information model: HRIM Coliza
    - Security:
      - DDS crypto, authentication and access control plugins
    - 1 x Robotiq Modular Grippers: ROS 2 enabled industrial end-of-arm-tooling.
    - Mechanical connector: H-ROS connector A
    - Power input: 48 Vdc
    - Communication: H-ROS robot communication bus
      - Link layer: 2 x Gigabit (1 Gbps) TSN Ethernet network interface
      - Middleware: Data Distribution Service (DDS)
    - On-board computation: Dual core ARM® Cortex-A9
    - Operating System: Real-Time Linux
    - ROS 2 version: Crystal Clemmys
    - Information model: HRIM Coliza
    - Security:
      - DDS crypto, authentication and access control plugins
- 1 x **Industrial PC**: ORC include:
- CPU: Intel i7 @ 3.70GHz (6 cores)
- RAM: Physical 16 GB DDR4 2133 MHz
- Storage: 256 GB M.2 SSD interface PCIExpress 3.0
- Communication: H-ROS robot communication bus
    - Link layer: 2 x Gigabit (1 Gbps) TSN Ethernet network interface
    - Middleware: Data Distribution Service (DDS)
- Operating System: Real-Time Linux
- ROS 2 version: Crystal Clemmys
- Information model: HRIM Coliza
- Security:
    - DDS crypto, authentication and access control plugins
- 1 x **Update Server: OTA**
- Virtual Machine running on AWS EC2
    - Operating System: Ubuntu 18.04.2 LTS
    - Software: Mender OTA server

Network

1. **Ethernet time sensitive (TSN) internal network**: Interconnection of modular joints in the MARA Robot is performed over a daisy chained Ethernet TSN channel. Each module acts as a switch, forming the internal network of the robot.

2. **Manufacturer (Acutronic Robotics) corporate private network**: A secure corporate wide-area network, that spans multiple cities. Only authenticated user with suitable credentials can connect to the network, and good security practices like password rotation are in place. This network is used to develop and deploy updates in the robotic systems. Managed by the original manufacturer.
3. **End-user corporate private network**: A secure corporate wide-area network, that spans multiple cities. Only authenticated user with suitable credentials can connect to the network, and good security practices like password rotation are in place.
4. **Cloud network**: A VPC network residing in a public cloud platform, containing multiple servers. The network follows good security practices, like implementation of security applicances, user password rotation and multi-factor authentication. Only allowed users can access the network. The OTA service is open to the internet. Uses of the cloud network: * Manufacturer (Acutronic Robotics) uploads OTA artifacts from their `Manufacturer corporate private network` to the Cloud Network. * Robotic Systems on the `End-user corporate private network` fetch those artifacts from the Cloud Network. * Robotic Systems on the `End-user corporate private network` send telemetry data for predictive maintenance to Cloud Network.

Software processes

In this section all the processes running on the robotic system in scope are detailed.

- **Onboard Modular Joints**
- `hros_servomotor_hans_lifecyle_node` this node is in charge of controlling the motors inside the H-ROS SoM. This nodes exposes some services, actions and topics described below. This node publishes joint states, joint velocities and joint efforts. It provides a topic for servoing the modular joint and an action that will be waiting for a trajectory.
- `hros_servomotor_hans_generic_node` a node which contains several generic services and topics with information about the modular joint, such as `power` measurement readings: voltage and current, `status` about the H-ROS SoM like cpu load or network stats, specifications about `communication` and `cpu`, the `URDF` or even the mesh file of the modular joint.
- Mender (OTA client) runs inside the H-ROS SoM. When an update is launched, the client downloads this version and it gets installed and available when the device is rebooted.
- **Onboard Modular Gripper**: *Undisclosed*.
- **Onboard Industrial PC**:
- MoveIt! motion planning framework.
- Manufacturing process control applications.
- Robot teleoperation utilities.
- **ROS 1 / ROS 2 bridges**: These bridges are needed to be able to run MoveIT! which is not yet ported to ROS 2. Right now there is an effort in the community to port this tool to ROS 2.

Software dependencies

In this section all the relevant third party software dependencies used within the different components among the scope of this threat model are listed.

- Linux OS / Kernel
- ROS 2 core libraries
- H-ROS core libraries and packages
- ROS 2 system dependencies as defined by rosdep:
- RMW implementation: In the H-ROS API there is the chance to choose the DDS implementation.

- The threat model describes attacks with the security enabled and disabled. If the security is enabled, the security plugins are assumed to be configured and enabled.

See Mara ROS 2 Tutorials to find more details about software dependencies.

External Actors

All the actors interacting with the robotic system are here gathered.

- End users
- Robotics user: Interacts with the robot for performing work labour tasks.
- Robotics operator: Performs maintenance tasks on the robot and integrates the robot with the industrial network.
- Robotics researcher: Develops new applications and algorithms for the robot.
- Manufacturer
- Robotics engineer: Develops new updates and features for the robot itself. Performs in-site robot setup and maintainance tasks.

Robot Data assets

In this section all the assets storing information within the system are displayed.

- ROS 2 network (topic, actions, services information)
- Private Data
  - Logging messages
  - Restricted Data
  - Robot Speed and Orientation. Some understanding of the current robot task may be reconstructed from those messages.
- H-ROS API
- Configuration and status data of hardware.
- Modules (joints and gripper)
- ROS logs on each module. Private data, metrics and configuration information that could lead to GDPR issues or disclosure of current robot tasks.
- Module embedded software (drivers, operating system, configuration, etc.)
  - Secret data. Intellectual Property (IP) theft is a critical issue here.
- Credentials. CI/CD, enterprise assets, SROS certificates.
- ORC Industrial PC
- Public information. Motion planning algorithms for driving the robot (MoveIt! motion planning framework).
- Configuration data. Private information with safety implications. Includes configuration for managing the robot.
- Programmed routines. Private information with safety implications.
- Credentials. CI/CD, enterprise assets, SROS certificates.
- CI/CD and OTA subsystem
- Configuration data.
- Credentials
- Firmware files and source code. Intellectual property, both end-user and manufacturer.
- Robot Data
- System and ROS logs are stored on each joint module's filesystem.
- Robot system is subject to physical attacks (physically removing the disk from the robot to read its data).

- Cloud Data
- Different versions of the software are stored on the OTA server.
- Secret Management
- DDS / ROS Topics
  - If SROS is enabled, private keys are stored on the local filesystem of each module.
- Mender OTA
- Mender OTA client certificates are stored on the robot file system.

Use case scenarios

As described above, the application considered is a MARA modular robot operating on an industrial environment while performing a `pick & place` activity. For this use case all possible external actor have been included. The actions they can perform on the robotic system have been limited to the following ones:

- **End-Users**: From the End-user perspective, the functions considered are the ones needed for a factory line to work. Among this functions, there are the ones keeping the robot working and being productive. On an industrial environment, this group of external actors are the ones making use of the robot on their facilities.
- **Robotics Researcher**: Development, testing and validation
  - A research engineer develops a pick and place task with the robot. They may:
  - Restart the robot
  - Restart the ROS graph
  - Physically interact with the robot
  - Receive software updates from OTA
  - Check for updates
  - Configure ORC control functionalities
- **Robotic User**: Collaborative tasks
  - Start the robot
  - Control the robot
  - Work alongside the robot
- **Robotics Operator**: Automation of industrial tasks
  - An industrial operator uses the robot in a factory. They may:
  - Start the robot
  - Control the robot
  - Configure the robot
  - Include the robot into the industrial network
  - Check for updates
  - Configure and interact with the ORC
- **Manufacturer**: From the manufacturer perspective, the application considered is the development of the MARA robotic platform, using an automated system for deployment of updates, following a CI/CD system. These are the actors who create and maintain the robotic system itself.
- **Robotics Engineer**: Development, testing and validation
  - Development of new functionality and improvements for the MARA robot.
  - Develop new software for the H-ROS SoM
  - Update OS and system libraries
  - Update ROS 2 subsystem and control nodes
  - Deployment of new updates to the robots and management of the fleet

    - In-Place robot maintenance

Entry points

The following section outlines the possible entry points an attacker could use as attack vectors to render the MARA vulnerable.

- **Physical Channels**
- Exposed debug ports.
- Internal field bus.
- Hidden development test points.
- **Communication Channels**
- DDS / ROS 2 Topics
    - Topics can be listened or written to by any actor: 1. Connected to a network where DDS packets are routed to. 1. Have necessary permissions (read / write) if SROS is enabled.
    - When SROS is enabled, attackers may try to compromise the CA authority or the private keys to generate or intercept private keys as well as emitting malicious certificates to allow spoofing.
- H-ROS API
    - H-ROS API access is possible to anyone on the same LAN or WAN (if port forwarding is enabled).
    - When authentication (in the H-ROS API) is enabled, attackers may try to vulnerate it.
- Mender OTA
    - Updates are pushed from the server.
    - Updates could be intercepted and modified before reaching the robot.
- **Deployed Software**
- ROS nodes running on hardware are compiled by the manufacturer and deployed directly. An attacker may tamper the software running in the hardware by compromising the OTA services.
- An attacker compromising the developer workstation could introduce a vulnerability in a binary which would then be deployed to the robot.
- MoveIt! motion planning library may contain exploitable vulnerabilities.

Trust Boundaries for MARA in `pick & place` application

The following content will apply the threat model to an industrial robot on its environment. The objective on this threat analysis is to identify the attack vectors for the MARA robotic platform. Those attack vectors will be identified and clasified depending on the risk and services implied. On the next sections MARA's components and risks will be detailed thoroughly using the threat model above, based on STRIDE and DREAD.

The diagram below illustrates MARA's application with different trust zones (trust boundaries showed with dashed green lines). The number and scope of trust zones is depending on the infrastructure behind.

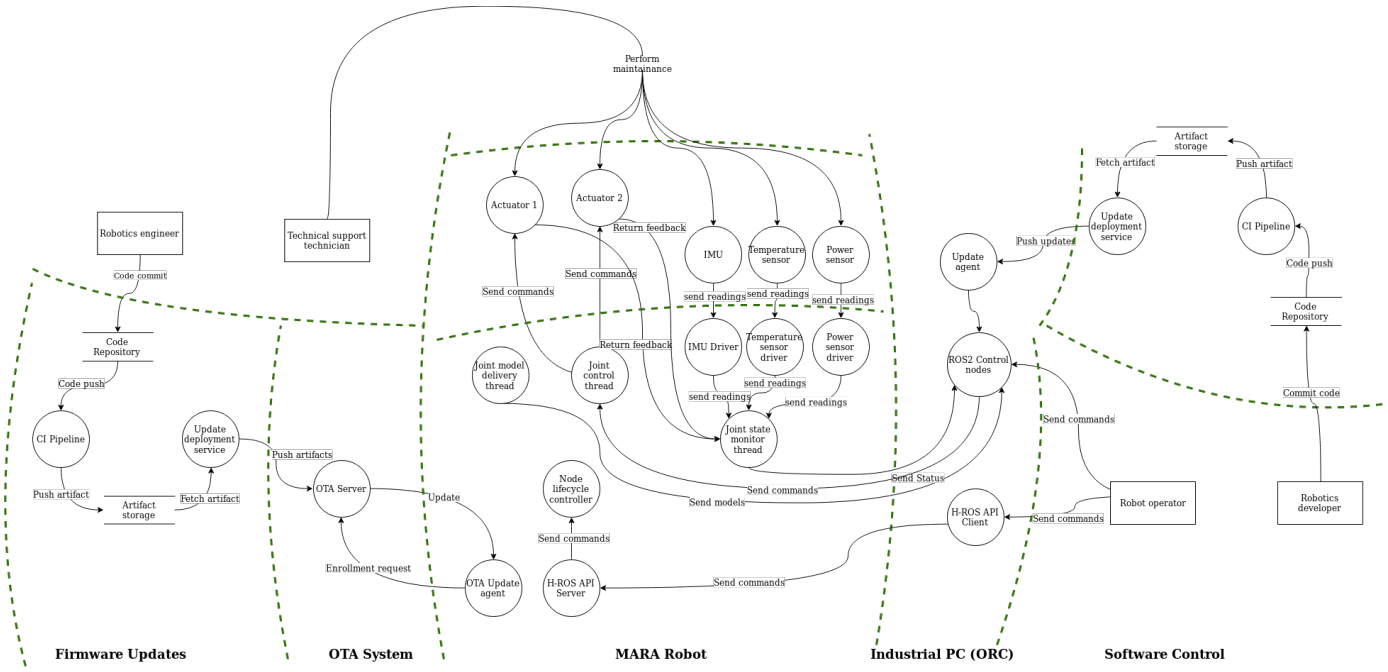


Figure 6 · threat_model_mara.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (edited with Draw.io)

The trust zones ilustrated above are the following:

- **Firmware Updates:** This zone is where the manufacturer develops the different firmware versions for each robot.
- **OTA System:** This zone is where the firmwares are stored for the robots to download.
- **MARA Robot:** All the robots' componentents and internal comunications are gathered in this zone.
- **Industrial PC (ORC):** The industrial PC itself is been considered a zone itself because it manages the software the end-user develops and sends it to the robot.
- **Software control:** This zone is where the end user develops software for the robot where the tasks to be performed are defined.

Threat Model

Each generic threat described in the main threat table can be instantiated on the MARA.

This table indicates which of MARA's particular assets and entry points are impacted by each threat. A check sign (✓) means impacted while a cross sign (✘) means not impacted. The "SROS Enabled?" column explicitly states out whether using SROS would mitigate the threat or not. A check sign (✓) means that the threat could be exploited while SROS is enabled while a cross sign (✘) means that the threat requires SROS to be disabled to be applicable.

> Historical matrix reflowed as vertical records. ✓, ✘ and ▲ retain their source meanings. SROS Enabled? means the attack can occur with SROS enabled, not that SROS prevents it. Scores and mitigation claims are historical and unverified in this review.

Embedded / Software / Communication / Inter-Component Communication

---

MARA-001 · An attacker spoofs a software component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS any node may have any name so spoofing is trivial.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result
Mitigating risk requires implementation of SROS on MARA.

Additional notes / open questions
No verification of components. An attacker could connect a fake joint directly. Direct access to the system is granted (No NAC).

MARA-002 · An attacker spoofs a software component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker deploys a malicious node which is not enabling DDS Security Extension and spoofs the `joy_node` forcing the robot to stop.

Mitigation

- DDS Security Governance document must set `allow_unauthenticated_participants` to False to avoid non-authenticated participants to be allowed to communicate with authenticated nodes.
- DDS Security Governance document must set `enable_join_access_control` to True to explicitly whitelist node-to-node-communication. permissions.xml should be as restricted as possible.

Mitigation result
Risk is mitigated

Additional notes / open questions

(Empty in source.)

MARA-003 · An attacker spoofs a software component identity.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✓

Attack
An attacker steals node credentials and spoofs the joint node forcing the robot to stop.

Mitigation

- Store node credentials in a secure location (secure enclave, RoT) to reduce the probability of having a private key leaked.
- Run nodes in isolated sandboxes to ensure one node cannot access another node data (including credentials)
- Permissions CA should digitally sign nodes binaries to prevent running tampered binaries.
- Permissions CA should be able to revoke certificates in case credentials get stolen.

Mitigation result
Mitigation risk requires additional work.

Additional notes / open questions

- AWS Robotics and Automation is currently evaluating the feasibility of storing DDS-Security credentials in a TPM.
- Complete mitigation would require isolation using e.g. Snap or Docker.
- Deploying an application with proper isolation would require us to revive discussions around [ROS 2 launch system][ros2_launch_design_pr]
- Yocto / OpenEmbedded / Snap support should be considered

MARA-004 · An attacker intercepts and alters a message.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS an attacker can modify `/goal_axis` or `trajectory_axis` messages sent through a network connection to e.g. stop the robot.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications. Message tampering is mitigated by DDS security as message authenticity is verified by default (with preshared HMACs / digital signatures)

Mitigation result
Risk is reduced if SROS is used.

Additional notes / open questions

(Empty in source.)

MARA-005 · An attacker writes to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS, any node can publish to any topic.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result

(Empty in source.)

Additional notes / open questions

(Empty in source.)

MARA-006 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS: any node can listen to any topic.

Mitigation

- Enable SROS / DDS Security Extension to authenticate and encrypt DDS communications.

Mitigation result
Risk is reduced if SROS is used.

Additional notes / open questions

(Empty in source.)

MARA-007 · An attacker listens to a communication channel without authorization.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
DDS participants are enumerated and [fingerprinted][aztarna] to look for potential vulnerabilities.

Mitigation

- DDS Security Governance document must set `metadata_protection_kind` to ENCRYPT to prevent malicious actors from observing communications.
- DDS Security Governance document must set `enable_discovery_protection` to True to prevent malicious actors from enumerating and fingerprinting DDS participants.
- DDS Security Governance document must set `enable_liveliness_protection` to True

Mitigation result
Risk is mitigated if DDS-Security is configured appropriately.

Additional notes / open questions

(Empty in source.)

MARA-008 · An attacker prevents a communication channel from being usable.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
Without SROS: any node can ""spam"" any other component.

Mitigation

- Enable SROS to use the DDS Security Extension. This does not prevent nodes from being flooded but it ensures that only communication from allowed participants are processed.

Mitigation result
Risk may be reduced when using SROS.

Additional notes / open questions

(Empty in source.)

MARA-009 · An attacker prevents a communication channel from being usable.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘

Attack
A node can ""spam"" another node it is allowed to communicate with.

Mitigation

- Implement rate limitation on topics
- Define a method for topics to declare their required bandwidth / rate.

Mitigation result
Mitigating risk requires additional work.

Additional notes / open questions
How to enforce when nodes are malicious? Observe and kill?

Embedded / Software / Communication / Remote Application Interface

---

MARA-010 · An attacker gains unauthenticated access to the remote application interface.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✓; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker connects to the H-ROS API in an unauthenticated way. Reads robot configuration and alters configuration values.

Mitigation

- Add authentication mechanisms to the H-ROS API.
- Enable RBAC to limit user interaction with the API.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-011 · An attacker could eavesdrop communications to the Robot’s remote application interface.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✓; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker executes a MitM attack, eavesdropping all unencrypted communications and commands sent to the API.

Mitigation
Encrypt the communications through the usage of HTTPS.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-012 · An attacker could alter data sent to the Robot’s remote application interface.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✓; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker could execute a MitM attack and alter commands being sent to the API.

Mitigation
Encrypt the communications through the usage of HTTPS.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Embedded / Software / OS & Kernel

MARA-013 · An attacker compromises the real-time clock to disrupt the kernel RT scheduling guarantees.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✓; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious actor attempts to write a compromised kernel to /boot

Mitigation

- Enable verified boot on Uboot to prevent booting altered kernels.
- Use built in TPM to store firmware public keys and define an RoT.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-014 · An attacker compromises the OS or kernel to alter robot data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✓; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious actor attempts to write a compromised kernel to /boot

Mitigation

- Enable verified boot on Uboot to prevent booting altered kernels.
- Use built in TPM to store firmware public keys and define an RoT.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-015 · An attacker compromises the OS or kernel to eavesdrop on robot data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✓; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
A malicious actor attempts to write a compromised kernel to /boot

Mitigation

- Enable verified boot on Uboot to prevent booting altered kernels.
- Use built in TPM to store firmware public keys and define an RoT.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Embedded / Software / Component-Oriented Architecture

MARA-016 · A node accidentally writes incorrect data to a communication channel.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A node writes random or invalid values to the /goal_axis topics.

Mitigation

- Expand DDS IDL to allow users to embed validity criteria to automate input sanitization (i.e. validate ranges, etc.)
- Expand RMW to define per-topic strategies for invalid messages (drop, throw, abort, etc.).

Mitigation result
Need to expand DDS IDL or RMW for mitigating the risk.

Additional notes / open questions

(Empty in source.)

MARA-017 · A node accidentally writes incorrect data to a communication channel.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
A node could to write out of physical bounds values to the `/goal_axis` or `/trajectory_axis` topics, causing damage to the robot.

Mitigation

- Define physical limitations of the different joints on the motor driver, limiting the possible movement to a safe range.
- Enable signature verification of executables to reduce the risks of inserting a malicious node.
- Limit control of actuators to only the required nodes. Enable AppArmor policies to isolate nodes.

Mitigation result
Risk is mitigated when applying limits on actuator drivers.

Additional notes / open questions

(Empty in source.)

MARA-018 · An attacker deploys a malicious node on the robot.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✓; End-user CI/CD: ✓; H-ROS API: ✘; OTA: ✓; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker deploys a malicious node to the robot. This node performs dangerous movements that compromise safety. The node attempts to perform physical or logical damage to the modules.

Mitigation

- Run each node in an isolated environment with limited privileges(sandboxing).
- Enable signing and verification of executables.

Mitigation result

- Running the component in a Ubuntu Core sandbox environment could limit the consequences of the attack.
- Enabling signature verification of executables would reduce the risks of inserting a malicious node.
- Limiting control of actuators to only the required nodes would reduce risk in case of a node compromise. Enable AppArmor policies to isolate nodes.

Additional notes / open questions

(Empty in source.)

MARA-019 · An attacker can prevent a component running on the robot from executing normally.

> Malformed original row: the supplied cell count does not match the header. All cells are retained below in source order; column alignment is unresolved. Do not infer a corrected meaning from these positions.

Source column 1
An attacker can prevent a component running on the robot from executing normally.

Source column 2
✓

Source column 3
✓

Source column 4
✓

Source column 5
✘

Source column 6
✘

Source column 7
✘

Source column 8
✘

Source column 9
✓

Source column 10
✘/✓

Source column 11
A malicious node running on the robot starts sending kill requests to other nodes in the system, disrupting the normal behaviour of the robot.

Source column 12
Having the abiliy to shutdown/kill nodes through API request supposes a problem on the ROS implementation. Deprecation of the function should be considered. Node restarting policie should be applied.

Source column 13
Deprecation of the shutdown API call needs to be considered.

Source column 14

(No cell supplied in source row; position follows original column order.)

Embedded / Software / Configuration Management

MARA-020 · An attacker modifies configuration values without authorization.

> Malformed original row: the supplied cell count does not match the header. All cells are retained below in source order; column alignment is unresolved. Do not infer a corrected meaning from these positions.

Source column 1
An attacker modifies configuration values without authorization.

Source column 2
✓

Source column 3
✓

Source column 4
✓

Source column 5
✘

Source column 6
✘

Source column 7
✘

Source column 8
✘

Source column 9
✘

Source column 10
✘

Source column 11
Node parameters are freely modifiable by any DDS domain participant.

Source column 12

Source column 13

Source column 14

Source column 15

MARA-021 · An attacker accesses configuration values without authorization.

> Malformed original row: the supplied cell count does not match the header. All cells are retained below in source order; column alignment is unresolved. Do not infer a corrected meaning from these positions.

Source column 1
An attacker accesses configuration values without authorization.

Source column 2
✓

Source column 3
✓

Source column 4
✓

Source column 5
✘

Source column 6
✘

Source column 7
✘

Source column 8
✘

Source column 9
✘

Source column 10
✘

Source column 11
Node parameters values can be read by any DDS domain participant.

Source column 12

Source column 13

Source column 14

Source column 15

MARA-022 · A user accidentally misconfigures the robot.

> Malformed original row: the supplied cell count does not match the header. All cells are retained below in source order; column alignment is unresolved. Do not infer a corrected meaning from these positions.

Source column 1
A user accidentally misconfigures the robot.

Source column 2
✓

Source column 3
✓

Source column 4
✓

Source column 5
✘

Source column 6
✘

Source column 7
✘

Source column 8
✘

Source column 9
✘

Source column 10
✘/✓

Source column 11
Node parameters values can be modified by anyone to any value.

Source column 12

Source column 13

Source column 14

Source column 15

Embedded / Software / Data Storage (File System)

MARA-023 · An attacker modifies the robot file system by physically accessing it.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker modifies the filesystem data within the robot

Mitigation
Enable filesystem encryption with LUKS or dm-crypt, storing keys on TPM device.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-024 · An attacker eavesdrops on the robot file system by physically accessing it.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker physically accesses the memory chip to eavesdrop credentials, logs or sensitive data.

Mitigation
Enable filesystem encryption with LUKS or dm-crypt, storing keys on TPM device.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-025 · An attacker saturates the robot disk with data.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✓; Manufacturer CI/CD: ✓; End-user CI/CD: ✓; H-ROS API: ✘; OTA: ✓; Physical: ✓

SROS Enabled? (historical legend)
✓

Attack
A malicious node writes random data that fills the robot disk.

Mitigation
Enable disk quotas on the system. Enable sandboxing of the processes. Separate disk into multiple partitions, sending non trivial data to temporary directories.

Mitigation result
Risk is partially mitigated. Disk cleanup routines and log rotation should also be implemented.

Additional notes / open questions

(Empty in source.)

Embedded / Software / Logs

MARA-026 · An attacker exfiltrates log data to a remote server.

Impacted assets

Human assets: ✘; Robot application: ✘

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker compromising the OTA server could request device log data and eavesdrop sensitive information.

Mitigation
Enable RBAC on the OTA server, limit access to sensitive functions.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Embedded / Hardware / Sensors

MARA-027 · An attacker spoofs a robot sensor (by e.g. replacing the sensor itself or manipulating the bus).

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✓

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker could physically tamper the readings from the sensors.

Mitigation
Add noise or out-of-bounds reading detection mechanism on the robot, causing to discard the readings or raise an alert to the user. Add detection of sensor disconnections.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Embedded / Hardware / Actuators

MARA-028 · An attacker spoofs a robot actuator.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker could insert a counterfeit modular joint in the robot, compromising the whole system (e.g. a modified gripper).

Mitigation

- Implement network access control systems, performing a verification of the part before granting access to the system.
- Implement certificate based, 802.1x authentication for the communication with the nodes, discarding any new modules that do not authenticate on the system.

Mitigation result
Risk is mitigated.

Additional notes / open questions
Additional evaluation should be performed. Authenticating nodes via certificates would require shipping the nodes with client certificates, and the validated manufacturers would require a subordinate CA to sign their modules (Kinda DRM-ish).

MARA-029 · An attacker modifies the command sent to the robot actuators (intercept & retransmit).

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker intercepts the communication channel traffic. The command is altered an retransmitted to the target joint.

Mitigation

- Implement network access control systems, performing a verification of the part before granting access to the system.
- Implement certificate based, 802.1x authentication for the communication with the nodes, discarding any new modules that do not authenticate on the system.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Embedded / Hardware / Communications

MARA-030 · An attacker connects to an exposed debug port.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker could connect to an exposed debug port and gain control over the robot through the execution of arbitrary commands.

Mitigation

- Limit access or remove exposed debug ports.
- Disable local debug terminals and functionality from the ports.
- Add authentication mechanisms to limit access to the ports only to authenticated devices and users.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-031 · An attacker connects to an internal communication bus.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✘; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker could connect to an internal communication bus to send arbitrary data or eavesdrop communication between different components of the robot.

Mitigation

- Limit access or remove unused communication ports.
- Physically limit access to the internal robot components and communication buses.
- Add physical tamper detection sensors to detect physical intrussions to the robot.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

Remote / Software Deployment

MARA-032 · An attacker spoofs the deployment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✓; End-user CI/CD: ✓; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker spoofs the update deployment server and serves malicious content to the devices.

Mitigation

- Validate the deployment server through Public Key Infrastructure.
- Prevent insecure connections to the server from the devices through HTTPS and HSTS policies.
- Certificate pinning on devices.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-033 · An attacker modifies the binaries sent by the deployment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker intercepts the communication to the deployment server and serves malicious content to the devices.

Mitigation

- Validate the deployment server through Public Key Infrastructure.
- Prevent insecure connections to the server from the devices through HTTPS and HSTS policies.
- Digitally sign the binaries sent to the devices.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-034 · An attacker intercepts the binaries sent by the depoyment service.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker intercepts the update communication and stores the binary sent to the devices, gaining access to intellectual property.

Mitigation

- Make use of secure, encrypted communication channels.
- Verify client devices through client certificates.
- Sign and Encrypt update files.

Mitigation result
Risk is mitigated.

Additional notes / open questions

(Empty in source.)

MARA-035 · An attacker prevents the robot and the deployment service from communicating.

Impacted assets

Human assets: ✓; Robot application: ✓

Impacted entry points

ROS 2 API (DDS): ✘; Manufacturer CI/CD: ✘; End-user CI/CD: ✘; H-ROS API: ✘; OTA: ✓; Physical: ✘

SROS Enabled? (historical legend)
✘/✓

Attack
An attacker blocks the robots update process.

Mitigation

- Deploy multiple update server endpoints.
- Deploy a distributed update system.

Mitigation result
Risk is partially mitigated.

Additional notes / open questions

(Empty in source.)

Attack Trees

In an atempt to analyze the different possible attacks before even happening, attack trees are created. This diagrams speculate about the possible attacks against the system in order to be able to counter them. Threats can be organized in the following attack trees. Attacks are ordered starting from the physical attack vector, continuing with network based attacks, and finishing with the support infrastructure for the robot.

Physical vector attack tree

The following attack tree describes the possible paths to be followed by an attacker for compromising the system.

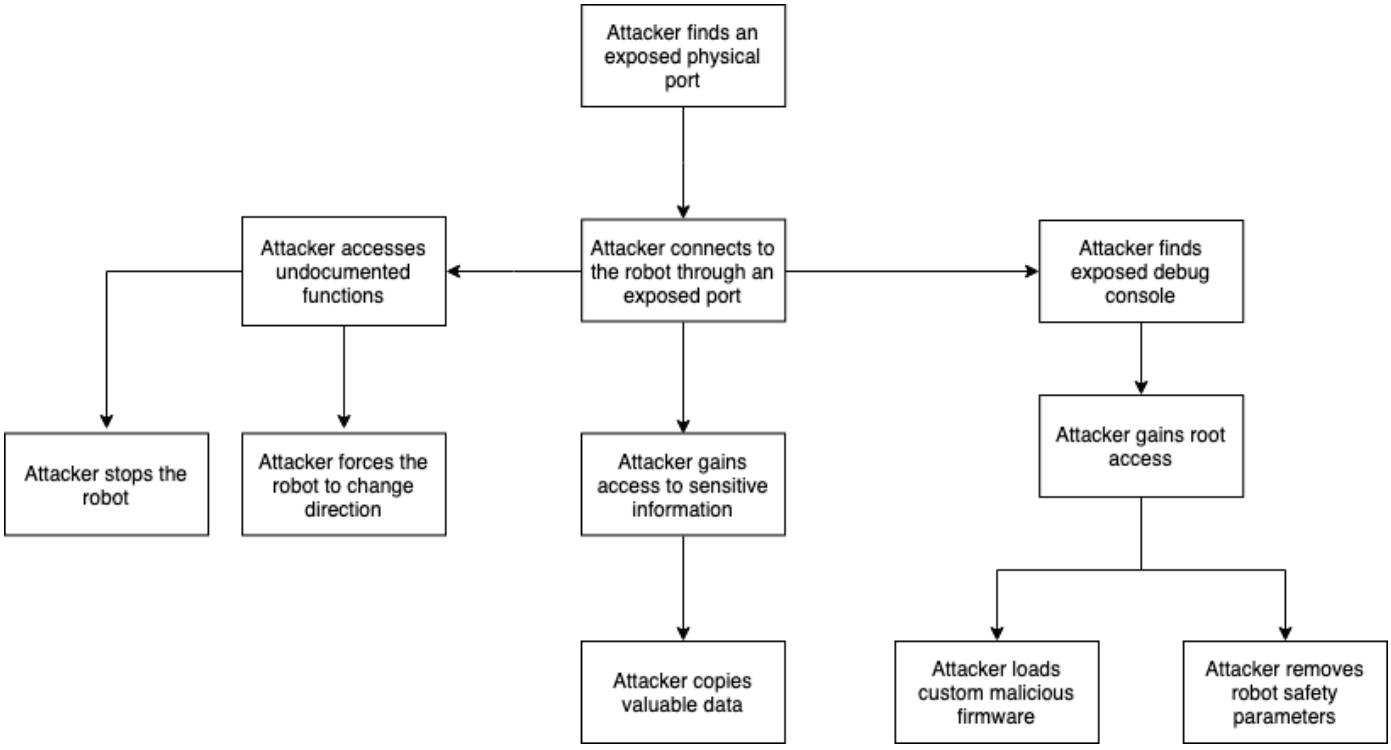


Figure 7 · attack_tree_physical.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

The next diagram shows the infrastructure affected on a possible attack based on a compromise of a physical communication port.

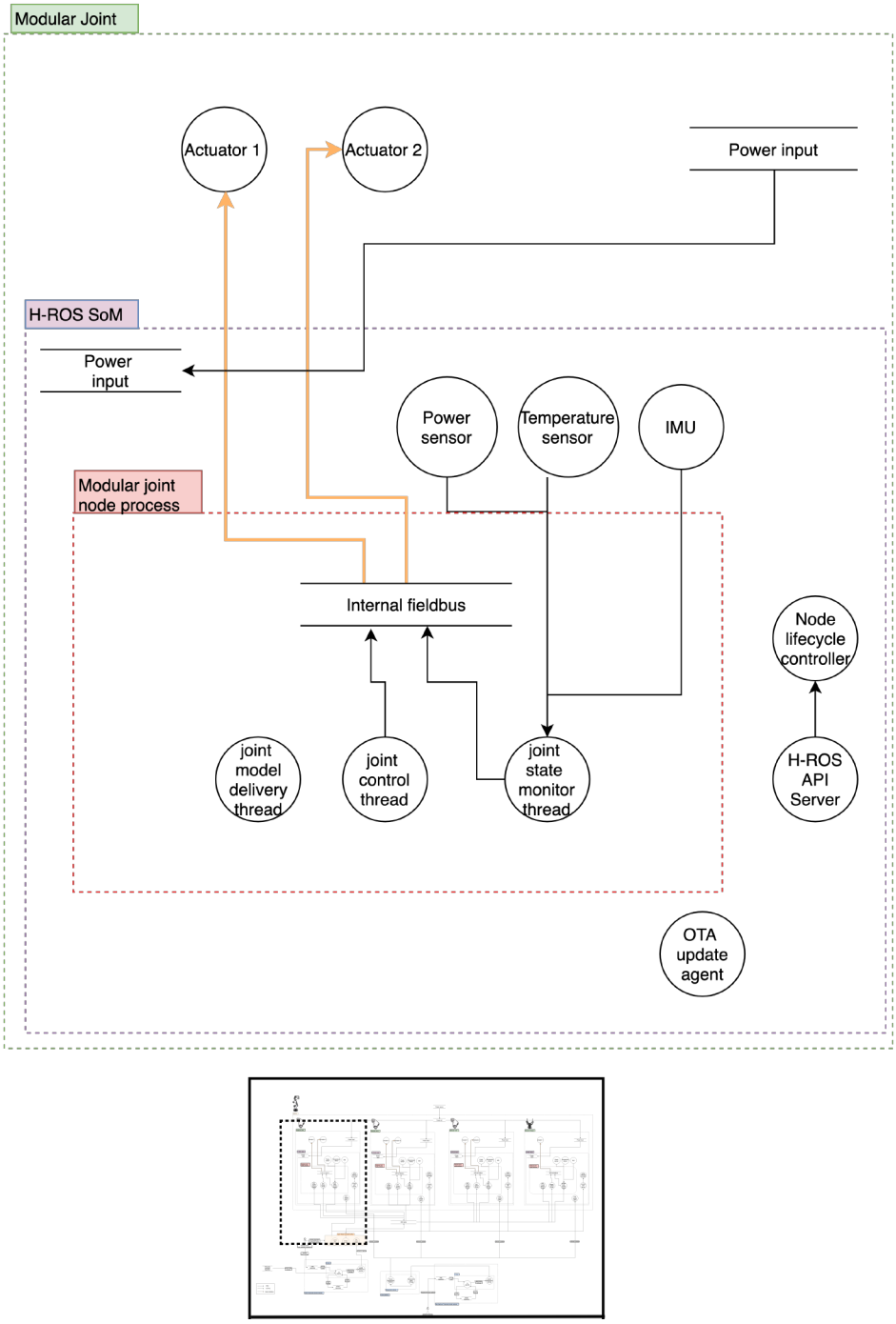


Figure 8 · attack_tree_physical_arch.png. Original figure; a larger view appears in the companion atlas.

ROS 2 API vector attack tree

The following attack tree describes the possible paths to be followed by an attacker for physically compromising the system.

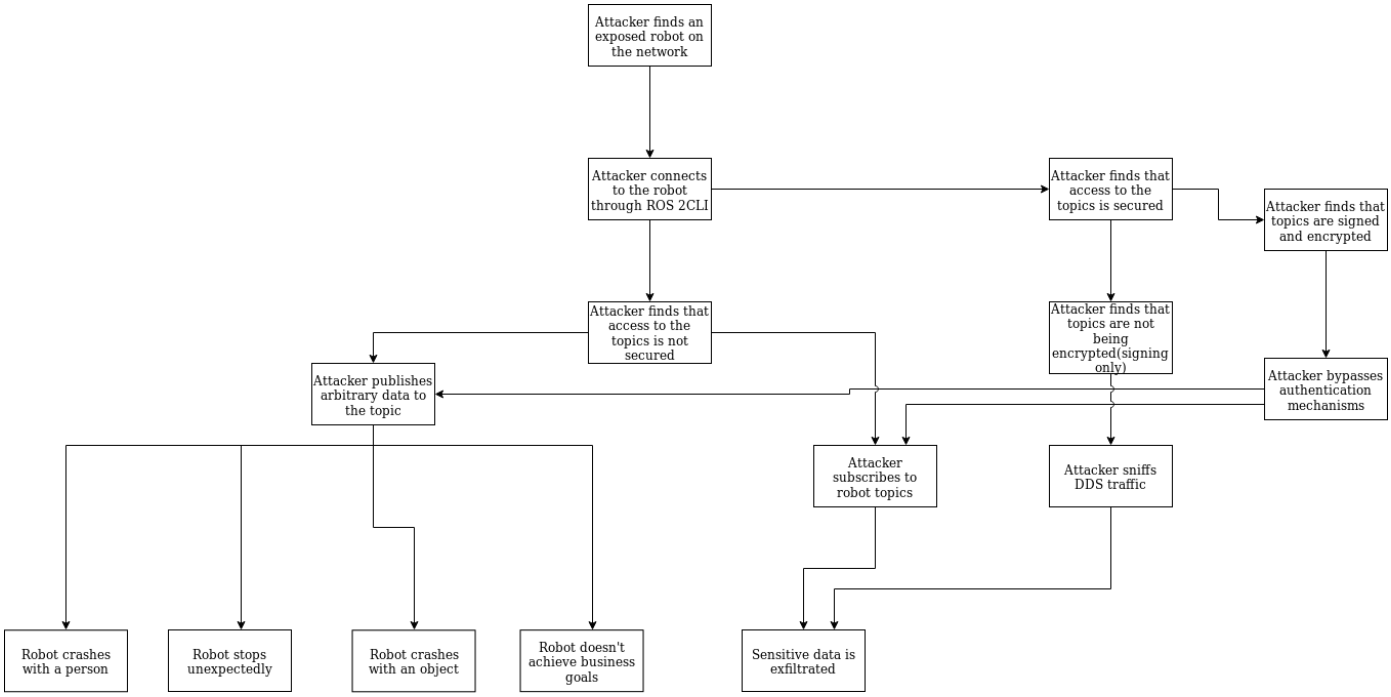


Figure 9 · attack_tree_ros2.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

The following diagram shows the infrastructure affected on a possible attack based on exploitation of the ROS 2 API.

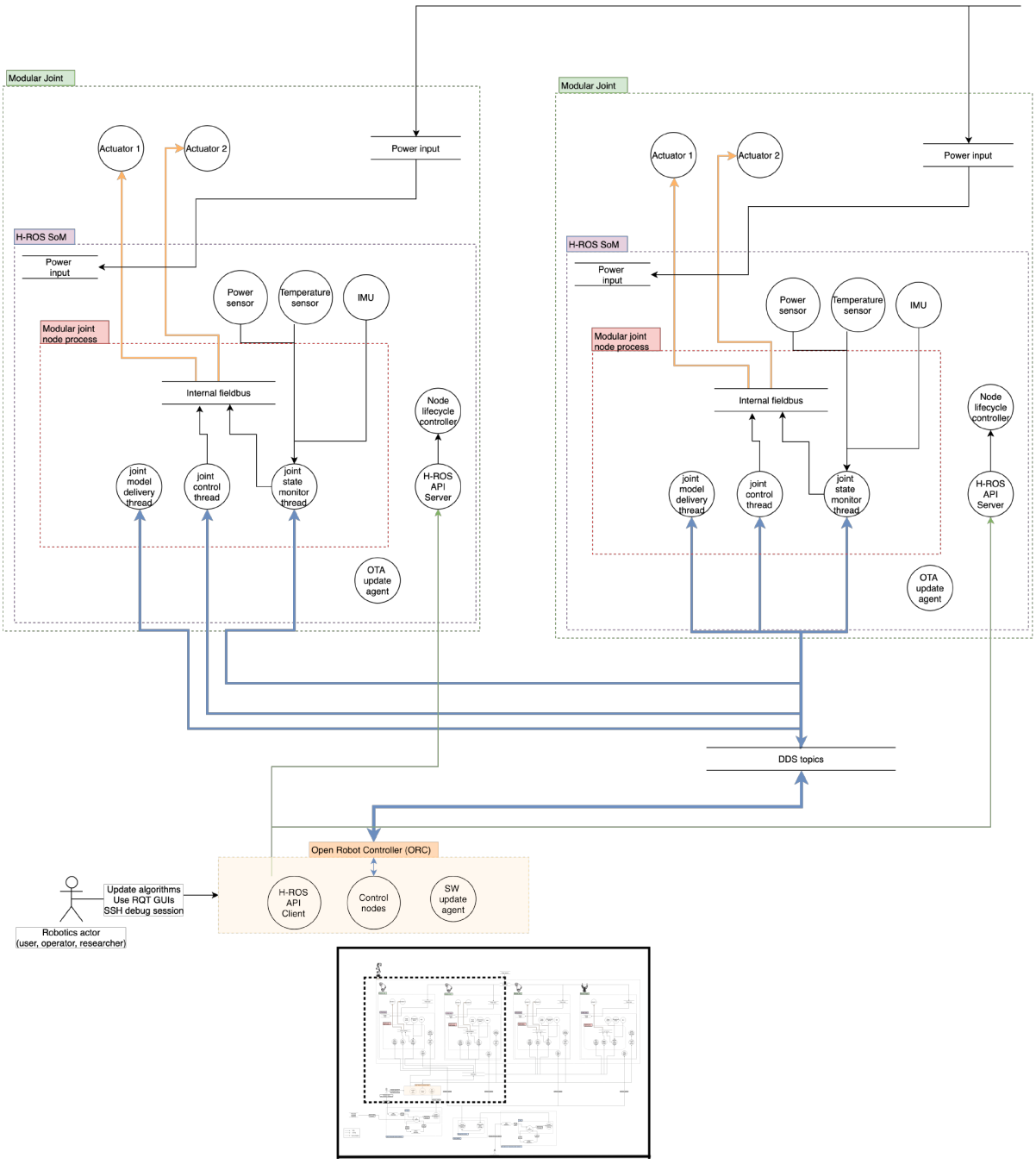


Figure 10 · attack_tree_ros2_arch.png. Original figure; a larger view appears in the companion atlas.

H-ROS API vector attack tree

The following attack tree describes the possible paths to be followed by an attacker for compromising the system.

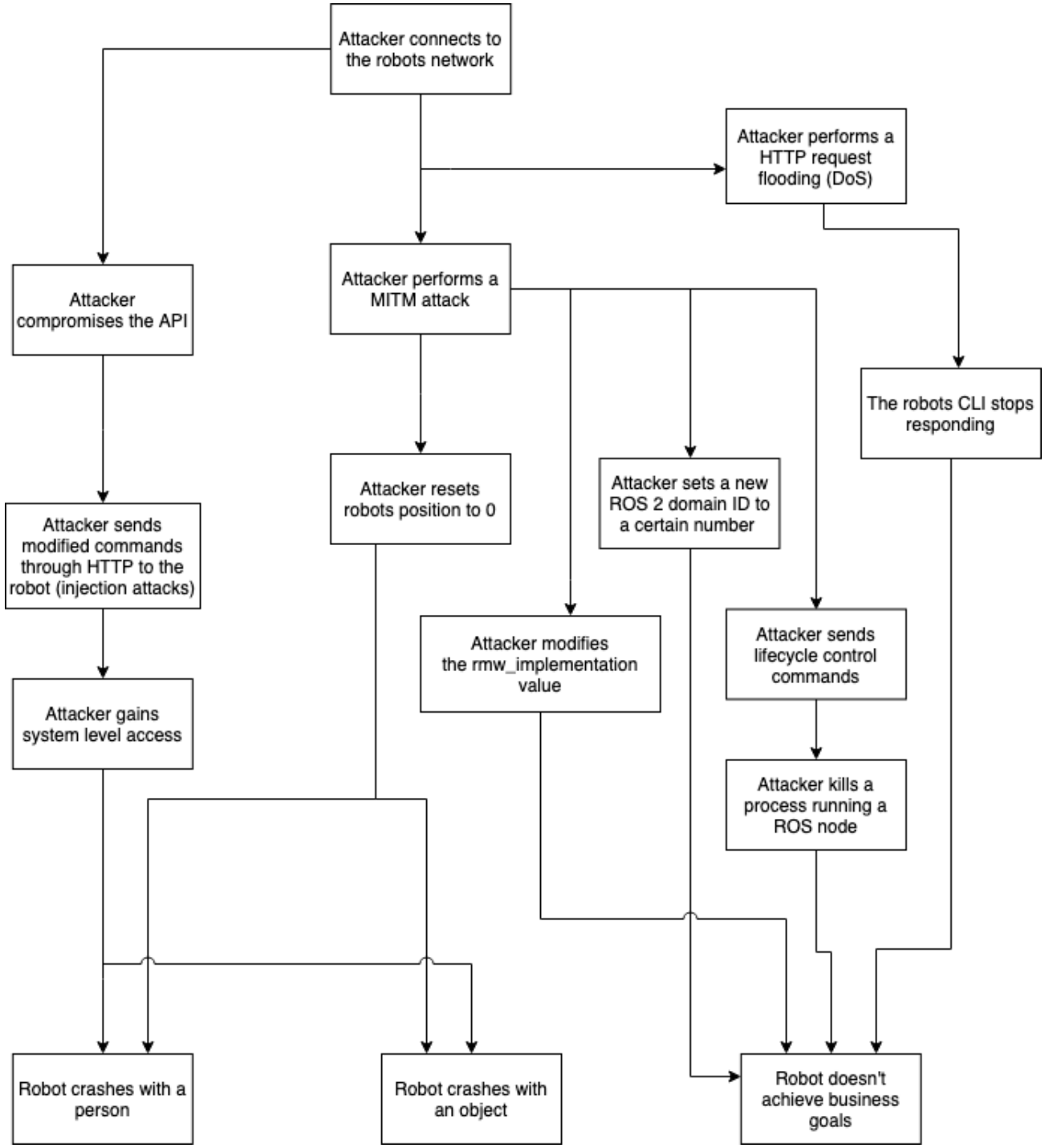


Figure 11 · attack_tree_api.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

The diagram below shows the infrastructure affected on a possible attack against MARA's H-ROS API.

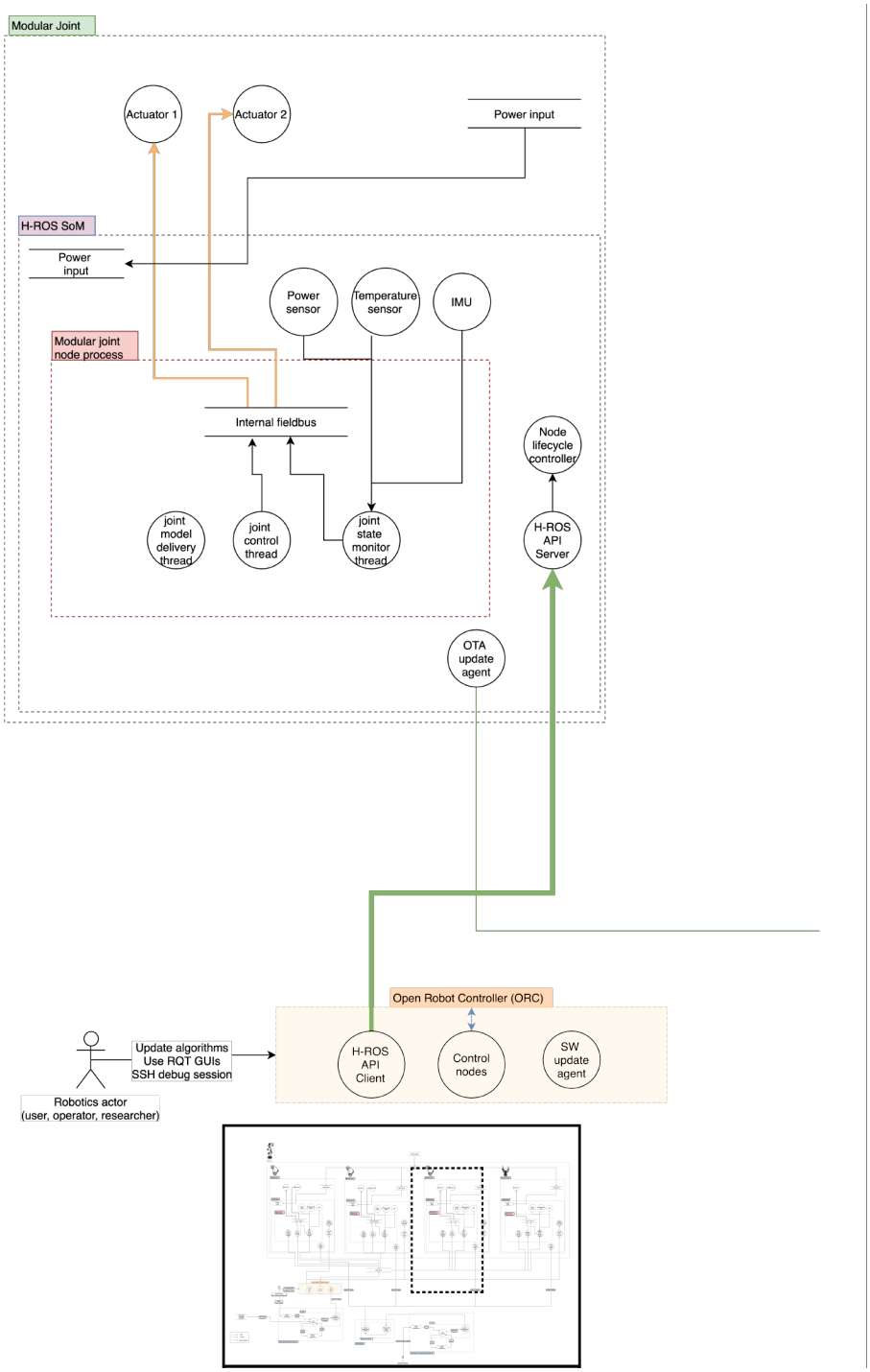


Figure 12 · attack_tree_api_arch.png. Original figure; a larger view appears in the companion atlas.

Code repository compromise vector attack tree

The following attack tree describes the possible paths to be followed by an attacker for compromising the system.

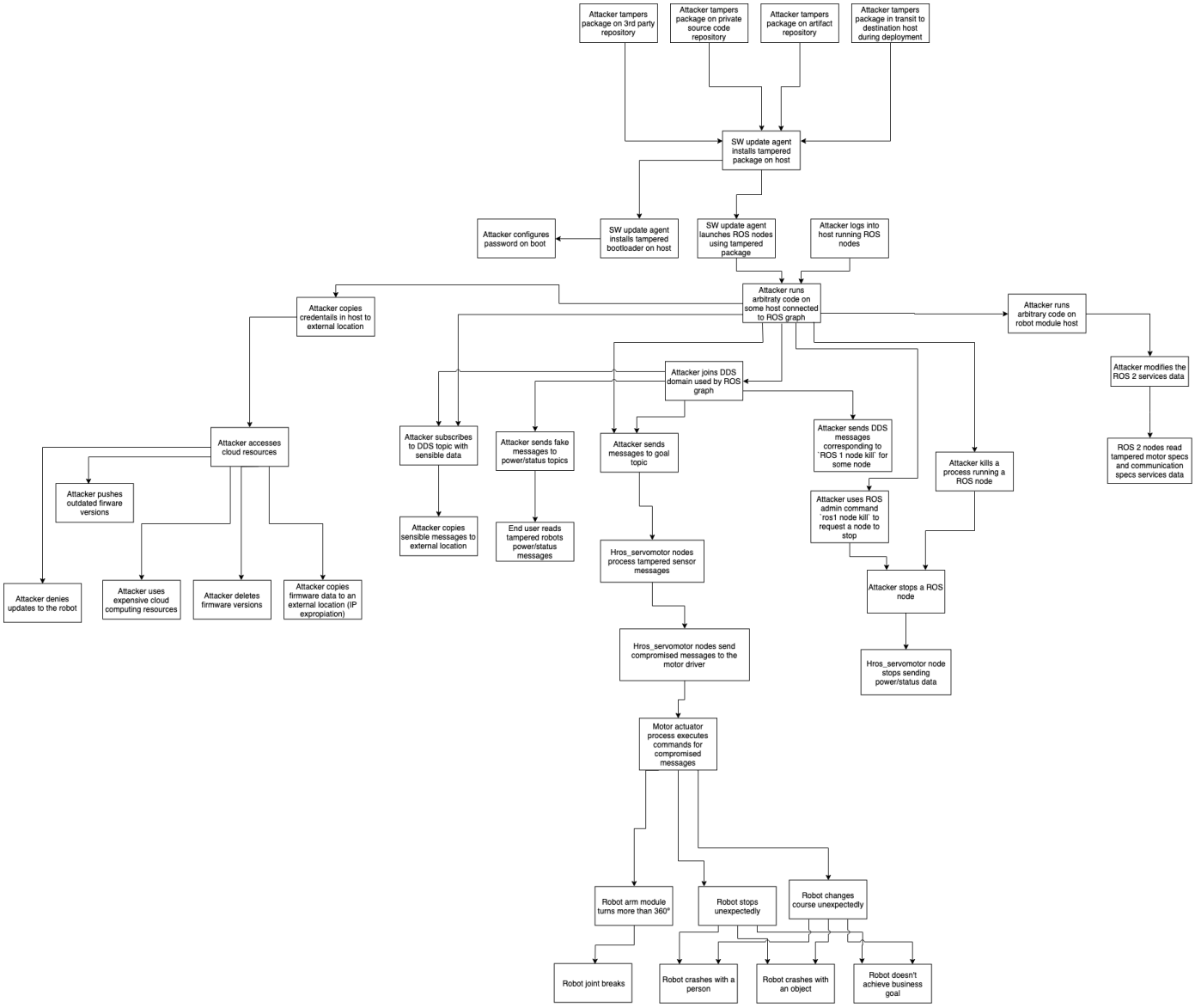


Figure 13 · attack_tree_repo.png. Original figure; a larger view appears in the companion atlas.

Diagram Source (draw.io)

The following diagram shows the infrastructure affected on a possible attack against the manufacturer code repository, showing its potential implications and consequences.

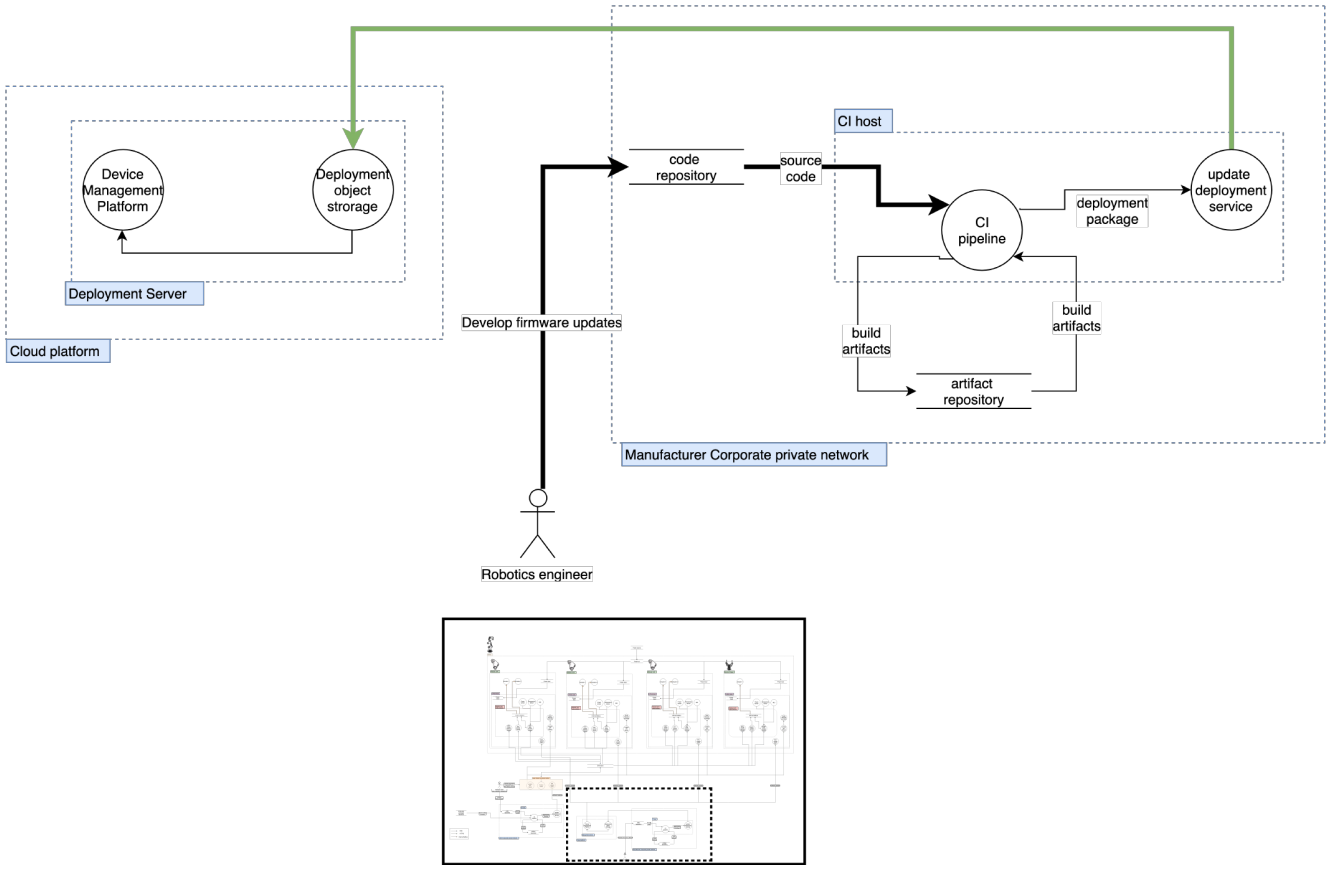


Figure 14 · attack_tree_repo_arch.png. Original figure; a larger view appears in the companion atlas.

Threat Model Validation Strategy

Validating this threat model end-to-end is a long-term effort meant to be distributed among the whole ROS 2 community. The threats list is made to evolve with this document and additional reference may be introduced in the future.

1. Setup a MARA with the exact software described in the Components MARA section of this document.
2. Penetration Testing * Attacks described in the spreadsheet should be implemented. For instance, a malicious `hros_actuator_servomotor_XXXXXXXXXXXX` could be implemented to try to disrupt the robot operations. * Once the vulnerability has been exploited, the exploit should be released to the community so that the results can be reproduced. * Whether the attack has been successful or not, this document should be updated accordingly. * If the attack was successful, a mitigation strategy should be implemented. It can either be done through improving ROS 2 core packages or it can be a platform-specific solution. In the second case, the mitigation will serve as an example to publish best practices for the development of secure robotic applications. * For trainning purposes, an online playground (RCTF) exists to challenge roboticists to learn and discover robot vulnerabilities. * For an overall evaluation of the robots' security measures, the Robot Security Framework (RSF) will be used. This validation has to be don e after the assessment is completed in order to have a realistic results.

Security Assessment preliminary results

Due to the iterative nature of this process, the results shown on this section may differ depending on the threat model's version.

The results here presented are a summary of the discoveries made during the Acutronic Robotics' MARA assessment. Prior to the assessment, in order to be time-effective, a threat analysis was done over the system and the most likely attack vectors were identified.

Introduction

Based on the threat model performed over MARA, an assessment has been performed in order to discover the vulnerabilities within. The content below is directly related to that threat model using all the attack vectors identified as starting points for the exercise.

This Security Assessment Report for Acutronic Robotics’ MARA components has been performed by Alias Robotics S.L. The targeted system for the assessment is the H-ROS powered MARA. H-ROS, the Hardware Robot Operating System, is a collection of hardware and software specifications and implementations that allows creating modular and distributed robot parts. H-ROS heavily relies on the use of ROS 2 as a robotic development framework.

This document presents an overview of the results obtained in the assessment. In it, we introduce a report focused on identifying security issues or vulnerabilities detected during the exercise.

Results

This assessment has evaluated the security measures present in the H-ROS SoM components. The evaluation has been done through a vulnerability and risk assessment, and the following sections are a summary of the findings.

The vulnerabilities are rated on a severity scale according to the guidelines stated in the Robot Vulnerability Scoring System (RVSS), resulting in a score between 0 and 10, according to the severity of the vulnerability. The RVSS follows the same principles as the Common Vulnerability Scoring System (CVSS), yet adding aspects related to robotics systems that are required to capture the complexity of robot vulnerabilities. Every evaluated aspect is defined in a vector, having its weight at the final score.

All assessment findings from the MARA are classified by technical severity. The following list explains how Alias Robotics rates vulnerabilities by their severity.

- **Critical**: Scores between 9 and 10
- **High**: Scores between 7 and 8.9
- **Medium**: Scores between 4 and 6.9
- **Low**: Scores between 0.1 and 3.9
- **Informational**: Scores 0.0

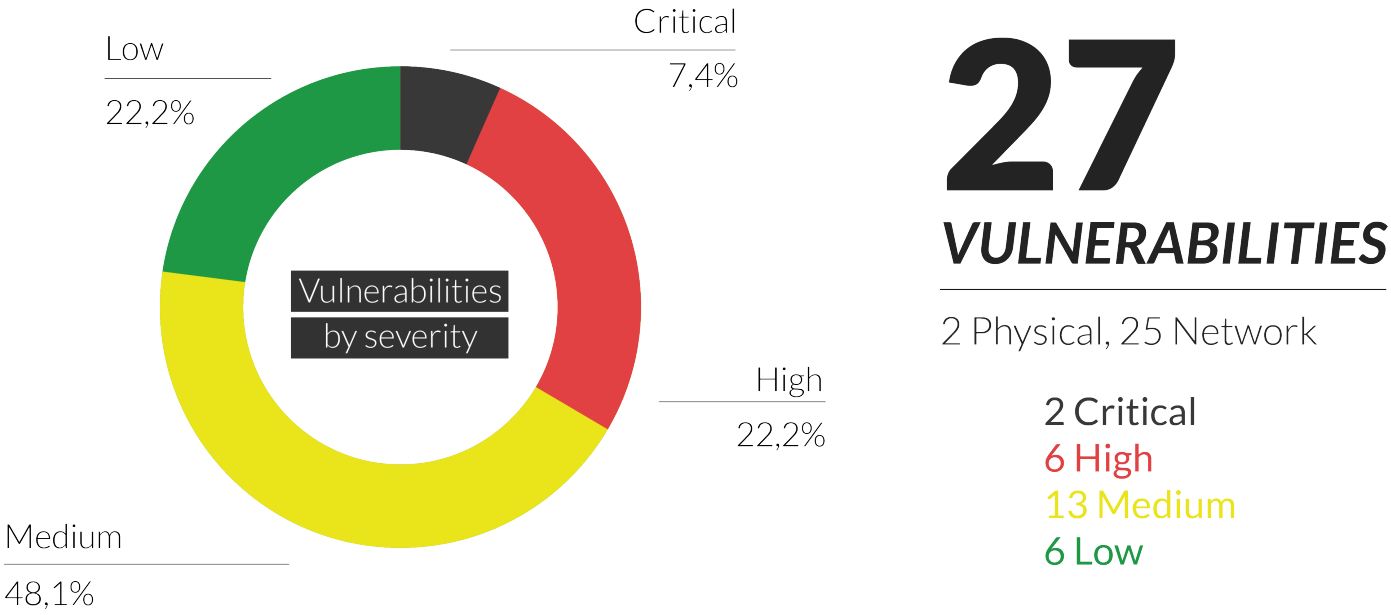


Figure 15 · vulns.png. Original figure; a larger view appears in the companion atlas.

In the course of a first funded vulnerability assessment, **2 critical**, **6 high**, **13 medium** and **6 low** vulnerabilities have been found. The ones fixed by Acutronic Robotics are disclosed below using the following format:

Name	Name of the vulnerability
ID	Vulnerability identifier
RVSS Score	Rating assigned according to the Robot Vulnerability Scoring System (RVSS), based on the severity of the vulnerability
Scoring Vector	Vector showing the different aspects that have defined the final score
Description	Information about the nature of the vulnerability
Remediation Status	Status of the remediation, with evidences and date (Active, Fixed)

Findings

Name	H-ROS API vulnerable to DoS attacks
ID	Vuln-07
RVSS Score	7.5
Scoring Vector	RVSS:1.0/AV:RN/AC:L/PR:N/UI:N/Y:Z/S:U/C:N/I:N/A:H/H:N
Description	

	The H-ROS API does not use any mechanism for limiting the requests that a user is able to perform in a determined set of time. This can lead to DoS attacks or premature wearing of the device.
Remediation Status	✓

Name	Insufficient limitation of hardware boundaries.
ID	Vuln-02
RVSS Score	6.8
Scoring Vector	RVSS:1.0/AV:IN/AC:L/PR:L/UI:N/Y:Z/S:U/C:N/I:L/A:H/H:E
Description	Actuator drivers do not correctly limit the movement boundaries, being able to execute commands above the physical capabilities of the actuator.
Remediation Status	✓

Name	ROS 2 Goal topic vulnerable to DoS attacks.
ID	Vuln-13
RVSS Score	5.5
Scoring Vector	RVSS:1.0/AV:IN/AC:L/PR:N/UI:N/Y:Z/S:U/C:N/I:N/A:H/H:U
Description	The ROS 2 nodes that control the motor fail when a big number of messages are sent in a small span of time. The application crashes and is not able to recover from failure, causing a DoS. This is probably caused by memory leaks, bugs or log storage.
Remediation Status	✓

Name	ROS 2 Trajectory topic vulnerable to DoS
ID	Vuln-14
RVSS Score	5.5
Scoring Vector	RVSS:1.0/AV:IN/AC:L/PR:N/UI:N/Y:Z/S:U/C:N/I:N/A:H/H:N
Description	

	Trajectory messages sent to the `/trajectory` topic of the H-ROS SoM cause an unrecoverable failure on the ROS node that can only be fixed by a system restart.
Remediation Status	✓

Name	OTA OpenSSH Linux distribution version disclosure
ID	Vuln-20
RVSS Score	5.3
Scoring Vector	RVSS:1.0/AV:RN/AC:L/PR:N/UI:N/Y:Z/S:U/C:L/I:N/A:N/H:N
Description	The OpenSSH server discloses the distribution name (Ubuntu) being used in the server in the connection headers, providing additional information to attackers.
Remediation Status	✓

Name	OTA OpenSSH version vulnerable to user enumeration attacks
ID	Vuln-21
RVSS Score	5.3
Scoring Vector	RVSS:1.0/AV:RN/AC:L/PR:N/UI:N/Y:Z/S:U/C:L/I:N/A:N/H:N
Description	TThe OpenSSH server version 7.6p1 is vulnerable to user enumeration attacks by timing.
Remediation Status	✓

References

1. Abera, Tigist, N. Asokan, Lucas Davi, Jan-Erik Ekberg, Thomas Nyman, Andrew Paverd, Ahmad-Reza Sadeghi, and Gene Tsudik. “C-FLAT: Control-FLow ATtestation for Embedded Systems Software.” ArXiv:1605.07763 [Cs], May 25, 2016. http://arxiv.org/abs/1605.07763.
2. Ahmad Yousef, Khalil, Anas AlMajali, Salah Ghalyon, Waleed Dweik, and Bassam Mohd. “Analyzing Cyber-Physical Threats on Robotic Platforms.” Sensors 18, no. 5 (May 21, 2018): 1643. https://doi.org/10.3390/s18051643.
3. Bonaci, Tamara, Jeffrey Herron, Tariq Yusuf, Junjie Yan, Tadayoshi Kohno, and Howard Jay Chizeck. “To Make a Robot Secure: An Experimental Analysis of Cyber Security Threats Against Teleoperated Surgical Robots.” ArXiv:1504.04339 [Cs], April 16, 2015. http://arxiv.org/abs/1504.04339.

4. Checkoway, Stephen, Damon McCoy, Brian Kantor, Danny Anderson, Hovav Shacham, Stefan Savage, Karl Koscher, Alexei Czeskis, Franziska Roesner, and Tadayoshi Kohno. “Comprehensive Experimental Analyses of Automotive Attack Surfaces.” In Proceedings of the 20th USENIX Conference on Security, 6–6. SEC’11. Berkeley, CA, USA: USENIX Association, 2011. http://dl.acm.org/citation.cfm?id=2028067.2028073.
5. Clark, George W., Michael V. Doran, and Todd R. Andel. “Cybersecurity Issues in Robotics.” In 2017 IEEE Conference on Cognitive and Computational Aspects of Situation Management (CogSIMA), 1–5. Savannah, GA, USA: IEEE, 2017. https://doi.org/10.1109/COGSIMA.2017.7929597.
6. Denning, Tamara, Cynthia Matuszek, Karl Koscher, Joshua R. Smith, and Tadayoshi Kohno. “A Spotlight on Security and Privacy Risks with Future Household Robots: Attacks and Lessons.” In Proceedings of the 11th International Conference on Ubiquitous Computing - Ubicomp ’09, 105. Orlando, Florida, USA: ACM Press, 2009. https://doi.org/10.1145/1620545.1620564.
7. Dessiatnikoff, Anthony, Yves Deswarte, Eric Alata, and Vincent Nicomette. “Potential Attacks on Onboard Aerospace Systems.” IEEE Security & Privacy 10, no. 4 (July 2012): 71–74. https://doi.org/10.1109/MSP.2012.104.
8. Dzung, D., M. Naedele, T.P. Von Hoff, and M. Crevatin. “Security for Industrial Communication Systems.” Proceedings of the IEEE 93, no. 6 (June 2005): 1152–77. https://doi.org/10.1109/JPROC.2005.849714.
9. Elmiligi, Haytham, Fayez Gebali, and M. Watheq El-Kharashi. “Multi-Dimensional Analysis of Embedded Systems Security.” Microprocessors and Microsystems 41 (March 2016): 29–36. https://doi.org/10.1016/j.micpro.2015.12.005.
10. Groza, Bogdan, and Toma-Leonida Dragomir. “Using a Cryptographic Authentication Protocol for the Secure Control of a Robot over TCP/IP.” In 2008 IEEE International Conference on Automation, Quality and Testing, Robotics, 184–89. Cluj-Napoca, Romania: IEEE, 2008.https://doi.org/10.1109/AQTR.2008.4588731.
11. Javaid, Ahmad Y., Weiqing Sun, Vijay K. Devabhaktuni, and Mansoor Alam. “Cyber Security Threat Analysis and Modeling of an Unmanned Aerial Vehicle System.” In 2012 IEEE Conference on Technologies for Homeland Security (HST), 585–90. Waltham, MA, USA: IEEE, 2012.https://doi.org/10.1109/THS.2012.6459914.
12. Kleidermacher, David, and Mike Kleidermacher. Embedded Systems Security: Practical Methods for Safe and Secure Software and Systems Development. Amsterdam: Elsevier/Newnes, 2012.
13. Klein, Gerwin, June Andronick, Matthew Fernandez, Ihor Kuz, Toby Murray, and Gernot Heiser. “Formally Verified Software in the Real World.” Communications of the ACM 61, no. 10 (September 26, 2018): 68–77.https://doi.org/10.1145/3230627.
14. Lee, Gregory S., and Bhavani Thuraisingham. “Cyberphysical Systems Security Applied to Telesurgical Robotics.” Computer Standards & Interfaces 34, no. 1 (January 2012): 225–29. https://doi.org/10.1016/j.csi.2011.09.001.
15. Lera, Francisco J. Rodríguez, Camino Fernández Llamas, Ángel Manuel Guerrero, and Vicente Matellán Olivera. “Cybersecurity of Robotics and Autonomous Systems: Privacy and Safety.” In Robotics - Legal, Ethical and Socioeconomic Impacts, edited by George Dekoulis. InTech, 2017.https://doi.org/10.5772/intechopen.69796.
16. McClean, Jarrod, Christopher Stull, Charles Farrar, and David Mascareñas. “A Preliminary Cyber-Physical Security Assessment of the Robot Operating System (ROS).” edited by Robert E. Karlsen, Douglas W. Gage, Charles M. Shoemaker, and Grant R. Gerhart, 874110. Baltimore, Maryland, USA, 2013.https://doi.org/10.1117/12.2016189.
17. Morante, Santiago, Juan G. Victores, and Carlos Balaguer. “Cryptobotics: Why Robots Need Cyber Safety.” Frontiers in Robotics and AI 2 (September 29, 2015). https://doi.org/10.3389/frobt.2015.00023.

18. Papp, Dorottya, Zhendong Ma, and Levente Buttyan. “Embedded Systems Security: Threats, Vulnerabilities, and Attack Taxonomy.” In 2015 13th Annual Conference on Privacy, Security and Trust (PST), 145–52. Izmir, Turkey: IEEE, 2015. https://doi.org/10.1109/PST.2015.7232966.
19. Pike, Lee, Pat Hickey, Trevor Elliott, Eric Mertens, and Aaron Tomb. “TrackOS: A Security-Aware Real-Time Operating System.” In Runtime Verification, edited by Yliès Falcone and César Sánchez, 10012:302–17. Cham: Springer International Publishing, 2016.https://doi.org/10.1007/978-3-319-46982-9_19.
20. Ravi, Srivaths, Paul Kocher, Ruby Lee, Gary McGraw, and Anand Raghunathan. “Security as a New Dimension in Embedded System Design.” In Proceedings of the 41st Annual Conference on Design Automation - DAC ’04, 753. San Diego, CA, USA: ACM Press, 2004. https://doi.org/10.1145/996566.996771.
21. Serpanos, Dimitrios N., and Artemios G. Voyiatzis. “Security Challenges in Embedded Systems.” ACM Transactions on Embedded Computing Systems 12, no. 1s (March 29, 2013): 1–10. https://doi.org/10.1145/2435227.2435262.
22. Vilches, Víctor Mayoral, Laura Alzola Kirschgens, Asier Bilbao Calvo, Alejandro Hernández Cordero, Rodrigo Izquierdo Pisón, David Mayoral Vilches, Aday Muñiz Rosas, et al. “Introducing the Robot Security Framework (RSF), a Standardized Methodology to Perform Security Assessments in Robotics.” ArXiv:1806.04042 [Cs], June 11, 2018. http://arxiv.org/abs/1806.04042.
23. Zubairi, Junaid Ahmed, and Athar Mahboob, eds. Cyber Security Standards, Practices and Industrial Applications: Systems and Methodologies. IGI Global, 2012. https://doi.org/10.4018/978-1-60960-851-4.
24. Akhtar, Naveed, and Ajmal Mian. “Threat of Adversarial Attacks on Deep Learning in Computer Vision: A Survey.” ArXiv:1801.00553 [Cs], January 2, 2018. http://arxiv.org/abs/1801.00553.
25. V. Mayoral Vilches, E. Gil-Uriarte, I. Zamalloa Ugarte, G. Olalde Mendia, R. Izquierdo Pisón, L. Alzola Kirschgens, A. Bilbao Calvo, A. Hernández Cordero, L. Apa, and C. Cerrudo, “Towards an open standard for assessing the severity of robot security vulnerabilities, the Robot Vulnerability Scoring System (RVSS),” ArXiv:807.10357 [Cs], Jul. 2018. https://arxiv.org/abs/1807.10357
26. G. Olalde Mendia, L. Usategui San Juan, X. Perez Bascaran, A. Bilbao Calvo, A. Hernández Cordero, I. Zamalloa Ugarte, A. Muñiz Rosas, D. Mayoral Vilches, U. Ayucar Carbajo, L. Alzola Kirschgens, V. Mayoral Vilches, and E. Gil-Uriarte, “Robotics CTF (RCTF), a playground for robot hacking,” ArXiv:810.02690 [Cs], Oct. 2018. https://arxiv.org/abs/1810.02690
27. L. Alzola Kirschgens, I. Zamalloa Ugarte, E. Gil Uriarte, A. Muñiz Rosas, and V. Mayoral Vilches, “Robot hazards: from safety to security,” ArXiv:1806.06681 [Cs], Jun. 2018. https://arxiv.org/abs/1806.06681
28. Víctor Mayoral Vilches, Gorka Olalde Mendia, Xabier Perez Baskaran, Alejandro Hernández Cordero, Lander Usategui San Juan, Endika Gil-Uriarte, Odei Olalde Saez de Urabain, Laura Alzola Kirschgens, "Aztarna, a footprinting tool for robots," ArXiv:1806.06681 [Cs], December 22, 2018. https://arxiv.org/abs/1812.09490

Supporting security passages

Historical excerpts from the same source revision; retained for context. Website OAuth code is inventoried in the report rather than included as robot-security instruction.

ROS Command Line Arguments

Michel Hidalgo

Original lines 250–268

Enclave assignments

Enclave assignment may be achieved using the `--enclave`/`-e` option. This option takes a single string `value` assignment statement, where `value` is a fully qualified enclave path used to locate the respective security artifacts within the configured keystore.

As an example, to assign an enclave path `/foo/bar` one may execute:

```
ros2 run some_package some_ros_executable --ros-args --enclave="/foo/bar"
```

or its shorter equivalent:

```
ros2 run some_package some_ros_executable --ros-args -e "/foo/bar"
```

As is, this enclave assignment applies to each and every Domain Participant that `some_ros_executable` spawns unless explicitly ignored in code or overridden via security environment variables.

Node to Participant mapping

Ivan Paunovic

Original lines 127–132

Security

Previously, each Node could have different security artifacts. That was possible because each Node was mapped to one Participant. The new approach allows to specify different security artifacts for each process. For more details, see ROS 2 Security Enclaves.

Node to Participant mapping

Ivan Paunovic

Original lines 147–161

Keyed topics could be used, with the Participant gid as the key. That would allow the Reader side of the topic to use a keep last, depth 1 history.

Using Participant/Writer/Reader userData QoS

Instead of using a custom topic to share the extra ROS specific discovery information, a combination of Participant/Reader/Writer `userData` could be used:

- Participant `userData`: list of Nodes created by the Participant (updated when Node created destroyed).
- Reader user `userData`: Node name/namespace.
- Writer user `userData`: Node name/namespace.

That information would be enough to satisfy ROS required graph API. This mechanism would also preclude having a topic that has to be read and written by all Nodes, which is better from a security perspective.

This alternative wasn't implemented because of lack of support in some of the currently supported DDS vendors.

Actions

Geoffrey Biggs Jacob Perron Shane Loretz

Original lines 423–428

Reasons against using separate topics for feedback and status: - Most anticipated use cases will not involve many goals/clients (premature optimization) - Topics dynamically namespaced (e.g. by goal ID) would complicate ROS security by not having deterministic topic names before runtime and outside user control. - Added complexity in C implementation and client libraries

ROS on DDS

William Woodall

Original lines 84–84

On the contrary, DDS seems to have an active and organic specification, which in the recent past has added, or is adding, more specifications for things like websockets, security over SSL, extensible types, request and response transport, and a new, more modern C++11 style API specification for the core API to replace the existing C++ interface.

Unique Network Flows

Ananya Muddukrishna, Ericsson AB

Original lines 58–58

- 5G network 5QI: The Network Exposure Function (NEF) [4] in the 5G core network provides robust and secure API for QoS specification. This API enables applications to programmatically (HTTP-JSON) specify required QoS by associating 5G QoS Identifiers (5QIs) to flow identifers, as shown in the figure next.

Sources and license notices

Source: ros2/design, revision 93a415bf928751d37fce6f83215c521658e20c93. Reviewed 2026-10-20. Article author bylines and original dates appear at the beginning of each article.

Original attribution: © Open Source Robotics Foundation, Inc. The source site footer states: “Except where otherwise noted, these design documents are licensed under Creative Commons Attribution 3.0.” CC BY 3.0.

Adaptation credit: Juan Carlos Nevarez · Gh0stlyKn1ght. Changes: extraction into a standalone reader; resolved templates/schema include; source links pinned; wide matrices expanded into vertical records with inherited rowspan values; images embedded and numbered; print pagination added. Original factual errors and historical mitigation claims remain present for comparison.

Repository LICENSE (verbatim)


```
                                 Apache License
                           Version 2.0, January 2004
                        http://www.apache.org/licenses/

   TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

   1. Definitions.

      "License" shall mean the terms and conditions for use, reproduction,
      and distribution as defined by Sections 1 through 9 of this document.

      "Licensor" shall mean the copyright owner or entity authorized by
      the copyright owner that is granting the License.

      "Legal Entity" shall mean the union of the acting entity and all
      other entities that control, are controlled by, or are under common
      control with that entity. For the purposes of this definition,
      "control" means (i) the power, direct or indirect, to cause the
      direction or management of such entity, whether by contract or
      otherwise, or (ii) ownership of fifty percent (50%) or more of the
      outstanding shares, or (iii) beneficial ownership of such entity.

      "You" (or "Your") shall mean an individual or Legal Entity
      exercising permissions granted by this License.

      "Source" form shall mean the preferred form for making modifications,
      including but not limited to software source code, documentation
      source, and configuration files.

      "Object" form shall mean any form resulting from mechanical
      transformation or translation of a Source form, including but
      not limited to compiled object code, generated documentation,
      and conversions to other media types.

      "Work" shall mean the work of authorship, whether in Source or
      Object form, made available under the License, as indicated by a
      copyright notice that is included in or attached to the work
      (an example is provided in the Appendix below).

      "Derivative Works" shall mean any work, whether in Source or Object
      form, that is based on (or derived from) the Work and for which the
      editorial revisions, annotations, elaborations, or other modifications
      represent, as a whole, an original work of authorship. For the purposes
      of this License, Derivative Works shall not include works that remain
      separable from, or merely link (or bind by name) to the interfaces of,
      the Work and Derivative Works thereof.
```

```
      "Contribution" shall mean any work of authorship, including
      the original version of the Work and any modifications or additions
      to that Work or Derivative Works thereof, that is intentionally
      submitted to Licensor for inclusion in the Work by the copyright owner
      or by an individual or Legal Entity authorized to submit on behalf of
      the copyright owner. For the purposes of this definition, "submitted"
      means any form of electronic, verbal, or written communication sent
      to the Licensor or its representatives, including but not limited to
      communication on electronic mailing lists, source code control systems,
      and issue tracking systems that are managed by, or on behalf of, the
      Licensor for the purpose of discussing and improving the Work, but
      excluding communication that is conspicuously marked or otherwise
      designated in writing by the copyright owner as "Not a Contribution."

      "Contributor" shall mean Licensor and any individual or Legal Entity
      on behalf of whom a Contribution has been received by Licensor and
      subsequently incorporated within the Work.

   2. Grant of Copyright License. Subject to the terms and conditions of
      this License, each Contributor hereby grants to You a perpetual,
      worldwide, non-exclusive, no-charge, royalty-free, irrevocable
      copyright license to reproduce, prepare Derivative Works of,
      publicly display, publicly perform, sublicense, and distribute the
      Work and such Derivative Works in Source or Object form.

   3. Grant of Patent License. Subject to the terms and conditions of
      this License, each Contributor hereby grants to You a perpetual,
      worldwide, non-exclusive, no-charge, royalty-free, irrevocable
      (except as stated in this section) patent license to make, have made,
      use, offer to sell, sell, import, and otherwise transfer the Work,
      where such license applies only to those patent claims licensable
      by such Contributor that are necessarily infringed by their
      Contribution(s) alone or by combination of their Contribution(s)
      with the Work to which such Contribution(s) was submitted. If You
      institute patent litigation against any entity (including a
      cross-claim or counterclaim in a lawsuit) alleging that the Work
      or a Contribution incorporated within the Work constitutes direct
      or contributory patent infringement, then any patent licenses
      granted to You under this License for that Work shall terminate
      as of the date such litigation is filed.

   4. Redistribution. You may reproduce and distribute copies of the
      Work or Derivative Works thereof in any medium, with or without
      modifications, and in Source or Object form, provided that You
      meet the following conditions:

      (a) You must give any other recipients of the Work or
          Derivative Works a copy of this License; and

      (b) You must cause any modified files to carry prominent notices
          stating that You changed the files; and

      (c) You must retain, in the Source form of any Derivative Works
          that You distribute, all copyright, patent, trademark, and
          attribution notices from the Source form of the Work,
          excluding those notices that do not pertain to any part of
          the Derivative Works; and

      (d) If the Work includes a "NOTICE" text file as part of its
          distribution, then any Derivative Works that You distribute must
          include a readable copy of the attribution notices contained
          within such NOTICE file, excluding those notices that do not
          pertain to any part of the Derivative Works, in at least one
          of the following places: within a NOTICE text file distributed
          as part of the Derivative Works; within the Source form or
          documentation, if provided along with the Derivative Works; or,
          within a display generated by the Derivative Works, if and
          wherever such third-party notices normally appear. The contents
          of the NOTICE file are for informational purposes only and
          do not modify the License. You may add Your own attribution
          notices within Derivative Works that You distribute, alongside
          or as an addendum to the NOTICE text from the Work, provided
          that such additional attribution notices cannot be construed
          as modifying the License.
```

```
   You may add Your own copyright statement to Your modifications and
   may provide additional or different license terms and conditions
   for use, reproduction, or distribution of Your modifications, or
   for any such Derivative Works as a whole, provided Your use,
   reproduction, and distribution of the Work otherwise complies with
   the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise,
   any Contribution intentionally submitted for inclusion in the Work
   by You to the Licensor shall be under the terms and conditions of
   this License, without any additional terms or conditions.
   Notwithstanding the above, nothing herein shall supersede or modify
   the terms of any separate license agreement you may have executed
   with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade
   names, trademarks, service marks, or product names of the Licensor,
   except as required for reasonable and customary use in describing the
   origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or
   agreed to in writing, Licensor provides the Work (and each
   Contributor provides its Contributions) on an "AS IS" BASIS,
   WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
   implied, including, without limitation, any warranties or conditions
   of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
   PARTICULAR PURPOSE. You are solely responsible for determining the
   appropriateness of using or redistributing the Work and assume any
   risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory,
   whether in tort (including negligence), contract, or otherwise,
   unless required by applicable law (such as deliberate and grossly
   negligent acts) or agreed to in writing, shall any Contributor be
   liable to You for damages, including any direct, indirect, special,
   incidental, or consequential damages of any character arising as a
   result of this License or out of the use or inability to use the
   Work (including but not limited to damages for loss of goodwill,
   work stoppage, computer failure or malfunction, or any and all
   other commercial damages or losses), even if such Contributor
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing
   the Work or Derivative Works thereof, You may choose to offer,
   and charge a fee for, acceptance of support, warranty, indemnity,
   or other liability obligations and/or rights consistent with this
   License. However, in accepting such obligations, You may act only
   on Your own behalf and on Your sole responsibility, not on behalf
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by reason
   of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

   To apply the Apache License to your work, attach the following
   boilerplate notice, with the fields enclosed by brackets "[]"
   replaced with your own identifying information. (Don't include
   the brackets!)  The text should be enclosed in the appropriate
   comment syntax for the file format. We also recommend that a
   file or class name and description of purpose be included on the
   same "printed page" as the copyright notice for easier
   identification within third-party archives.

Copyright 2014 Open Source Robotics Foundation, Inc.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0
```

```
   Unless required by applicable law or agreed to in writing, software
   distributed under the License is distributed on an "AS IS" BASIS,
   WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
   See the License for the specific language governing permissions and
   limitations under the License.
```